COIMBATORE INSTITUTE OF TECHNOLOGY

(Government Aided Autonomous Institution Approved by AICTE, New Delhi & Affiliated to Anna University, Chennai)

Department of Computing

Internet of Things Laboratory

(20MSS84)

NAME: SHREE KOTTES J

REG.NO: 7176 22 31 050

CLASS: M.Sc., SOFTWARE SYSTEMS

(8th Semester)

MAIL ID: 71762231050@cit.edu.in

INDEX

S. No	Date	Title	Sign
1.	26/12/2025	Examination Alarm System	
2.	26/12/2025	Traffic Control System	
3.	04/01/2026	Odometer	
4.	30/01/2026	Music Visualizer	
5.	30/01/2026	Motion Detection and Photo Resistor	
6.	06/02/2026	Gas Leakage Detection	
7.	20/02/2026	MQTT Client Server Communication	
8.	27/02/2026	MQTT Arithmetic Operations	
9.	27/02/2026	Smart Temperature and Humidity Monitoring	
10.	20/03/2026	Smart Four Wheeler Vehicle Parking	
11.	20/03/2026	Electronic Road Passing System	
12.	20/03/2026	Smart Gas Leakage Detection System	
13.	20/03/2026	Bank Locker Access System	
14.	20/03/2026	Tic-Tac-Toe	

EX.NO: 1	EXAMINATION ALARM SYSTEM
26/12/2025	

AIM

To build an exam alarm system that will be able to beep every hour, half an hour, or with custom time interval.

APPARATUS REQUIRED

- Arduino Uno
- Breadboard
- RTC DS1307 module
- I2C LCD Display (16x2)
- Pushbuttons (x4) (Mode, Up, Down, Set)
- Buzzer
- Jumper wires

PRE-CONDITIONS

- Hardware components are wired correctly, ensuring the RTC module and I2C LCD are securely connected to the Arduino I2C pins (A4, A5).
- Required libraries (`Wire.h`, `RTClib.h`, `LiquidCrystal_I2C.h`) are installed in the Arduino IDE.
- The RTC DS1307 module is properly configured with the current date and time.
- The code is successfully compiled and uploaded to the Arduino Uno board.

POST-CONDITIONS

- The LCD correctly displays the current time, elapsed/remaining time, and configuration menus.
- The user can configure alarm intervals (half-hour, one-hour, custom, or random) using the pushbuttons.
- The buzzer sounds alerts automatically at the configured intervals while the exam is running.
- The exam process starts and stops gracefully upon interacting with the Set button.

CODE

```
#include <Wire.h>
#include <RTClib.h>
#include <LiquidCrystal_I2C.h>

// Initialize LCD (address 0x27, 16 columns, 2 rows)
LiquidCrystal_I2C lcd(0x27, 16, 2);
```

```
RTC_DS1307 rtc;

// Pin definitions
#define BUZZER_PIN 9
#define BTN_MODE 2
#define BTN_UP 3
#define BTN_DOWN 4
#define BTN_SET 5

// Configuration variables
enum AlarmPattern { HALF_HOUR, ONE_HOUR, CUSTOM, RANDOM };
AlarmPattern alarmPattern = HALF_HOUR;

enum DisplayMode { ELAPSED, REMAINING };
DisplayMode displayMode = ELAPSED;

enum SoundType { SINGLE_BEEP, DOUBLE_BEEP, LONG_BEEP };
SoundType soundType = SINGLE_BEEP;

// Timing variables
unsigned long examStartTime = 0;
unsigned long examDuration = 3600000; // Default 1 hour in milliseconds
unsigned long lastAlarmTime = 0;
unsigned long nextAlarmTime = 0;
bool examStarted = false;
bool alarmEnabled = true;

// Custom interval (in minutes)
int customInterval = 20; // Default 20 minutes

// Random interval configuration (in minutes)
int randomMinInterval = 15; // Minimum 15 minutes
int randomMaxInterval = 60; // Maximum 60 minutes

// Menu system
enum MenuState { MAIN_SCREEN, MENU_ALARM_PATTERN, MENU_CUSTOM_INTERVAL,
                 MENU_RANDOM_MIN, MENU_RANDOM_MAX, MENU_DISPLAY_MODE,
                 MENU_SOUND_TYPE, MENU_EXAM_DURATION, MENU_START_EXAM };
MenuState menuState = MAIN_SCREEN;
int menuSelection = 0;

// Button debouncing
unsigned long lastButtonPress = 0;
const unsigned long debounceDelay = 200;

void setup() {
  Serial.begin(9600);
```

```
  // Initialize LCD
  lcd.init();
  lcd.backlight();
  lcd.clear();

  // Initialize RTC
  if (!rtc.begin()) {
    lcd.setCursor(0, 0);
    lcd.print("RTC Not Found!");
    while (1);
  }

  if (!rtc.isrunning()) {
    rtc.adjust(DateTime(F(__DATE__), F(__TIME__)));
  }

  // Initialize pins
  pinMode(BUZZER_PIN, OUTPUT);
  pinMode(BTN_MODE, INPUT_PULLUP);
  pinMode(BTN_UP, INPUT_PULLUP);
  pinMode(BTN_DOWN, INPUT_PULLUP);
  pinMode(BTN_SET, INPUT_PULLUP);

  digitalWrite(BUZZER_PIN, LOW);

  lcd.setCursor(0, 0);
  lcd.print("Exam Alarm Sys");
  lcd.setCursor(0, 1);
  lcd.print("Ready to Config");
  delay(2000);
}

void loop() {
  handleButtons();

  if (menuState == MAIN_SCREEN) {
    if (examStarted) {
      displayExamStatus();
      checkAlarm();
    } else {
      displayMainScreen();
    }
  } else {
    displayMenu();
  }

  delay(100);
}
```

```
void handleButtons() {
  unsigned long currentTime = millis();

  if (currentTime - lastButtonPress < debounceDelay) {
    return;
  }

  if (digitalRead(BTN_MODE) == LOW) {
    lastButtonPress = currentTime;
    handleModeButton();
  } else if (digitalRead(BTN_UP) == LOW) {
    lastButtonPress = currentTime;
    handleUpButton();
  } else if (digitalRead(BTN_DOWN) == LOW) {
    lastButtonPress = currentTime;
    handleDownButton();
  } else if (digitalRead(BTN_SET) == LOW) {
    lastButtonPress = currentTime;
    handleSetButton();
  }
}

void handleModeButton() {
  if (examStarted) return; // Don't allow menu during exam

  if (menuState == MAIN_SCREEN) {
    menuState = MENU_ALARM_PATTERN;
    menuSelection = 0;
  } else {
    // Navigate to next menu
    int nextMenu = (int)menuState + 1;

    // Skip menus based on alarm pattern selection
    if (nextMenu == MENU_CUSTOM_INTERVAL && alarmPattern != CUSTOM) {
      nextMenu++; // Skip custom interval if not selected
    }
    if (nextMenu == MENU_RANDOM_MIN && alarmPattern != RANDOM) {
      nextMenu += 2; // Skip both random menus if not selected
    } else if (nextMenu == MENU_RANDOM_MAX && alarmPattern != RANDOM) {
      nextMenu++; // Skip random max if not selected
    }

    if (nextMenu > MENU_START_EXAM) {
      nextMenu = MENU_ALARM_PATTERN;
      menuSelection = 0;
    }
```

```
    menuState = (MenuState)nextMenu;
  }
  lcd.clear();
}

void handleUpButton() {
  if (menuState == MAIN_SCREEN) return;

  switch (menuState) {
    case MENU_ALARM_PATTERN:
      alarmPattern = (AlarmPattern)((alarmPattern + 1) % 4);
      break;

    case MENU_CUSTOM_INTERVAL:
      customInterval += 1; // Increment by 5 minutes
      if (customInterval > 180) customInterval = 180; // Max 3 hours
      break;

    case MENU_RANDOM_MIN:
      randomMinInterval += 5; // Increment by 5 minutes
      if (randomMinInterval > 120) randomMinInterval = 120; // Max 2
       ↪  hours
      if (randomMinInterval >= randomMaxInterval) {
        randomMaxInterval = randomMinInterval + 5;
      }
      break;

    case MENU_RANDOM_MAX:
      randomMaxInterval += 5; // Increment by 5 minutes
      if (randomMaxInterval > 180) randomMaxInterval = 180; // Max 3
       ↪  hours
      break;

    case MENU_DISPLAY_MODE:
      displayMode = (DisplayMode)((displayMode + 1) % 2);
      break;

    case MENU_SOUND_TYPE:
      soundType = (SoundType)((soundType + 1) % 3);
      break;

    case MENU_EXAM_DURATION:
      examDuration += 300000; // Add 5 minutes
      if (examDuration > 14400000) examDuration = 14400000; // Max 4
       ↪  hours
      break;
  }
}
```

```
void handleDownButton() {
  if (menuState == MAIN_SCREEN) return;

  switch (menuState) {
    case MENU_ALARM_PATTERN:
      alarmPattern = (AlarmPattern)((alarmPattern - 1 + 4) % 4);
      break;

    case MENU_CUSTOM_INTERVAL:
      if (customInterval > 0) {
        customInterval -= 1; // Decrement by 5 minutes
      }
      break;

    case MENU_RANDOM_MIN:
      if (randomMinInterval > 5) {
        randomMinInterval -= 5; // Decrement by 5 minutes
      }
      break;

    case MENU_RANDOM_MAX:
      randomMaxInterval -= 5; // Decrement by 5 minutes
      if (randomMaxInterval < randomMinInterval + 5) {
        randomMaxInterval = randomMinInterval + 5;
      }
      if (randomMaxInterval < 10) randomMaxInterval = 10;
      break;

    case MENU_DISPLAY_MODE:
      displayMode = (DisplayMode)((displayMode - 1 + 2) % 2);
      break;

    case MENU_SOUND_TYPE:
      soundType = (SoundType)((soundType - 1 + 3) % 3);
      break;

    case MENU_EXAM_DURATION:
      if (examDuration > 300000) {
        examDuration -= 300000; // Subtract 5 minutes
      }
      break;
  }
}

void handleSetButton() {
  if (!examStarted) {
    startExam();
```

```
    menuState = MAIN_SCREEN;
  } else if (examStarted) {
    stopExam();
  }
  lcd.clear();
}

void displayMainScreen() {
  DateTime now = rtc.now();

  lcd.setCursor(0, 0);
  lcd.print("Time: ");
  printTime(now.hour(), now.minute(), now.second());

  lcd.setCursor(0, 1);
  lcd.print("Press MODE menu");
}

void displayMenu() {
  lcd.setCursor(0, 0);

  switch (menuState) {
    case MENU_ALARM_PATTERN:
      lcd.print("Alarm Pattern: ");
      lcd.setCursor(0, 1);
      if (alarmPattern == HALF_HOUR) lcd.print("Every Half Hour");
      else if (alarmPattern == ONE_HOUR) lcd.print("Every One Hour ");
      else if (alarmPattern == CUSTOM) lcd.print("Custom Interval");
      else lcd.print("Random Interval");
      break;

    case MENU_CUSTOM_INTERVAL:
      lcd.print("Custom Interval:");
      lcd.setCursor(0, 1);
      lcd.print(customInterval);
      lcd.print(" Minutes      ");
      break;

    case MENU_RANDOM_MIN:
      lcd.print("Random Min Int:");
      lcd.setCursor(0, 1);
      lcd.print(randomMinInterval);
      lcd.print(" Minutes      ");
      break;

    case MENU_RANDOM_MAX:
      lcd.print("Random Max Int:");
      lcd.setCursor(0, 1);
```

```
      lcd.print(randomMaxInterval);
      lcd.print(" Minutes      ");
      break;

    case MENU_DISPLAY_MODE:
      lcd.print("Display Mode:  ");
      lcd.setCursor(0, 1);
      if (displayMode == ELAPSED) lcd.print("Elapsed Time   ");
      else lcd.print("Remaining Time ");
      break;

    case MENU_SOUND_TYPE:
      lcd.print("Sound Type:    ");
      lcd.setCursor(0, 1);
      if (soundType == SINGLE_BEEP) lcd.print("Single Beep    ");
      else if (soundType == DOUBLE_BEEP) lcd.print("Double Beep    ");
      else lcd.print("Long Beep      ");
      break;

    case MENU_EXAM_DURATION:
      lcd.print("Exam Duration: ");
      lcd.setCursor(0, 1);
      unsigned long hours = examDuration / 3600000;
      unsigned long mins = (examDuration % 3600000) / 60000;
      lcd.print(hours);
      lcd.print(" Hrs ");
      lcd.print(mins);
      lcd.print(" Mins  ");
      break;
  }
}

void displayExamStatus() {
  unsigned long currentTime = millis();
  unsigned long elapsed = currentTime - examStartTime;

  lcd.setCursor(0, 0);

  if (displayMode == ELAPSED) {
    lcd.print("Elapsed: ");
    printDuration(elapsed);
  } else {
    lcd.print("Remain:  ");
    if (elapsed < examDuration) {
      printDuration(examDuration - elapsed);
    } else {
      lcd.print("00:00   ");
    }
```

```
  }

  lcd.setCursor(0, 1);
  DateTime now = rtc.now();
  lcd.print("Time: ");
  printTime(now.hour(), now.minute(), now.second());

  // Check if exam is over
  if (elapsed >= examDuration) {
    examFinished();
  }
}

void printTime(int h, int m, int s) {
  if (h < 10) lcd.print("0");
  lcd.print(h);
  lcd.print(":");
  if (m < 10) lcd.print("0");
  lcd.print(m);
  lcd.print(":");
  if (s < 10) lcd.print("0");
  lcd.print(s);
}

void printDuration(unsigned long duration) {
  unsigned long hours = duration / 3600000;
  unsigned long mins = (duration % 3600000) / 60000;

  if (hours < 10) lcd.print("0");
  lcd.print(hours);
  lcd.print(":");
  if (mins < 10) lcd.print("0");
  lcd.print(mins);
  lcd.print("   ");
}

void startExam() {
  examStarted = true;
  examStartTime = millis();
  lastAlarmTime = examStartTime;

  // Set next alarm based on pattern
  if (alarmPattern == HALF_HOUR) {
    nextAlarmTime = examStartTime + 1800000; // 30 minutes
  } else if (alarmPattern == ONE_HOUR) {
    nextAlarmTime = examStartTime + 3600000; // 60 minutes
  } else if (alarmPattern == CUSTOM) {
    nextAlarmTime = examStartTime + (customInterval * 60000UL); // Custom
 ↪  minutes
```

```
  } else { // RANDOM
    unsigned long minMs = randomMinInterval * 60000UL;
    unsigned long maxMs = randomMaxInterval * 60000UL;
    nextAlarmTime = examStartTime + random(minMs, maxMs);
  }

  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Exam Started!");
  lcd.setCursor(0, 1);
  lcd.print("Good Luck!");
  delay(2000);
  lcd.clear();
}

void stopExam() {
  examStarted = false;
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Exam Stopped!");
  delay(2000);
  lcd.clear();
}

void examFinished() {
  examStarted = false;
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Exam Finished!");
  lcd.setCursor(0, 1);
  lcd.print("Well Done!");
  playSound();
  delay(3000);
  lcd.clear();
}

void checkAlarm() {
  unsigned long currentTime = millis();

  if (currentTime >= nextAlarmTime && alarmEnabled) {
    triggerAlarm();

    // Set next alarm
    lastAlarmTime = currentTime;
    if (alarmPattern == HALF_HOUR) {
      nextAlarmTime = currentTime + 1800000;
    } else if (alarmPattern == ONE_HOUR) {
      nextAlarmTime = currentTime + 3600000;
```

```
    } else if (alarmPattern == CUSTOM) {
      nextAlarmTime = currentTime + (customInterval * 60000UL);
    } else { // RANDOM
      unsigned long minMs = randomMinInterval * 60000UL;
      unsigned long maxMs = randomMaxInterval * 60000UL;
      nextAlarmTime = currentTime + random(minMs, maxMs);
    }
  }
}

void triggerAlarm() {
  playSound();
}

void playSound() {
  switch (soundType) {
    case SINGLE_BEEP:
      tone(BUZZER_PIN, 1000, 200);
      break;

    case DOUBLE_BEEP:
      tone(BUZZER_PIN, 1000, 200);
      delay(800);
      tone(BUZZER_PIN, 1000, 200);
      break;

    case LONG_BEEP:
      tone(BUZZER_PIN, 1000, 1000);
      break;
  }
}
```

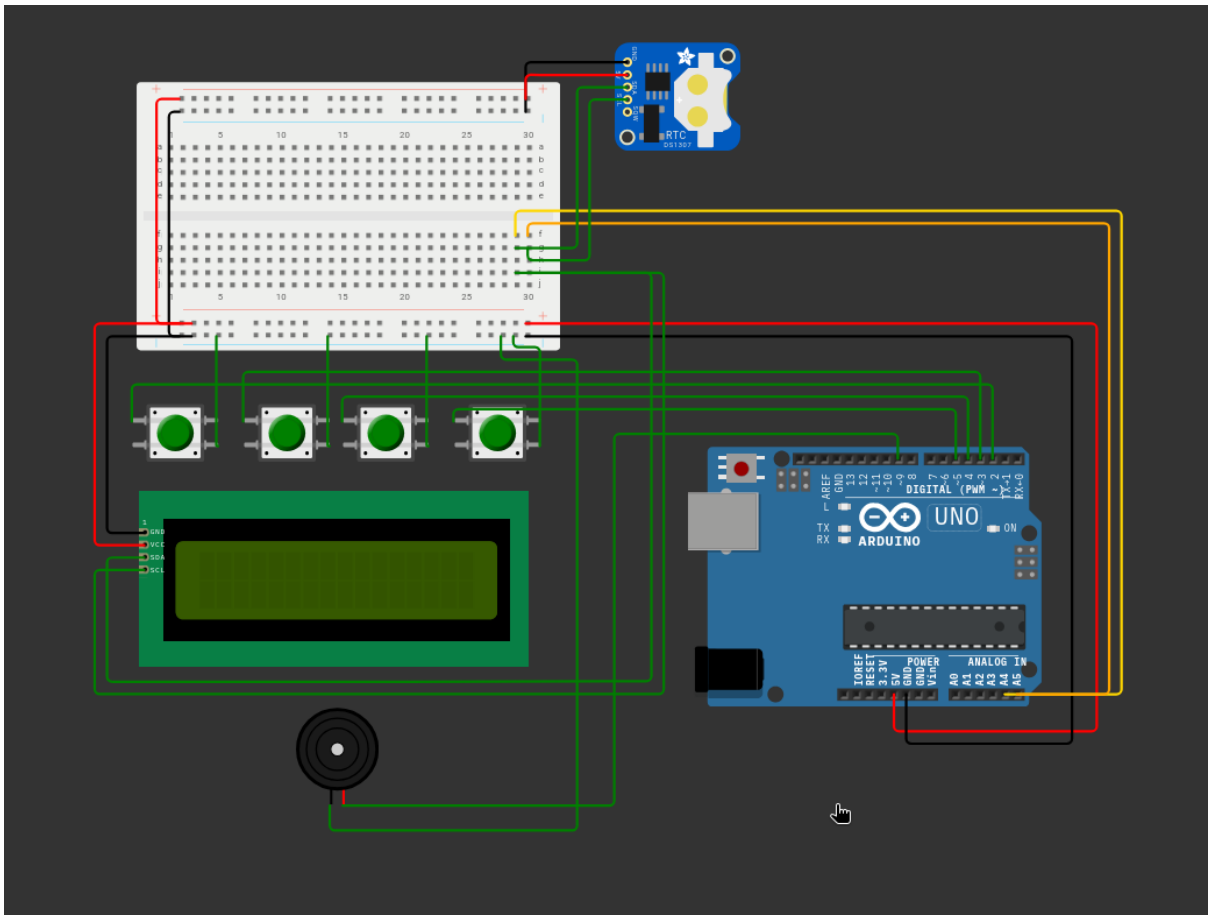
JSON

```
{
  "version": 1,
  "author": "Shree Kottes",
  "editor": "wokwi",
  "parts": [
    { "type": "wokwi-breadboard-half", "id": "bb1", "top": -204.6,
     ↪  "left": -131.6, "attrs": {} },
    { "type": "wokwi-arduino-uno", "id": "uno", "top": 77.4, "left": 297,
     ↪  "attrs": {} },
    { "type": "wokwi-ds1307", "id": "rtc1", "top": -235.8, "left": 240.1,
     ↪  "attrs": {} },
    {
      "type": "wokwi-pushbutton",
```

```
      "id": "btn1",
      "top": 44.6,
      "left": -134.4,
      "attrs": { "color": "green", "xray": "1" }
    },
    {
      "type": "wokwi-lcd1602",
      "id": "lcd1",
      "top": 112,
      "left": -128.8,
      "attrs": { "pins": "i2c" }
    },
    {
      "type": "wokwi-pushbutton",
      "id": "btn2",
      "top": 44.6,
      "left": -48,
      "attrs": { "color": "green", "xray": "1" }
    },
    {
      "type": "wokwi-pushbutton",
      "id": "btn3",
      "top": 44.6,
      "left": 28.8,
      "attrs": { "color": "green", "xray": "1" }
    },
    {
      "type": "wokwi-pushbutton",
      "id": "btn4",
      "top": 44.6,
      "left": 115.2,
      "attrs": { "color": "green", "xray": "1" }
    },
    {
      "type": "wokwi-buzzer",
      "id": "bz1",
      "top": 271.2,
      "left": -7.8,
      "attrs": { "volume": "0.1" }
    }
  ],
  "connections": [
    [ "uno:5V", "bb1:bp.25", "red", [ "v28.7", "h157.4", "v-315.9" ] ],
    [ "uno:GND.2", "bb1:bn.25", "black", [ "v38.3", "h128.7", "v-315.5" ]
     ↪ ],
    [ "uno:A4", "bb1:29b.f", "gold", [ "h90.6", "v-374.5", "h-470.4" ] ],
    [ "uno:A5", "bb1:30b.f", "orange", [ "h71.5", "v-364.9", "h-451.2" ]
     ↪ ],
```

```
    [ "bb1:bp.1", "bb1:tp.1", "red", [ "h-17.6", "v-174" ] ],
    [ "bb1:bn.1", "bb1:tn.1", "black", [ "h-8", "v-174" ] ],
    [ "rtc1:GND", "bb1:tn.25", "black", [ "h0" ] ],
    [ "rtc1:5V", "bb1:tp.25", "red", [ "h-67.2", "v-0.4", "h-11.2" ] ],
    [ "rtc1:SDA", "bb1:29b.g", "green", [ "h-38.4", "v124.5", "h-48" ] ],
    [ "rtc1:SCL", "bb1:30b.g", "green", [ "h-28.8", "v124.6", "h-48" ] ],
    [ "lcd1:GND", "bb1:bn.2", "black", [ "h-28.8", "v-152.3" ] ],
    [ "lcd1:VCC", "bb1:bp.2", "red", [ "h-38.4", "v-171.8" ] ],
    [
      "lcd1:SDA",
      "bb1:29b.i",
      "green",
      [ "h-28.8", "v96.2", "h230.4", "v-0.4", "h192", "v-316.4" ]
    ],
    [ "lcd1:SCL", "bb1:30b.i", "green", [ "h-38.4", "v96.3", "h441.6",
     ↪  "v-326.4" ] ],
    [ "btn1:1.l", "uno:2", "green", [ "v-28.8", "h667.9" ] ],
    [ "btn2:1.l", "uno:3", "green", [ "v-38.4", "h572" ] ],
    [ "btn3:1.l", "uno:4", "green", [ "v-19.2", "h485.7" ] ],
    [ "btn4:1.l", "uno:5", "green", [ "v-9.6", "h389.8" ] ],
    [ "bz1:2", "uno:9", "green", [ "h210.8", "v-288", "h220" ] ],
    [ "bz1:1", "bb1:bn.23", "green", [ "v19.2", "h192", "v-364.8",
     ↪  "h-59.2" ] ],
    [ "btn4:2.r", "bb1:bn.24", "green", [ "h0.2", "v-76.6", "h-20.8" ] ],
    [ "bb1:bn.4", "btn1:2.r", "green", [ "v0" ] ],
    [ "btn2:2.r", "bb1:bn.11", "green", [ "h0" ] ],
    [ "btn3:2.r", "bb1:bn.18", "green", [ "h0" ] ]
  ],
  "dependencies": {}
}
```

PIN CONFIGURATION



EX.NO: 2	TRAFFIC CONTROL SYSTEM
26/12/2025	

AIM

To build a Traffic Control System using Arduino Board.

APPARATUS REQUIRED

- Arduino Uno
- Breadboards
- RTC DS3231 module
- I2C LCD Display (16x2)
- LEDs (4x Red, 4x Yellow, 4x Green)
- 220-ohm Resistors (x12)
- Pushbuttons (x3) (Menu, Up, Down)
- Jumper wires

PRE-CONDITIONS

- The 12 LEDs (representing 4 directions: N, S, E, W) and their current-limiting resistors are wired correctly to the Arduino digital pins.
- RTC and I2C LCD are properly connected to the I2C pins.
- The required libraries (`Wire.h`, `RTClib.h`, `LiquidCrystal_I2C.h`) are installed in the Arduino IDE.
- The code is successfully compiled and uploaded to the Arduino Uno board.

POST-CONDITIONS

- The traffic lights correctly sequence through Green, Yellow, and Red for North/South and East/West directions.
- The LCD displays the current time, status (Normal or Peak hours), and the active traffic phase.
- The timing dynamically adjusts based on peak hours retrieved from the RTC.
- The user can adjust timings (Peak Green, Normal Green, Yellow, Peak Hours) using the menu system.

CODE

```
#include <Wire.h>
#include <RTClib.h>
#include <LiquidCrystal_I2C.h>

// Initialize RTC and LCD
RTC_DS3231 rtc;
```

```
LiquidCrystal_I2C lcd(0x27, 16, 2); // Address 0x27, 16x2 LCD

// Traffic Light Pin Definitions
// Direction 1 (North)
const int N_RED = 2;
const int N_YELLOW = 3;
const int N_GREEN = 4;

// Direction 2 (South)
const int S_RED = 5;
const int S_YELLOW = 6;
const int S_GREEN = 7;

// Direction 3 (East)
const int E_RED = 8;
const int E_YELLOW = 9;
const int E_GREEN = 10;

// Direction 4 (West)
const int W_RED = 11;
const int W_YELLOW = 12;
const int W_GREEN = 13;

// Button Pins
const int BTN_MENU = A0;
const int BTN_UP = A1;
const int BTN_DOWN = A2;

// Timing Configuration (in seconds)
int peakGreenTime = 20;      // Green light during peak hours
int normalGreenTime = 30;    // Green light during normal hours
int yellowTime = 3;          // Yellow light duration
int allRedTime = 2;          // All red safety time

// Peak Hours Configuration (24-hour format)
int peakStartHour = 7;       // Peak hours start at 7:00 AM
int peakEndHour = 10;        // Peak hours end at 10:00 AM

// System Variables
unsigned long previousMillis = 0;
int currentPhase = 0;        // 0=N/S Green, 1=N/S Yellow, 2=All Red,
 ↪  3=E/W Green, 4=E/W Yellow, 5=All Red
int phaseTimer = 0;
bool isPeakHour = false;
int currentGreenTime = normalGreenTime;

// Menu System
enum MenuState {
```

```
  MENU_MAIN,
  MENU_PEAK_GREEN,
  MENU_NORMAL_GREEN,
  MENU_YELLOW,
  MENU_PEAK_START,
  MENU_PEAK_END
};

MenuState currentMenu = MENU_MAIN;
bool inConfigMode = false;
unsigned long lastButtonPress = 0;
const unsigned long buttonDebounce = 200;

void setup() {
  Serial.begin(9600);

  // Initialize RTC
  if (!rtc.begin()) {
    Serial.println("RTC not found!");
    while (1);
  }

  // If RTC lost power, set to compile time
  if (rtc.lostPower()) {
    rtc.adjust(DateTime(F(__DATE__), F(__TIME__)));
  }

  // Initialize LCD
  lcd.init();
  lcd.backlight();
  lcd.clear();
  lcd.print("Traffic System");
  lcd.setCursor(0, 1);
  lcd.print("Initializing...");
  delay(2000);

  // Initialize Traffic Light Pins
  pinMode(N_RED, OUTPUT);
  pinMode(N_YELLOW, OUTPUT);
  pinMode(N_GREEN, OUTPUT);
  pinMode(S_RED, OUTPUT);
  pinMode(S_YELLOW, OUTPUT);
  pinMode(S_GREEN, OUTPUT);
  pinMode(E_RED, OUTPUT);
  pinMode(E_YELLOW, OUTPUT);
  pinMode(E_GREEN, OUTPUT);
  pinMode(W_RED, OUTPUT);
  pinMode(W_YELLOW, OUTPUT);
```

```
  pinMode(W_GREEN, OUTPUT);

  // Initialize Button Pins with internal pull-up
  pinMode(BTN_MENU, INPUT_PULLUP);
  pinMode(BTN_UP, INPUT_PULLUP);
  pinMode(BTN_DOWN, INPUT_PULLUP);

  // Start with all red
  allRed();

  lcd.clear();
}

void loop() {
  // Check buttons
  checkButtons();

  if (inConfigMode) {
    handleConfigMenu();
  } else {
    // Normal traffic light operation
    runTrafficSequence();
    displayStatus();
  }
}

void runTrafficSequence() {
  DateTime now = rtc.now();

  // Check if current time is in peak hours
  int currentHour = now.hour();
  isPeakHour = (currentHour >= peakStartHour && currentHour <
 ↪  peakEndHour);
  currentGreenTime = isPeakHour ? peakGreenTime : normalGreenTime;

  unsigned long currentMillis = millis();

  // Update phase based on timer
  if (currentMillis - previousMillis >= 1000) { // Every second
    previousMillis = currentMillis;
    phaseTimer++;

    // Phase transitions
    switch (currentPhase) {
      case 0: // North/South Green
        if (phaseTimer >= currentGreenTime) {
          phaseTimer = 0;
          currentPhase = 1;
```

```
        }
        break;

      case 1: // North/South Yellow
        if (phaseTimer >= yellowTime) {
          phaseTimer = 0;
          currentPhase = 2;
        }
        break;

      case 2: // All Red (transition)
        if (phaseTimer >= allRedTime) {
          phaseTimer = 0;
          currentPhase = 3;
        }
        break;

      case 3: // East/West Green
        if (phaseTimer >= currentGreenTime) {
          phaseTimer = 0;
          currentPhase = 4;
        }
        break;

      case 4: // East/West Yellow
        if (phaseTimer >= yellowTime) {
          phaseTimer = 0;
          currentPhase = 5;
        }
        break;

      case 5: // All Red (transition)
        if (phaseTimer >= allRedTime) {
          phaseTimer = 0;
          currentPhase = 0;
        }
        break;
    }

    // Update lights based on current phase
    updateLights();
  }
}

void updateLights() {
  switch (currentPhase) {
    case 0: // North/South Green, East/West Red
      digitalWrite(N_GREEN, HIGH);
```

```
    digitalWrite(N_YELLOW, LOW);
    digitalWrite(N_RED, LOW);
    digitalWrite(S_GREEN, HIGH);
    digitalWrite(S_YELLOW, LOW);
    digitalWrite(S_RED, LOW);
    digitalWrite(E_RED, HIGH);
    digitalWrite(E_YELLOW, LOW);
    digitalWrite(E_GREEN, LOW);
    digitalWrite(W_RED, HIGH);
    digitalWrite(W_YELLOW, LOW);
    digitalWrite(W_GREEN, LOW);
    break;

  case 1: // North/South Yellow, East/West Red
    digitalWrite(N_GREEN, LOW);
    digitalWrite(N_YELLOW, HIGH);
    digitalWrite(S_GREEN, LOW);
    digitalWrite(S_YELLOW, HIGH);
    break;

  case 2: // All Red
    allRed();
    break;

  case 3: // East/West Green, North/South Red
    digitalWrite(N_RED, HIGH);
    digitalWrite(N_YELLOW, LOW);
    digitalWrite(N_GREEN, LOW);
    digitalWrite(S_RED, HIGH);
    digitalWrite(S_YELLOW, LOW);
    digitalWrite(S_GREEN, LOW);
    digitalWrite(E_GREEN, HIGH);
    digitalWrite(E_YELLOW, LOW);
    digitalWrite(E_RED, LOW);
    digitalWrite(W_GREEN, HIGH);
    digitalWrite(W_YELLOW, LOW);
    digitalWrite(W_RED, LOW);
    break;

  case 4: // East/West Yellow, North/South Red
    digitalWrite(E_GREEN, LOW);
    digitalWrite(E_YELLOW, HIGH);
    digitalWrite(W_GREEN, LOW);
    digitalWrite(W_YELLOW, HIGH);
    break;

  case 5: // All Red
    allRed();
```

```
      break;
  }
}

void allRed() {
  digitalWrite(N_RED, HIGH);
  digitalWrite(N_YELLOW, LOW);
  digitalWrite(N_GREEN, LOW);
  digitalWrite(S_RED, HIGH);
  digitalWrite(S_YELLOW, LOW);
  digitalWrite(S_GREEN, LOW);
  digitalWrite(E_RED, HIGH);
  digitalWrite(E_YELLOW, LOW);
  digitalWrite(E_GREEN, LOW);
  digitalWrite(W_RED, HIGH);
  digitalWrite(W_YELLOW, LOW);
  digitalWrite(W_GREEN, LOW);
}

void displayStatus() {
  DateTime now = rtc.now();

  lcd.setCursor(0, 0);
  lcd.print(now.hour() < 10 ? "0" : "");
  lcd.print(now.hour());
  lcd.print(":");
  lcd.print(now.minute() < 10 ? "0" : "");
  lcd.print(now.minute());
  lcd.print(" ");
  lcd.print(isPeakHour ? "PEAK" : "NORM");
  lcd.print("   ");

  lcd.setCursor(0, 1);
  switch (currentPhase) {
    case 0:
      lcd.print("N/S:GRN E/W:RED ");
      break;
    case 1:
      lcd.print("N/S:YEL E/W:RED ");
      break;
    case 2:
      lcd.print("ALL RED        ");
      break;
    case 3:
      lcd.print("E/W:GRN N/S:RED ");
      break;
    case 4:
      lcd.print("E/W:YEL N/S:RED ");
```

```
      break;
    case 5:
      lcd.print("ALL RED        ");
      break;
  }
}

void checkButtons() {
  unsigned long currentTime = millis();

  if (currentTime - lastButtonPress < buttonDebounce) {
    return;
  }

  if (digitalRead(BTN_MENU) == LOW) {
    lastButtonPress = currentTime;
    if (!inConfigMode) {
      inConfigMode = true;
      currentMenu = MENU_MAIN;
      lcd.clear();
    } else {
      // Cycle through menu options
      currentMenu = (MenuState)((currentMenu + 1) % 6);

      if (currentMenu == MENU_MAIN) {
        inConfigMode = false;
        lcd.clear();
      }
    }
  }

  if (inConfigMode) {
    if (digitalRead(BTN_UP) == LOW) {
      lastButtonPress = currentTime;
      adjustValue(1);
    }

    if (digitalRead(BTN_DOWN) == LOW) {
      lastButtonPress = currentTime;
      adjustValue(-1);
    }
  }
}

void adjustValue(int direction) {
  switch (currentMenu) {
    case MENU_PEAK_GREEN:
      peakGreenTime = constrain(peakGreenTime + direction, 5, 120);
```

```
      break;
    case MENU_NORMAL_GREEN:
      normalGreenTime = constrain(normalGreenTime + direction, 5, 120);
      break;
    case MENU_YELLOW:
      yellowTime = constrain(yellowTime + direction, 1, 10);
      break;
    case MENU_PEAK_START:
      peakStartHour = constrain(peakStartHour + direction, 0, 23);
      break;
    case MENU_PEAK_END:
      peakEndHour = constrain(peakEndHour + direction, 0, 23);
      break;
    default:
      break;
  }
}

void handleConfigMenu() {
  lcd.setCursor(0, 0);

  switch (currentMenu) {
    case MENU_MAIN:
      lcd.print("Config Menu     ");
      lcd.setCursor(0, 1);
      lcd.print("Press Menu Btn  ");
      break;

    case MENU_PEAK_GREEN:
      lcd.print("Peak Green Time ");
      lcd.setCursor(0, 1);
      lcd.print(peakGreenTime);
      lcd.print(" sec    ");
      break;

    case MENU_NORMAL_GREEN:
      lcd.print("Normal Grn Time ");
      lcd.setCursor(0, 1);
      lcd.print(normalGreenTime);
      lcd.print(" sec    ");
      break;

    case MENU_YELLOW:
      lcd.print("Yellow Time     ");
      lcd.setCursor(0, 1);
      lcd.print(yellowTime);
      lcd.print(" sec    ");
      break;
```

```
    case MENU_PEAK_START:
      lcd.print("Peak Start Hour ");
      lcd.setCursor(0, 1);
      lcd.print(peakStartHour);
      lcd.print(":00    ");
      break;

    case MENU_PEAK_END:
      lcd.print("Peak End Hour   ");
      lcd.setCursor(0, 1);
      lcd.print(peakEndHour);
      lcd.print(":00    ");
      break;
  }
}
```

JSON

```
{
  "version": 1,
  "author": "Shree Kottes",
  "editor": "wokwi",
  "parts": [
    { "type": "wokwi-breadboard-half", "id": "bb1", "top": 342.6, "left":
     ↪  540.4, "attrs": {} },
    { "type": "wokwi-breadboard-mini", "id": "bb2", "top": 94.6, "left":
     ↪  -88.8, "attrs": {} },
    { "type": "wokwi-arduino-uno", "id": "uno", "top": 29.4, "left":
     ↪  508.2, "attrs": {} },
    { "type": "wokwi-led", "id": "led1", "top": -138, "left": -34.6,
     ↪  "attrs": { "color": "red" } },
    {
      "type": "wokwi-led",
      "id": "led2",
      "top": -90,
      "left": -34.6,
      "attrs": { "color": "yellow" }
    },
    {
      "type": "wokwi-led",
      "id": "led3",
      "top": -32.4,
      "left": -34.6,
      "attrs": { "color": "green", "flip": "" }
    },
    {
```

```
      "type": "wokwi-led",
      "id": "led4",
      "top": 284.4,
      "left": -34.6,
      "attrs": { "color": "green", "flip": "" }
    },
    {
      "type": "wokwi-led",
      "id": "led5",
      "top": 351.6,
      "left": -34.6,
      "attrs": { "color": "yellow" }
    },
    {
      "type": "wokwi-led",
      "id": "led6",
      "top": 418.8,
      "left": -34.6,
      "attrs": { "color": "red" }
    },
    {
      "type": "wokwi-led",
      "id": "led7",
      "top": 130.8,
      "left": -351.4,
      "attrs": { "color": "red" }
    },
    {
      "type": "wokwi-led",
      "id": "led8",
      "top": 130.8,
      "left": -293.8,
      "attrs": { "color": "yellow" }
    },
    {
      "type": "wokwi-led",
      "id": "led9",
      "top": 130.8,
      "left": -236.2,
      "attrs": { "color": "green", "flip": "" }
    },
    {
      "type": "wokwi-led",
      "id": "led10",
      "top": 121.2,
      "left": 282.2,
      "attrs": { "color": "red" }
    },
```

```
    {
      "type": "wokwi-led",
      "id": "led11",
      "top": 121.2,
      "left": 215,
      "attrs": { "color": "yellow" }
    },
    {
      "type": "wokwi-led",
      "id": "led12",
      "top": 121.2,
      "left": 157.4,
      "attrs": { "color": "green", "flip": "" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r1",
      "top": -101.65,
      "left": 9.6,
      "attrs": { "value": "220" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r2",
      "top": -53.65,
      "left": 9.6,
      "attrs": { "value": "220" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r3",
      "top": 4.25,
      "left": 8.2,
      "attrs": { "value": "220" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r4",
      "top": 196.8,
      "left": 153.05,
      "rotate": 90,
      "attrs": { "value": "220" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r5",
      "top": 196.8,
      "left": 210.65,
```

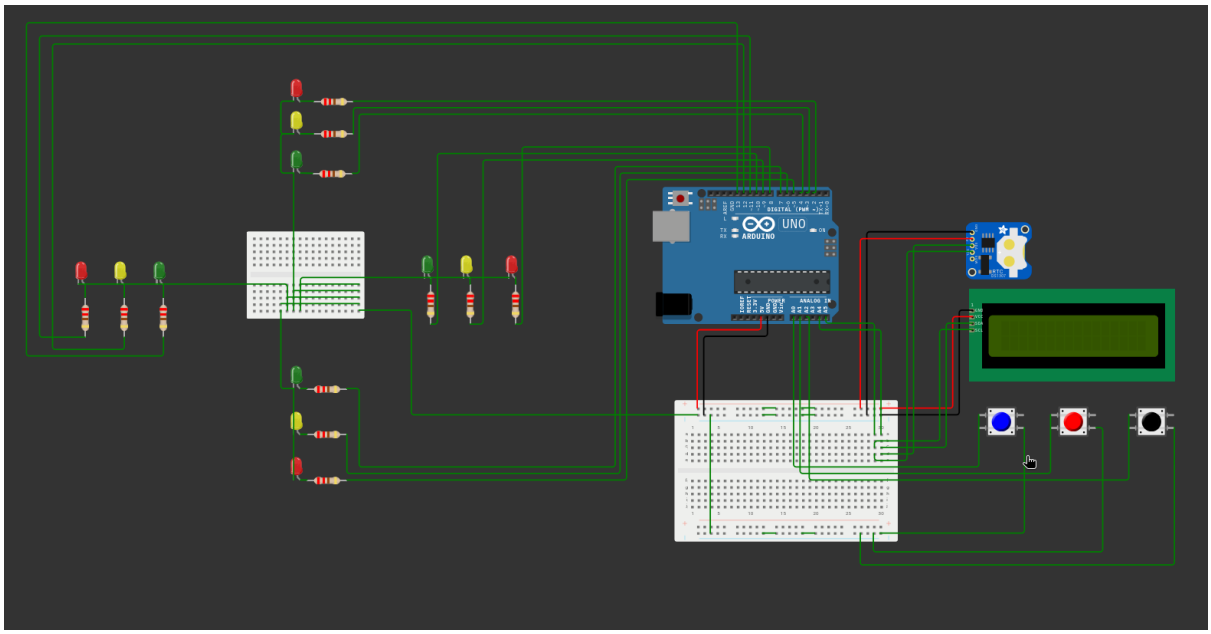
```
      "rotate": 90,
      "attrs": { "value": "220" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r6",
      "top": 196.8,
      "left": 277.85,
      "rotate": 90,
      "attrs": { "value": "220" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r7",
      "top": 320.75,
      "left": 0,
      "attrs": { "value": "220" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r8",
      "top": 387.95,
      "left": 0,
      "attrs": { "value": "220" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r9",
      "top": 455.15,
      "left": 0,
      "attrs": { "value": "220" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r10",
      "top": 216,
      "left": -355.75,
      "rotate": 90,
      "attrs": { "value": "220" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r11",
      "top": 216,
      "left": -298.15,
      "rotate": 90,
      "attrs": { "value": "220" }
    },
```

```
    {
      "type": "wokwi-resistor",
      "id": "r12",
      "top": 216,
      "left": -240.55,
      "rotate": 90,
      "attrs": { "value": "220" }
    },
    {
      "type": "wokwi-lcd1602",
      "id": "lcd1",
      "top": 179.2,
      "left": 975.2,
      "attrs": { "pins": "i2c" }
    },
    { "type": "wokwi-ds1307", "id": "rtc1", "top": 81, "left": 969.7,
     ↪  "attrs": {} },
    {
      "type": "wokwi-pushbutton",
      "id": "btn1",
      "top": 351.8,
      "left": 988.8,
      "attrs": { "color": "blue", "xray": "1" }
    },
    {
      "type": "wokwi-pushbutton",
      "id": "btn2",
      "top": 351.8,
      "left": 1094.4,
      "attrs": { "color": "red", "xray": "1" }
    },
    {
      "type": "wokwi-pushbutton",
      "id": "btn3",
      "top": 351.8,
      "left": 1209.6,
      "attrs": { "color": "black", "xray": "1" }
    }
  ],
  "connections": [
    [ "led7:A", "r10:1", "green", [ "v0" ] ],
    [ "led8:A", "r11:1", "green", [ "v0" ] ],
    [ "led9:A", "r12:1", "green", [ "v0" ] ],
    [ "led4:A", "r7:1", "green", [ "v0" ] ],
    [ "led5:A", "r8:1", "green", [ "v0" ] ],
    [ "led6:A", "r9:1", "green", [ "v0" ] ],
    [ "led12:A", "r4:1", "green", [ "v0" ] ],
    [ "led11:A", "r5:1", "green", [ "v0" ] ],
```

```
    [ "led10:A", "r6:1", "green", [ "v0" ] ],
    [ "led3:A", "r3:1", "green", [ "v0" ] ],
    [ "led2:A", "r2:1", "green", [ "v0" ] ],
    [ "led1:A", "r1:1", "green", [ "v0" ] ],
    [ "led1:C", "led2:C", "green", [ "h-18.8", "v48" ] ],
    [ "led3:C", "led2:C", "green", [ "h-18.8", "v-57.6" ] ],
    [ "led3:C", "bb2:7b.j", "green", [ "v0" ] ],
    [ "led11:C", "led12:C", "green", [ "v0" ] ],
    [ "led10:C", "led11:C", "green", [ "v0" ] ],
    [ "led12:C", "bb2:8b.j", "green", [ "h-124.4", "v0", "h-57.6" ] ],
    [ "led7:C", "led8:C", "green", [ "v0" ] ],
    [ "led8:C", "led9:C", "green", [ "v0" ] ],
    [ "led9:C", "bb2:6b.j", "green", [ "h192.4", "v38.4" ] ],
    [ "led6:C", "led5:C", "green", [ "v0" ] ],
    [ "led4:C", "led5:C", "green", [ "v0" ] ],
    [ "led4:C", "bb2:5b.j", "green", [ "h-18.8", "v-115.2" ] ],
    [ "r1:2", "uno:2", "green", [ "h680.4", "v134.4" ] ],
    [ "r2:2", "uno:3", "green", [ "v-38.4", "h670.8", "v124.8" ] ],
    [ "r3:2", "uno:4", "green", [ "h9.8", "v-86.7", "h652.8", "v115.2" ]
    ↪ ],
    [ "r9:2", "uno:5", "green", [ "h411.6", "v-441.6", "h245.8" ] ],
    [ "r8:2", "uno:6", "green", [ "v57.6", "h402", "v-441.6", "h245.9" ]
    ↪ ],
    [ "r7:2", "uno:7", "green", [ "h18", "v115.2", "h376.8", "v-441.6",
    ↪ "h243.6" ] ],
    [ "r6:2", "uno:8", "green", [ "h9.6", "v-260.4", "h364.4" ] ],
    [ "r5:2", "uno:9", "green", [ "h19.2", "v-241.2", "h412" ] ],
    [ "r4:2", "uno:10", "green", [ "h9.6", "v-250.8", "h469.7" ] ],
    [ "r10:2", "uno:11", "green", [ "h-67.2", "v-442.8", "h1045.8" ] ],
    [ "r11:2", "uno:12", "green", [ "v18", "h-105.6", "v-448.8",
    ↪ "h1017.1" ] ],
    [ "r12:2", "uno:13", "green", [ "v27.6", "h-201.6", "v-489.6",
    ↪ "h1046" ] ],
    [ "uno:5V", "bb1:tp.1", "red", [ "v19.1", "h-93.8" ] ],
    [ "bb2:17b.j", "bb1:tn.1", "green", [ "h76.8", "v153.7" ] ],
    [ "uno:GND.2", "bb1:tn.2", "black", [ "v28.7", "h-93.7" ] ],
    [ "bb1:tn.10", "bb1:tn.11", "green", [ "v0" ] ],
    [ "bb1:tp.10", "bb1:tp.11", "green", [ "v0" ] ],
    [ "bb1:tp.15", "bb1:tp.16", "green", [ "v0" ] ],
    [ "bb1:tn.15", "bb1:tn.16", "green", [ "v0" ] ],
    [ "lcd1:GND", "bb1:tn.25", "black", [ "h-19.2", "v153.7" ] ],
    [ "lcd1:VCC", "bb1:tp.25", "red", [ "h-28.8", "v134.5", "h-107.2" ]
    ↪ ],
    [ "lcd1:SDA", "bb1:30t.c", "green", [ "h-38.4", "v182.6" ] ],
    [ "lcd1:SCL", "bb1:29t.c", "green", [ "h-48", "v163.5", "h-96" ] ],
    [ "rtc1:GND", "bb1:tn.23", "black", [ "h0" ] ],
    [ "rtc1:5V", "bb1:tp.22", "red", [ "h0" ] ],
    [ "rtc1:SDA", "bb1:30t.d", "green", [ "h-86.4", "v306.9" ] ],
```

```
    [ "rtc1:SCL", "bb1:29t.d", "green", [ "h-96", "v307", "h-48" ] ],
    [ "bb1:30t.a", "uno:A4", "green", [ "v-153.6", "h-90.6" ] ],
    [ "bb1:29t.a", "uno:A5", "green", [ "v-163.2", "h-71.5" ] ],
    [ "btn1:2.r", "bb1:bn.25", "green", [ "h0.2", "v155.1" ] ],
    [ "btn2:2.r", "bb1:bn.24", "green", [ "h9.8", "v182.6", "h-337.6" ]
     ↪  ],
    [ "btn3:2.r", "bb1:bn.22", "green", [ "h0.2", "v201.8", "h-462.4" ]
     ↪  ],
    [ "bb1:tn.3", "bb1:bn.3", "green", [ "v0" ] ],
    [ "bb1:bn.10", "bb1:bn.11", "green", [ "v0" ] ],
    [ "bb1:bn.15", "bb1:bn.16", "green", [ "v0" ] ],
    [ "btn1:1.l", "uno:A0", "green", [ "v76.8", "h-272.6" ] ],
    [ "btn2:1.l", "uno:A1", "green", [ "v86.4", "h-368.7" ] ],
    [ "btn3:1.l", "uno:A2", "green", [ "v96", "h-470.4", "v-239.9" ] ],
    [ "bb2:17b.f", "bb2:8b.f", "green", [ "v0" ] ],
    [ "bb2:17b.g", "bb2:7b.g", "green", [ "v0" ] ],
    [ "bb2:17b.h", "bb2:6b.h", "green", [ "v0" ] ],
    [ "bb2:17b.i", "bb2:5b.i", "green", [ "v0" ] ]
  ],
  "dependencies": {}
}
```

PIN CONFIGURATION



EX.NO: 3	ODOMETER
04/01/2026	

AIM

To simulate the odometer(distance calculator)

APPARATUS REQUIRED

- Arduino Uno
- Breadboard
- Stepper Motor (NEMA 17)
- A4988 Stepper Motor Driver
- I2C LCD Display (16x2)
- Pushbuttons (x4) (Start/Stop, Increase Speed, Decrease Speed, Reset)
- Jumper wires

PRE-CONDITIONS

- The stepper motor, A4988 driver module, pushbuttons, and I2C LCD are wired securely to the Arduino.
- The A4988 driver is connected with proper microstepping pins configured (MS1, MS2, MS3) and appropriate power.
- The `Wire.h` and `LiquidCrystal_I2C.h` libraries are installed in the Arduino IDE.
- The code is successfully compiled and uploaded to the Arduino Uno board.

POST-CONDITIONS

- The stepper motor acts as a wheel simulator, turning at the specified target speed.
- The LCD accurately displays the simulated speed (m/s) and total distance traveled (m).
- The pushbuttons effectively start/stop the motor, adjust the simulated target speed, and reset the distance counter.
- System handles speed adjustments gracefully without sudden jumps.

CODE

```
float targetSpeed = 0.5;          // m/s (adjustable)
float maxSpeed = 5000.0;             // m/s (increased limit)
float minSpeed = 0.1;             // m/s
float speedStep = 0.2;            // m/s increment#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// === PIN DEFINITIONS ===
#define STEP_PIN 8
```

```
#define DIR_PIN 9
#define ENABLE_PIN 10
#define RESET_PIN 11

#define BTN1_PIN 2   // Start/Stop
#define BTN2_PIN 3   // Increase Speed
#define BTN3_PIN 4   // Decrease Speed
#define BTN4_PIN 5   // Reset Distance

// Track button states
volatile bool btn2Pressed = false;
volatile bool btn3Pressed = false;
volatile bool btn4Pressed = false;

// === STEPPER MOTOR PARAMETERS ===
#define STEPS_PER_REV 200        // 1.8° per step
#define MICROSTEPS 16            // A4988 microstepping (MS1, MS2, MS3
 ↪  high)
#define WHEEL_DIAMETER 0.1       // 10cm diameter in meters
#define WHEEL_CIRCUMFERENCE (PI * WHEEL_DIAMETER)
#define STEPS_PER_METER (STEPS_PER_REV * MICROSTEPS) /
 ↪  WHEEL_CIRCUMFERENCE

// === LCD SETUP ===
LiquidCrystal_I2C lcd(0x27, 16, 2); // Address 0x27, 16x2 display

// === VARIABLES ===
volatile long totalSteps = 0;
volatile unsigned long lastStepTime = 0;
unsigned long lastDisplayUpdate = 0;
unsigned long lastSpeedCalcTime = 0;

float currentSpeed = 0.0;        // m/s
float totalDistance = 0.0;       // meters


bool motorRunning = false;
bool speedChanged = false;

// Button debounce variables
unsigned long lastBtnTime[4] = {0, 0, 0, 0};
#define DEBOUNCE_DELAY 200

// === INTERRUPT HANDLERS ===
void btn1ISR() {
  if (millis() - lastBtnTime[0] < DEBOUNCE_DELAY) return;
  lastBtnTime[0] = millis();
  motorRunning = !motorRunning;
```

```
  if (motorRunning) {
    digitalWrite(ENABLE_PIN, LOW);  // Enable motor
  } else {
    digitalWrite(ENABLE_PIN, HIGH); // Disable motor
  }
}

void btn2ISR() {
  btn2Pressed = true;
}

void btn3ISR() {
  btn3Pressed = true;
}

void btn4ISR() {
  btn4Pressed = true;
}

// === SETUP ===
void setup() {
  Serial.begin(9600);

  // Initialize pins
  pinMode(STEP_PIN, OUTPUT);
  pinMode(DIR_PIN, OUTPUT);
  pinMode(ENABLE_PIN, OUTPUT);
  pinMode(RESET_PIN, OUTPUT);

  pinMode(BTN1_PIN, INPUT_PULLUP);
  pinMode(BTN2_PIN, INPUT_PULLUP);
  pinMode(BTN3_PIN, INPUT_PULLUP);
  pinMode(BTN4_PIN, INPUT_PULLUP);

  // Initialize motor control
  digitalWrite(RESET_PIN, HIGH);   // Release reset
  digitalWrite(ENABLE_PIN, HIGH);  // Disable motor initially
  digitalWrite(DIR_PIN, HIGH);     // Set direction (CCW)

  // Initialize LCD
  lcd.init();
  lcd.backlight();
  lcd.print("Odometer Ready!");
  delay(2000);
  lcd.clear();

  // Attach interrupts
```

```
  attachInterrupt(digitalPinToInterrupt(BTN1_PIN), btn1ISR, FALLING);
  attachInterrupt(digitalPinToInterrupt(BTN2_PIN), btn2ISR, FALLING);
  attachInterrupt(digitalPinToInterrupt(BTN3_PIN), btn3ISR, FALLING);
  attachInterrupt(digitalPinToInterrupt(BTN4_PIN), btn4ISR, FALLING);

  lastDisplayUpdate = millis();
  lastSpeedCalcTime = millis();
}

// === MAIN LOOP ===
void loop() {
  // Handle button presses with debouncing
  handleButtonInputs();

  // Step the motor if running
  if (motorRunning) {
    stepMotor();
  }

  // Calculate speed (steps per second to m/s)
  unsigned long currentTime = millis();
  if (currentTime - lastSpeedCalcTime >= 500) {
    updateSpeed(currentTime - lastSpeedCalcTime);
    lastSpeedCalcTime = currentTime;
  }

  // Update LCD display
  if (currentTime - lastDisplayUpdate >= 200) {
    updateLCD();
    lastDisplayUpdate = currentTime;
  }
}

// === BUTTON HANDLER ===
void handleButtonInputs() {
  static unsigned long lastBtn2Time = 0;
  static unsigned long lastBtn3Time = 0;
  static unsigned long lastBtn4Time = 0;
  unsigned long currentTime = millis();

  // Handle button 2 (Increase Speed)
  if (btn2Pressed) {
    if (currentTime - lastBtn2Time >= DEBOUNCE_DELAY) {
      targetSpeed += speedStep;
      if (targetSpeed > maxSpeed) targetSpeed = maxSpeed;
      speedChanged = true;
      lastBtn2Time = currentTime;
    }
```

```
    btn2Pressed = false;
  }

  // Handle button 3 (Decrease Speed)
  if (btn3Pressed) {
    if (currentTime - lastBtn3Time >= DEBOUNCE_DELAY) {
      targetSpeed -= speedStep;
      if (targetSpeed < minSpeed) targetSpeed = minSpeed;
      speedChanged = true;
      lastBtn3Time = currentTime;
    }
    btn3Pressed = false;
  }

  // Handle button 4 (Reset Distance)
  if (btn4Pressed) {
    if (currentTime - lastBtn4Time >= DEBOUNCE_DELAY) {
      totalSteps = 0;
      totalDistance = 0.0;
      currentSpeed = 0.0;
      lastBtn4Time = currentTime;
    }
    btn4Pressed = false;
  }

  // Print speed if changed
  if (speedChanged) {
    Serial.print("Target Speed: ");
    Serial.print(targetSpeed, 2);
    Serial.println(" m/s");
    speedChanged = false;
  }
}

// === STEPPER MOTOR CONTROL ===
void stepMotor() {
  // Calculate delay based on target speed
  // Speed (m/s) = steps/second / steps_per_meter
  // steps/second = speed * steps_per_meter
  float stepsPerSecond = targetSpeed * STEPS_PER_METER;
  float delayMicros = 1000000.0 / stepsPerSecond;

  unsigned long currentMicros = micros();

  // Only step if enough time has passed
  if (currentMicros - lastStepTime >= delayMicros) {
    // Pulse the STEP pin
    digitalWrite(STEP_PIN, HIGH);
```

```
    delayMicroseconds(5);
    digitalWrite(STEP_PIN, LOW);

    lastStepTime = currentMicros;
    totalSteps++;

    // Update distance (in meters)
    totalDistance = (float)totalSteps / STEPS_PER_METER;
  }
}

// === SPEED CALCULATION ===
void updateSpeed(unsigned long deltaTime) {
  // Current speed based on recent step rate
  // This is smoothed over the last update interval
  float deltaSeconds = deltaTime / 1000.0;

  // If moving, report target speed; otherwise 0
  if (motorRunning && targetSpeed > 0) {
    currentSpeed = targetSpeed;
  } else {
    currentSpeed = 0.0;
  }
}

// === LCD UPDATE ===
void updateLCD() {
  // Line 1: Speed
  lcd.setCursor(0, 0);
  lcd.print("Speed:");
  lcd.setCursor(7, 0);
  lcd.print(currentSpeed, 2);
  lcd.print(" m/s ");

  // Line 2: Distance
  lcd.setCursor(0, 1);
  lcd.print("Dist:");
  lcd.setCursor(6, 1);
  lcd.print(totalDistance, 2);
  lcd.print("m   ");

  // Debug output to Serial
  Serial.print("Speed: ");
  Serial.print(currentSpeed, 2);
  Serial.print(" m/s | Distance: ");
  Serial.print(totalDistance, 2);
  Serial.print(" m | Running: ");
  Serial.println(motorRunning ? "YES" : "NO");
```

```
}

// === UTILITY FUNCTIONS ===
void displayStatus(const char* msg) {
  lcd.clear();
  lcd.print(msg);
  delay(1000);
}
```

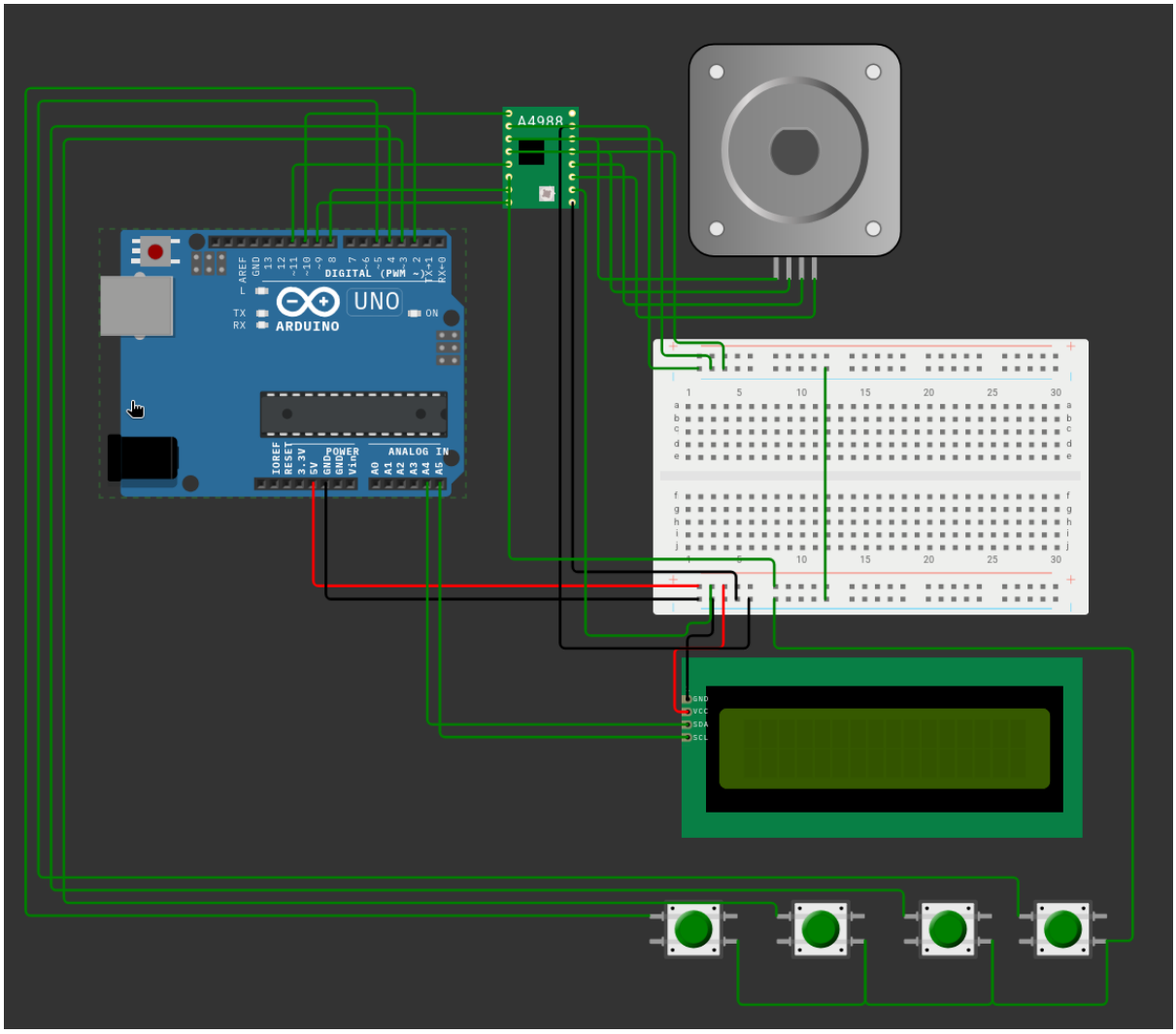
JSON

```
{
  "version": 1,
  "author": "Shree Kottes",
  "editor": "wokwi",
  "parts": [
    { "type": "wokwi-breadboard-half", "id": "bb1", "top": 141, "left":
     ↪  175.6, "attrs": {} },
    { "type": "wokwi-arduino-uno", "id": "uno", "top": 58.2, "left":
     ↪  -240.6, "attrs": {} },
    {
      "type": "wokwi-stepper-motor",
      "id": "stepper1",
      "top": -82.79,
      "left": 202.03,
      "attrs": { "size": "17" }
    },
    { "type": "wokwi-a4988", "id": "drv1", "top": -33.6, "left": 62.4,
     ↪  "attrs": {} },
    {
      "type": "wokwi-lcd1602",
      "id": "lcd1",
      "top": 380.8,
      "left": 197.6,
      "attrs": { "pins": "i2c" }
    },
    {
      "type": "wokwi-pushbutton",
      "id": "btn1",
      "top": 563,
      "left": 172.8,
      "attrs": { "color": "green", "xray": "1" }
    },
    {
      "type": "wokwi-pushbutton",
      "id": "btn2",
      "top": 563,
```

```
      "left": 268.8,
      "attrs": { "color": "green", "xray": "1" }
    },
    {
      "type": "wokwi-pushbutton",
      "id": "btn3",
      "top": 563,
      "left": 364.8,
      "attrs": { "color": "green", "xray": "1" }
    },
    {
      "type": "wokwi-pushbutton",
      "id": "btn4",
      "top": 563,
      "left": 451.2,
      "attrs": { "color": "green", "xray": "1" }
    }
  ],
  "connections": [
    [ "uno:5V", "bb1:bp.1", "red", [ "v0" ] ],
    [ "uno:GND.2", "bb1:bn.1", "black", [ "v0" ] ],
    [ "lcd1:GND", "bb1:bn.2", "black", [ "v-48", "h19.2", "v-46.7" ] ],
    [ "lcd1:VCC", "bb1:bp.3", "red", [ "h-9.6", "v-47.9", "h36.8" ] ],
    [ "lcd1:SCL", "uno:A5", "green", [ "h0" ] ],
    [ "lcd1:SDA", "uno:A4", "green", [ "h0" ] ],
    [ "drv1:ENABLE", "uno:10", "green", [ "h-153.6", "v96" ] ],
    [ "drv1:RESET", "uno:11", "green", [ "h-163.2", "v57.6" ] ],
    [ "drv1:STEP", "uno:8", "green", [ "h0" ] ],
    [ "drv1:DIR", "uno:9", "green", [ "h0" ] ],
    [ "drv1:GND.1", "bb1:bn.4", "black", [ "v278.48", "h123.35" ] ],
    [ "drv1:GND.2", "bb1:bn.5", "black", [ "h-9.45", "v393.68", "h142.4"
    ↪  ] ],
    [ "drv1:VDD", "bb1:bp.2", "green", [ "h9.75", "v336.08", "h76.8",
    ↪  "v-9.6", "h17.6" ] ],
    [ "stepper1:A-", "drv1:2B", "green", [ "h-134.4", "v-105.68" ] ],
    [ "stepper1:A+", "drv1:2A", "green", [ "v9.6", "h-134.4", "v-105.68"
    ↪  ] ],
    [ "stepper1:B+", "drv1:1A", "green", [ "v19.2", "h-134.4", "v-105.68"
    ↪  ] ],
    [ "stepper1:B-", "drv1:1B", "green", [ "v28.8", "h-134.41",
    ↪  "v-105.68" ] ],
    [ "bb1:bn.10", "bb1:tn.10", "green", [ "v0" ] ],
    [ "drv1:MS1", "bb1:tn.1", "green", [ "h105.6", "v182.5" ] ],
    [ "drv1:MS2", "bb1:tn.2", "green", [ "h115.2", "v163.2", "h36.8" ] ],
    [ "drv1:MS3", "bb1:tn.3", "green", [ "h124.8", "v144", "h36.8" ] ],
    [ "drv1:SLEEP", "bb1:bp.6", "green", [ "v288", "h200" ] ],
    [ "btn1:2.r", "btn2:2.r", "green", [ "v48.2", "h96" ] ],
    [ "btn2:2.r", "btn3:2.r", "green", [ "v48.2", "h96" ] ],
```

```
    [ "btn3:2.r", "btn4:2.r", "green", [ "v48.2", "h86.4" ] ],
    [ "btn4:2.r", "bb1:bn.6", "green", [ "h19.4", "v-220.6", "h-270.4" ]
     ↪  ],
    [ "btn1:1.l", "uno:2", "green", [ "h-470.4", "v-624", "h293.5" ] ],
    [ "uno:3", "btn2:1.l", "green", [ "v-76.8", "h-255.2", "v576",
     ↪  "h537.6" ] ],
    [ "uno:4", "btn3:1.l", "green", [ "v-86.4", "h-255.3", "v576",
     ↪  "h643.2" ] ],
    [ "uno:5", "btn4:1.l", "green", [ "v-105.6", "h-255.4", "v585.6",
     ↪  "h739.2" ] ]
  ],
  "dependencies": {}
}
```

PIN CONFIGURATION



EX.NO: 4	MUSIC VISUALIZER
30/01/2026	

AIM

To simulate the Music Visualizer

APPARATUS REQUIRED

- Arduino Uno
- MAX7219 8x8 LED Matrix Module
- 10-Segment LED Bar Graph
- Buzzer
- Resistors (x10)
- Pushbuttons (x2) (Next, Previous)
- Jumper wires

PRE-CONDITIONS

- The LED matrix is correctly connected via SPI (DIN, CS, CLK).
- The 10-Segment LED Bar Graph is properly wired with current-limiting resistors to the Arduino digital and analog pins.
- The buzzer and pushbuttons are appropriately connected.
- The `LedControl.h` library is installed in the Arduino IDE.
- The code is successfully compiled and uploaded to the Arduino Uno board.

POST-CONDITIONS

- The buzzer plays melodies based on predefined musical notes and duration.
- The LED Bar Graph lights up proportionally to the frequency of the current note being played.
- The 8x8 LED Matrix visualizes the audio frequencies using dynamic bar heights and simulated harmonics.
- The pushbuttons allow the user to smoothly switch between different songs in the playlist.

CODE

```
#include <LedControl.h>

LedControl matrix = LedControl(11, 13, 10, 1);

const int ledPins[] = {2, 3, 4, 5, 6, 7, A0, A1, A2, A3};
const int ledCount = 10;
const int buzzerPin = 12;
```

```
const int btnNext = 8;
const int btnPrev = 9;

#define NOTE_B0  31
#define NOTE_C1  33
#define NOTE_CS1 35
#define NOTE_D1  37
#define NOTE_DS1 39
#define NOTE_E1  41
#define NOTE_F1  44
#define NOTE_FS1 46
#define NOTE_G1  49
#define NOTE_GS1 52
#define NOTE_A1  55
#define NOTE_AS1 58
#define NOTE_B1  62
#define NOTE_C2  65
#define NOTE_CS2 69
#define NOTE_D2  73
#define NOTE_DS2 78
#define NOTE_E2  82
#define NOTE_F2  87
#define NOTE_FS2 93
#define NOTE_G2  98
#define NOTE_GS2 104
#define NOTE_A2  110
#define NOTE_AS2 117
#define NOTE_B2  123
#define NOTE_C3  131
#define NOTE_CS3 139
#define NOTE_D3  147
#define NOTE_DS3 156
#define NOTE_E3  165
#define NOTE_F3  175
#define NOTE_FS3 185
#define NOTE_G3  196
#define NOTE_GS3 208
#define NOTE_A3  220
#define NOTE_AS3 233
#define NOTE_B3  247
#define NOTE_C4  262
#define NOTE_CS4 277
#define NOTE_D4  294
#define NOTE_DS4 311
#define NOTE_E4  330
#define NOTE_F4  349
#define NOTE_FS4 370
#define NOTE_G4  392
```

```
#define NOTE_GS4 415
#define NOTE_A4  440
#define NOTE_AS4 466
#define NOTE_B4  494
#define NOTE_C5  523
#define NOTE_CS5 554
#define NOTE_D5  587
#define NOTE_DS5 622
#define NOTE_E5  659
#define NOTE_F5  698
#define NOTE_FS5 740
#define NOTE_G5  784
#define NOTE_GS5 831
#define NOTE_A5  880
#define NOTE_AS5 932
#define NOTE_B5  988
#define NOTE_C6  1047
#define NOTE_CS6 1109
#define NOTE_D6  1175
#define NOTE_DS6 1245
#define NOTE_E6  1319
#define NOTE_F6  1397
#define NOTE_FS6 1480
#define NOTE_G6  1568
#define NOTE_GS6 1661
#define NOTE_A6  1760
#define NOTE_AS6 1865
#define NOTE_B6  1976
#define NOTE_C7  2093
#define NOTE_CS7 2217
#define NOTE_D7  2349
#define NOTE_DS7 2489
#define NOTE_E7  2637
#define NOTE_F7  2794
#define NOTE_FS7 2960
#define NOTE_G7  3136
#define NOTE_GS7 3322
#define NOTE_A7  3520
#define NOTE_AS7 3729
#define NOTE_B7  3951
#define NOTE_C8  4186
#define NOTE_CS8 4435
#define NOTE_D8  4699
#define NOTE_DS8 4978
#define REST 0

struct Note {
  int frequency;
```

```
  int duration;
};

struct Song {
  const char* name;
  const char* album;
  Note* notes;
  int length;
};

Note marioTheme[] = {
  {NOTE_E5, 150}, {NOTE_E5, 150}, {REST, 150}, {NOTE_E5, 150},
  {REST, 150}, {NOTE_C5, 150}, {NOTE_E5, 150}, {REST, 150},
  {NOTE_G5, 150}, {REST, 450}, {NOTE_G4, 150}, {REST, 450},

  {NOTE_C5, 150}, {REST, 300}, {NOTE_G4, 150}, {REST, 300},
  {NOTE_E4, 150}, {REST, 300}, {NOTE_A4, 150}, {REST, 150},
  {NOTE_B4, 150}, {REST, 150}, {NOTE_AS4, 150}, {NOTE_A4, 150},
  {REST, 150},

  {NOTE_G4, 120}, {NOTE_E5, 120}, {NOTE_G5, 120}, {NOTE_A5, 150},
  {REST, 150}, {NOTE_F5, 150}, {NOTE_G5, 150}, {REST, 150},
  {NOTE_E5, 150}, {REST, 150}, {NOTE_C5, 150}, {NOTE_D5, 150},
  {NOTE_B4, 150}, {REST, 300}
};

Note marioUnderworld[] = {
  {NOTE_C4, 150}, {NOTE_C5, 150}, {NOTE_A3, 150}, {NOTE_A4, 150},
  {NOTE_AS3, 150}, {NOTE_AS4, 150}, {REST, 300},

  {NOTE_C4, 150}, {NOTE_C5, 150}, {NOTE_A3, 150}, {NOTE_A4, 150},
  {NOTE_AS3, 150}, {NOTE_AS4, 150}, {REST, 300},

  {NOTE_F3, 150}, {NOTE_F4, 150}, {NOTE_D3, 150}, {NOTE_D4, 150},
  {NOTE_DS3, 150}, {NOTE_DS4, 150}, {REST, 300},

  {NOTE_F3, 150}, {NOTE_F4, 150}, {NOTE_D3, 150}, {NOTE_D4, 150},
  {NOTE_DS3, 150}, {NOTE_DS4, 150}, {REST, 300},

  {NOTE_DS4, 225}, {NOTE_CS4, 225}, {NOTE_D4, 225},
  {NOTE_CS4, 150}, {NOTE_DS4, 150}, {NOTE_DS4, 150}, {NOTE_GS3, 150},
  {NOTE_G3, 150}, {NOTE_CS4, 150}
};

Note marioPowerUp[] = {
  {NOTE_G4, 100}, {NOTE_B4, 100}, {NOTE_D5, 100}, {NOTE_G5, 100},
  {NOTE_B5, 100}, {NOTE_G5, 100}, {NOTE_B5, 100}, {NOTE_D6, 100},
  {NOTE_G6, 100}, {NOTE_B6, 100}, {NOTE_G6, 100}, {NOTE_B6, 100},
```

```
  {NOTE_D7, 100}, {REST, 200}
};

Note marioCoin[] = {
  {NOTE_B5, 100}, {NOTE_E6, 400}, {REST, 200}
};

Note marioGameOver[] = {
  {NOTE_C5, 150}, {NOTE_G4, 150}, {NOTE_E4, 150},
  {NOTE_A4, 225}, {NOTE_B4, 225}, {NOTE_A4, 225},
  {NOTE_GS4, 225}, {NOTE_AS4, 225}, {NOTE_GS4, 225},
  {NOTE_G4, 150}, {NOTE_D4, 150}, {NOTE_E4, 150}, {REST, 300}
};

Note marioJump[] = {
  {NOTE_C6, 75}, {NOTE_E6, 75}, {NOTE_G6, 75}, {NOTE_C7, 75}
};

Note zeldaTheme[] = {
  {NOTE_AS4, 200}, {NOTE_F4, 75}, {NOTE_F4, 75}, {NOTE_AS4, 150},
  {NOTE_GS4, 200}, {NOTE_FS4, 200}, {NOTE_GS4, 200},
  {NOTE_AS4, 300}, {NOTE_FS4, 150}, {NOTE_GS4, 150},
  {NOTE_AS4, 200}, {NOTE_FS4, 75}, {NOTE_FS4, 75},

  {NOTE_AS4, 150}, {NOTE_A4, 200}, {NOTE_F4, 200},
  {NOTE_AS4, 400}, {REST, 200},

  {NOTE_AS4, 200}, {NOTE_F4, 75}, {NOTE_F4, 75}, {NOTE_AS4, 150},
  {NOTE_GS4, 200}, {NOTE_FS4, 200}, {NOTE_GS4, 200},
  {NOTE_AS4, 300}, {NOTE_FS4, 150}, {NOTE_GS4, 150},
  {NOTE_AS4, 200}, {NOTE_FS4, 75}, {NOTE_FS4, 75},

  {NOTE_AS4, 150}, {NOTE_CS5, 200}, {NOTE_DS5, 200},
  {NOTE_E5, 200}, {NOTE_FS5, 150}, {NOTE_E5, 150},
  {NOTE_DS5, 200}, {NOTE_CS5, 150}, {NOTE_DS5, 150},
  {NOTE_E5, 200}, {NOTE_CS5, 150}, {NOTE_DS5, 150},

  {NOTE_E5, 150}, {NOTE_CS5, 150}, {NOTE_C5, 150}, {NOTE_CS5, 150},
  {NOTE_D5, 200}, {NOTE_E5, 200}, {NOTE_FS5, 150},
  {NOTE_E5, 150}, {NOTE_DS5, 200}, {NOTE_CS5, 150},
  {NOTE_DS5, 150}, {NOTE_E5, 200}, {NOTE_CS5, 150},

  {NOTE_DS5, 150}, {NOTE_E5, 150}, {NOTE_CS5, 150},
  {NOTE_C5, 150}, {NOTE_CS5, 150}, {NOTE_D5, 200},
  {NOTE_E5, 200}, {NOTE_FS5, 150}, {NOTE_E5, 150},
  {NOTE_DS5, 200}, {NOTE_CS5, 150}, {NOTE_DS5, 150},

  {NOTE_E5, 200}, {NOTE_CS5, 150}, {NOTE_C5, 200},
```

```
  {NOTE_CS5, 600}, {REST, 200}
};

Song playlist[] = {
  {"Main Theme", "Super Mario Bros", marioTheme, 38},
  {"Underworld", "Super Mario Bros", marioUnderworld, 43},
  {"Power Up", "Super Mario Bros", marioPowerUp, 14},
  {"Coin Sound", "Super Mario Bros", marioCoin, 3},
  {"Game Over", "Super Mario Bros", marioGameOver, 13},
  {"Jump Sound", "Super Mario Bros", marioJump, 4},
  {"Main Theme", "Legend of Zelda", zeldaTheme, 70}
};

int currentSong = 0;
int totalSongs = 7;
int noteIndex = 0;
unsigned long lastNote = 0;
bool isPlaying = false;
int barHeights[8] = {0, 0, 0, 0, 0, 0, 0, 0};
int targetHeights[8] = {0, 0, 0, 0, 0, 0, 0, 0};

void setup() {
  for (int i = 0; i < ledCount; i++) {
    pinMode(ledPins[i], OUTPUT);
    digitalWrite(ledPins[i], LOW);
  }
  pinMode(btnNext, INPUT_PULLUP);
  pinMode(btnPrev, INPUT_PULLUP);
  pinMode(buzzerPin, OUTPUT);

  matrix.shutdown(0, false);
  matrix.setIntensity(0, 8);
  matrix.clearDisplay(0);

  isPlaying = true;
}

void loop() {
  if (digitalRead(btnNext) == LOW) {
    stopSong();
    currentSong = (currentSong + 1) % totalSongs;
    resetSong();
    delay(300);
  }

  if (digitalRead(btnPrev) == LOW) {
    stopSong();
    currentSong = (currentSong - 1 + totalSongs) % totalSongs;
```

```
    resetSong();
    delay(300);
  }

  if (isPlaying) {
    Song* song = &playlist[currentSong];

    if (millis() - lastNote >= song->notes[noteIndex].duration) {
      playCurrentNote();
      noteIndex++;

      if (noteIndex >= song->length) {
        noteIndex = 0;
      }
      lastNote = millis();
    }
  }

  updateMatrixBars();
}

void playCurrentNote() {
  Song* song = &playlist[currentSong];
  int freq = song->notes[noteIndex].frequency;

  if (freq == REST) {
    noTone(buzzerPin);
    updateLEDBar(0);
    for (int i = 0; i < 8; i++) {
      targetHeights[i] = 0;
    }
  } else {
    tone(buzzerPin, freq);

    int level = map(freq, 31, 4978, 1, ledCount);
    level = constrain(level, 1, ledCount);
    updateLEDBar(level);

    updateMatrixTargets(freq, level);
  }
}

void updateMatrixTargets(int freq, int level) {
  int baseHeight = map(level, 0, ledCount, 0, 8);

  for (int i = 0; i < 8; i++) {
    int variance = random(-2, 3);
    targetHeights[i] = constrain(baseHeight + variance, 0, 8);
```

```
  }

  int harmonic1 = map(freq % 500, 0, 500, 0, 8);
  int harmonic2 = map((freq * 2) % 800, 0, 800, 0, 8);

  targetHeights[1] = constrain(targetHeights[1] + harmonic1 / 2, 0, 8);
  targetHeights[4] = constrain(targetHeights[4] + harmonic2 / 2, 0, 8);
  targetHeights[6] = constrain(targetHeights[6] + harmonic1 / 3, 0, 8);
}

void updateMatrixBars() {
  matrix.clearDisplay(0);

  for (int col = 0; col < 8; col++) {
    if (barHeights[col] < targetHeights[col]) {
      barHeights[col]++;
    } else if (barHeights[col] > targetHeights[col]) {
      barHeights[col]--;
    }

    for (int row = 0; row < 8; row++) {
      if (row < barHeights[col]) {
        matrix.setLed(0, 7 - row, col, true);
      }
    }
  }
}

void updateLEDBar(int level) {
  for (int i = 0; i < ledCount; i++) {
    if (i < level) {
      digitalWrite(ledPins[i], HIGH);
    } else {
      digitalWrite(ledPins[i], LOW);
    }
  }
}

void stopSong() {
  noTone(buzzerPin);
  for (int i = 0; i < ledCount; i++) {
    digitalWrite(ledPins[i], LOW);
  }
  for (int i = 0; i < 8; i++) {
    targetHeights[i] = 0;
    barHeights[i] = 0;
  }
  matrix.clearDisplay(0);
```

```
}

void resetSong() {
  noteIndex = 0;
  matrix.clearDisplay(0);
  isPlaying = true;
  for (int i = 0; i < 8; i++) {
    targetHeights[i] = 0;
    barHeights[i] = 0;
  }
}
```

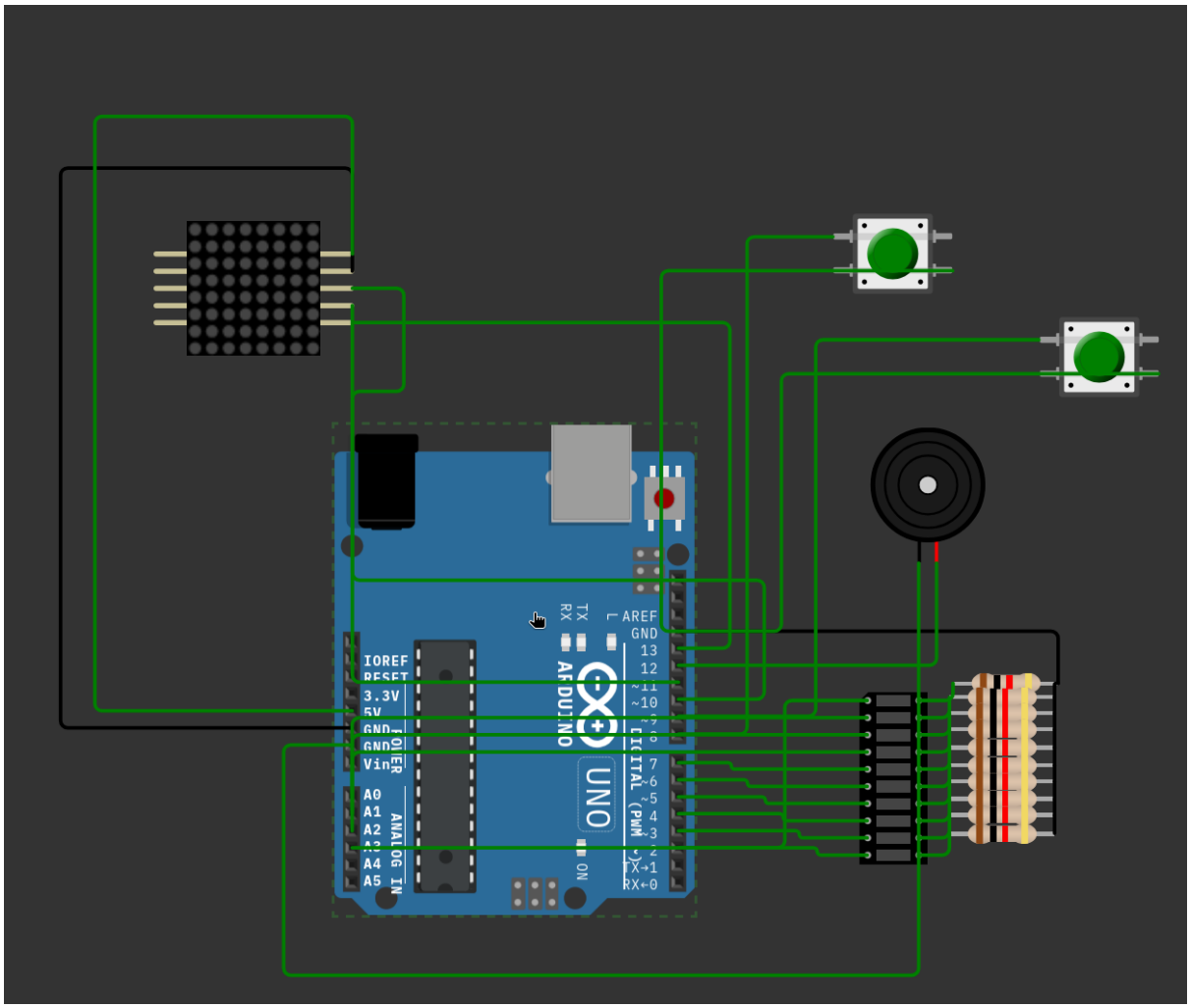
JSON

```
{
  "version": 1,
  "author": "Uri Shaked",
  "editor": "wokwi",
  "parts": [
    {
      "type": "wokwi-arduino-uno",
      "id": "uno",
      "top": 208.2,
      "left": -190.6,
      "rotate": 90,
      "attrs": {}
    },
    {
      "type": "wokwi-led-bar-graph",
      "id": "bargraph1",
      "top": 321.6,
      "left": 139.2,
      "attrs": { "color": "BCYR" }
    },
    { "type": "wokwi-resistor", "id": "r1", "top": 311.15, "left": 192,
     ↪  "attrs": {} },
    { "type": "wokwi-resistor", "id": "r2", "top": 319, "left": 190,
     ↪  "attrs": {} },
    { "type": "wokwi-resistor", "id": "r3", "top": 328, "left": 190,
     ↪  "attrs": {} },
    { "type": "wokwi-resistor", "id": "r4", "top": 337, "left": 190,
     ↪  "attrs": {} },
    { "type": "wokwi-resistor", "id": "r5", "top": 347, "left": 190,
     ↪  "attrs": {} },
    { "type": "wokwi-resistor", "id": "r6", "top": 357, "left": 190,
     ↪  "attrs": {} },
    { "type": "wokwi-resistor", "id": "r7", "top": 366, "left": 190,
     ↪  "attrs": {} },
```

```
    { "type": "wokwi-resistor", "id": "r8", "top": 376, "left": 190,
     ↪  "attrs": {} },
    { "type": "wokwi-resistor", "id": "r9", "top": 385, "left": 190,
     ↪  "attrs": {} },
    { "type": "wokwi-resistor", "id": "r10", "top": 395, "left": 190,
     ↪  "attrs": {} },
    {
      "type": "wokwi-max7219-matrix",
      "id": "matrix1",
      "top": 58.2,
      "left": -255.39,
      "attrs": { "chain": "1" }
    },
    {
      "type": "wokwi-buzzer",
      "id": "bz1",
      "top": 165.6,
      "left": 145.8,
      "attrs": { "volume": "0.1" }
    },
    {
      "type": "wokwi-pushbutton",
      "id": "btn1",
      "top": 54.2,
      "left": 124.8,
      "attrs": { "color": "green", "xray": "1" }
    },
    {
      "type": "wokwi-pushbutton",
      "id": "btn2",
      "top": 111.8,
      "left": 240,
      "attrs": { "color": "green", "xray": "1" }
    }
  ],
  "connections": [
    [ "bargraph1:A10", "uno:2", "green", [ "h-28", "v31" ] ],
    [ "bargraph1:A9", "uno:3", "green", [ "h-38", "v31" ] ],
    [ "bargraph1:A8", "uno:4", "green", [ "h-47", "v31" ] ],
    [ "bargraph1:A7", "uno:5", "green", [ "h-57", "v31" ] ],
    [ "bargraph1:A6", "uno:6", "green", [ "h-67", "v31" ] ],
    [ "bargraph1:A5", "uno:7", "green", [ "h-76", "v31" ] ],
    [ "r1:1", "bargraph1:C1", "green", [ "v0" ] ],
    [ "r2:1", "bargraph1:C2", "green", [ "v0" ] ],
    [ "r3:1", "bargraph1:C3", "green", [ "v0" ] ],
    [ "r4:1", "bargraph1:C4", "green", [ "v0" ] ],
    [ "r5:1", "bargraph1:C5", "green", [ "v0" ] ],
    [ "r6:1", "bargraph1:C6", "green", [ "v0" ] ],
```

```
    [ "r7:1", "bargraph1:C7", "green", [ "v0" ] ],
    [ "r8:1", "bargraph1:C8", "green", [ "v0" ] ],
    [ "r9:1", "bargraph1:C9", "green", [ "v0" ] ],
    [ "r10:1", "bargraph1:C10", "green", [ "v0" ] ],
    [ "r10:2", "r9:2", "black", [ "v0" ] ],
    [ "r8:2", "r9:2", "black", [ "v0" ] ],
    [ "r8:2", "r7:2", "black", [ "v0" ] ],
    [ "r7:2", "r6:2", "black", [ "v0" ] ],
    [ "r6:2", "r5:2", "black", [ "v0" ] ],
    [ "r5:2", "r4:2", "black", [ "v0" ] ],
    [ "r4:2", "r3:2", "black", [ "v0" ] ],
    [ "r3:2", "r2:2", "black", [ "v0" ] ],
    [ "r2:2", "r1:2", "black", [ "v0" ] ],
    [ "uno:GND.1", "r1:2", "black", [ "h0" ] ],
    [ "matrix1:GND", "uno:GND.2", "black", [ "v-57.6", "h-393.6",
     ↪  "v312.9" ] ],
    [ "matrix1:V+", "uno:5V", "green", [ "v-76.8", "h-364.8", "v332.2" ]
     ↪  ],
    [ "matrix1:DIN", "uno:11", "green", [ "h28.8", "v57.6", "h-28.8",
     ↪  "v162.6" ] ],
    [ "matrix1:CS", "uno:10", "green", [ "v153.6", "h230.4", "v66.5" ] ],
    [ "matrix1:CLK", "uno:13", "green", [ "h211.2", "v182" ] ],
    [ "bz1:2", "uno:12", "green", [ "v0" ] ],
    [ "bz1:1", "uno:GND.3", "green", [ "v230.4", "h-355.2", "v-128.8" ]
     ↪  ],
    [ "bargraph1:A4", "uno:A0", "green", [ "h0" ] ],
    [ "bargraph1:A3", "uno:A1", "green", [ "h0" ] ],
    [ "bargraph1:A2", "uno:A2", "green", [ "h0" ] ],
    [ "bargraph1:A1", "uno:A3", "green", [ "h-46.08", "v90.86" ] ],
    [ "btn1:1.l", "uno:8", "green", [ "h-48", "v278" ] ],
    [ "btn2:1.l", "uno:9", "green", [ "h-124.8", "v210.4" ] ],
    [ "btn1:2.r", "uno:GND.1", "green", [ "h-163", "v77" ] ],
    [ "btn2:2.r", "uno:GND.1", "green", [ "h-211", "v143.9" ] ]
  ],
  "dependencies": {}
}
```

PIN CONFIGURATION



EX.NO: 5	MOTION DETECTION AND PHOTO RESISTOR
30/01/2026	

AIM

To simulate the motion detection and usage of photo resistor

APPARATUS REQUIRED

- Arduino Uno
- Breadboard
- PIR Motion Sensor
- Photoresistor (LDR) Sensor Module
- LEDs (x3) (White, Cyan, Orange)
- 220-ohm Resistors (x3)
- Jumper wires

PRE-CONDITIONS

- The PIR sensor, LDR module, and LEDs (with resistors) are correctly wired to the designated Arduino pins.
- The LDR module's analog (AO) and digital (DO) outputs are properly connected.
- The code is successfully compiled and uploaded to the Arduino Uno board.
- Serial Monitor is opened at 9600 baud rate to view the debug messages.

POST-CONDITIONS

- The White LED activates when the PIR sensor detects motion.
- The Cyan LED's brightness dynamically adjusts (PWM) based on the ambient light detected by the LDR's analog pin.
- The Orange LED activates exclusively when the LDR's digital threshold drops indicating a dark environment.
- The Serial Monitor displays continuous real-time status of motion detection, calculated lux levels, and system diagnostics.

CODE

```
const int PIR_PIN = 2;
const int PHOTO_DIGITAL_PIN = 3;
const int PHOTO_ANALOG_PIN = A0;

const int MOTION_LED = 8;   // White LED
const int ANALOG_LED = 9;   // Blue LED
const int DIGITAL_LED = 10; // Orange LED
```

```
const int LIGHT_THRESHOLD = 512;

void setup()
{
  Serial.begin(9600);

  pinMode(PIR_PIN, INPUT);
  pinMode(PHOTO_DIGITAL_PIN, INPUT);
  pinMode(MOTION_LED, OUTPUT);
  pinMode(ANALOG_LED, OUTPUT);
  pinMode(DIGITAL_LED, OUTPUT);

  Serial.println("========================================");
  Serial.println("Motion & Photoresistor Sensor Demo");
  Serial.println("========================================");
  Serial.println("White LED (Pin 8)  - Motion Detection");
  Serial.println("Blue LED (Pin 9)   - Analog Light Level");
  Serial.println("Orange LED (Pin 10) - Digital Light Status");
  Serial.println("========================================");
  delay(2000);
  Serial.println("System Ready!\n");
}

void loop()
{
  Serial.println("========================================");

  // PIR Motion Sensor
  int motionDetected = digitalRead(PIR_PIN);

  if (motionDetected == HIGH)
  {
    digitalWrite(MOTION_LED, HIGH);
    Serial.println(">>> MOTION DETECTED! <<<");
  }
  else
  {
    digitalWrite(MOTION_LED, LOW);
    Serial.println("Motion: None");
  }

  // Photoresistor Analog Output
  int analogValue = analogRead(PHOTO_ANALOG_PIN);

  // Exponential curve for visible brightness changes
  float normalizedValue = (1023.0 - analogValue) / 1023.0;
  float gamma = 2.5;
  float correctedValue = pow(normalizedValue, gamma);
```

```
int ledBrightness = (int)(correctedValue * 255.0);
ledBrightness = constrain(ledBrightness, 0, 255);

analogWrite(ANALOG_LED, ledBrightness);

// Convert to lux
const float GAMMA_LUX = 0.7;
const float RL10 = 50;
float voltage = analogValue / 1024.0 * 5.0;
float resistance = 2000.0 * voltage / (1.0 - voltage / 5.0);
float lux = pow(RL10 * 1e3 * pow(10, GAMMA_LUX) / resistance, (1.0 /
 ↪  GAMMA_LUX));

Serial.println("\n--- ANALOG OUTPUT (Blue LED) ---");
Serial.print("Raw Value: ");
Serial.print(analogValue);
Serial.print(" | Voltage: ");
Serial.print(voltage, 2);
Serial.print("V | Light: ");

if (isfinite(lux))
{
  Serial.print((int)lux);
  Serial.println(" lux");
}
else
{
  Serial.println("Very Bright");
}

Serial.print("Brightness: ");
Serial.print(ledBrightness);
Serial.print("/255 (");
Serial.print((ledBrightness * 100) / 255);
Serial.print("%) ");

if (ledBrightness < 25)
{
  Serial.println("- OFF/VERY DIM");
}
else if (ledBrightness < 80)
{
  Serial.println("- DIM");
}
else if (ledBrightness < 150)
{
  Serial.println("- MEDIUM");
}
```

```
else if (ledBrightness < 220)
{
  Serial.println("- BRIGHT");
}
else
{
  Serial.println("- VERY BRIGHT");
}

Serial.print("Bar: [");
int bars = map(ledBrightness, 0, 255, 0, 20);
for (int i = 0; i < bars; i++)
  Serial.print(" ");
for (int i = bars; i < 20; i++)
  Serial.print(" ");
Serial.println("]");

// Photoresistor Digital Output
int digitalValue = digitalRead(PHOTO_DIGITAL_PIN);

if (digitalValue == LOW)
{
  digitalWrite(DIGITAL_LED, HIGH);
  Serial.println("\n--- DIGITAL OUTPUT (Orange LED) ---");
  Serial.println("Status: BRIGHT | Orange LED: ON");
}
else
{
  digitalWrite(DIGITAL_LED, LOW);
  Serial.println("\n--- DIGITAL OUTPUT (Orange LED) ---");
  Serial.println("Status: DARK | Orange LED: OFF");
}

// Custom Threshold
Serial.println("\n--- CUSTOM THRESHOLD ---");
if (analogValue > LIGHT_THRESHOLD)
{
  Serial.print("Value (");
  Serial.print(analogValue);
  Serial.print(") > (");
  Serial.print(LIGHT_THRESHOLD);
  Serial.println(") = DARK");
}
else
{
  Serial.print("Value (");
  Serial.print(analogValue);
  Serial.print(")   (");
```

```
    Serial.print(LIGHT_THRESHOLD);
    Serial.println(") = BRIGHT");
  }

  Serial.println("========================================\n");
  delay(1000);
}
```

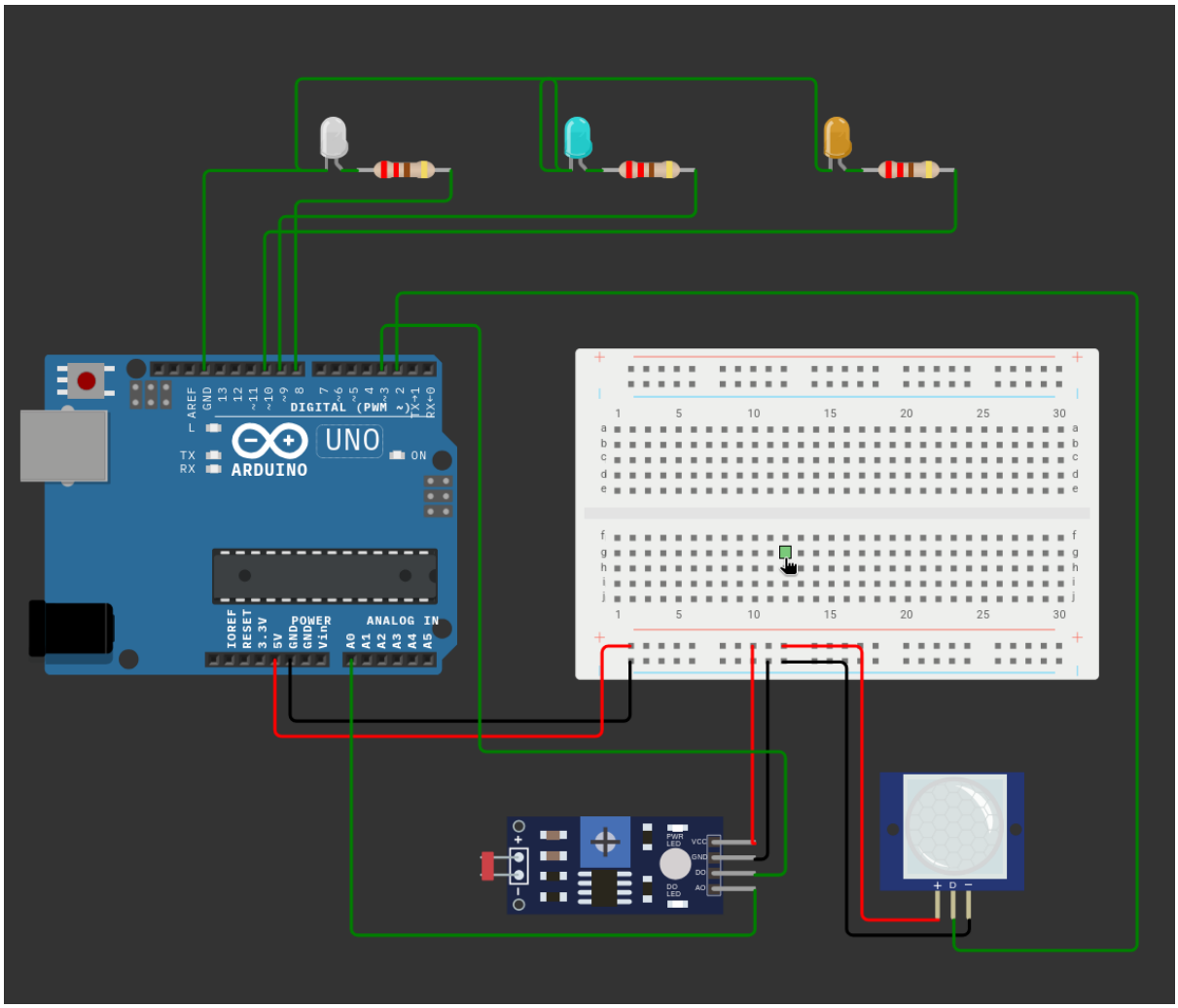
JSON

```
{
  "version": 1,
  "author": "Shree Kottes",
  "editor": "wokwi",
  "parts": [
    { "type": "wokwi-breadboard-half", "id": "bb1", "top": -60.6, "left":
     ↪  233.2, "attrs": {} },
    { "type": "wokwi-arduino-uno", "id": "uno", "top": -57, "left":
     ↪  -115.8, "attrs": {} },
    { "type": "wokwi-pir-motion-sensor", "id": "pir1", "top": 205.6,
     ↪  "left": 424.62, "attrs": {} },
    {
      "type": "wokwi-photoresistor-sensor",
      "id": "ldr1",
      "top": 233.6,
      "left": 173.6,
      "attrs": {}
    },
    {
      "type": "wokwi-led",
      "id": "led1",
      "top": -214.8,
      "left": 61.4,
      "attrs": { "color": "white" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r1",
      "top": -178.45,
      "left": 96,
      "attrs": { "value": "220" }
    },
    {
      "type": "wokwi-led",
      "id": "led2",
      "top": -214.8,
      "left": 215,
```

```
      "attrs": { "color": "cyan" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r2",
      "top": -178.45,
      "left": 249.6,
      "attrs": { "value": "220" }
    },
    {
      "type": "wokwi-led",
      "id": "led3",
      "top": -214.8,
      "left": 378.2,
      "attrs": { "color": "orange" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r3",
      "top": -178.45,
      "left": 412.8,
      "attrs": { "value": "220" }
    }
  ],
  "connections": [
    [ "uno:GND.2", "bb1:bn.1", "black", [ "v38.3", "h213.5" ] ],
    [ "uno:5V", "bb1:bp.1", "red", [ "v47.9", "h205.4", "v-56.7" ] ],
    [ "pir1:VCC", "bb1:bp.10", "red", [ "h-48", "v-171.9" ] ],
    [ "pir1:GND", "bb1:bn.10", "black", [ "v9.6", "h-77.06", "v-171.5" ]
     ↪ ],
    [ "pir1:OUT", "uno:2", "green", [ "v19.2", "h115.06", "v-412.8",
     ↪ "h-464.9" ] ],
    [ "ldr1:VCC", "bb1:bp.8", "red", [ "h0" ] ],
    [ "ldr1:GND", "bb1:bn.9", "black", [ "h0" ] ],
    [ "ldr1:DO", "uno:3", "green", [ "h19.2", "v-77.4", "h-192",
     ↪ "v-267.6", "h-61.6" ] ],
    [ "ldr1:AO", "uno:A0", "green", [ "v28.1", "h-253.4" ] ],
    [ "led1:A", "r1:1", "green", [ "v0" ] ],
    [ "led2:A", "r2:1", "green", [ "v0" ] ],
    [ "led3:A", "r3:1", "green", [ "v0" ] ],
    [ "led1:C", "led2:C", "green", [ "v0", "h-18.8", "v-57.6", "h163.2",
     ↪ "v57.6" ] ],
    [ "led2:C", "led3:C", "green", [ "v0", "h-18.8", "v-57.6", "h172.8",
     ↪ "v57.6" ] ],
    [ "r1:2", "uno:8", "green", [ "v19.2", "h-1.2" ] ],
    [ "r2:2", "uno:9", "green", [ "v28.8", "h-1.2" ] ],
    [ "r3:2", "uno:10", "green", [ "v38.4", "h-433.2" ] ],
    [ "led1:C", "uno:GND.1", "green", [ "v0", "h-76.4" ] ]
```

```
  ],
  "dependencies": {}
}
```

PIN CONFIGURATION



EX.NO: 6	GAS LEAKAGE DETECTION
06/02/2026	

AIM

To simulate the gas leakage detection

APPARATUS REQUIRED

- Arduino Uno
- Gas Sensor (MQ Series)
- RGB LED
- Buzzer
- Relay Module
- I2C OLED Display (128x64)
- 200-ohm Resistors (x3)
- Jumper wires

PRE-CONDITIONS

- The Gas Sensor, RGB LED, Buzzer, Relay module, and I2C OLED display are correctly wired to the Arduino.
- Libraries `Wire.h`, `Adafruit_GFX.h`, and `Adafruit_SSD1306.h` are installed in the Arduino IDE.
- The code is successfully compiled and uploaded to the Arduino board.
- The sensor has had appropriate time to preheat if dealing with a real hardware MQ sensor.

POST-CONDITIONS

- The OLED screen displays the real-time gas concentration values and system status (SAFE, WARNING, CRITICAL).
- In a SAFE state, the RGB LED glows green, the buzzer is off, and the relay (exhaust) is deactivated.
- In a WARNING state, the RGB LED glows yellow and the buzzer beeps intermittently.
- In a CRITICAL state, the RGB LED glows red, the buzzer sounds a continuous alarm, and the relay activates the exhaust fan.

CODE

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
```

```
// OLED Configuration
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET     -1
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

const int gasSensorPin = A0;
const int buzzerPin = 8;
const int redPin = 11;
const int greenPin = 10;
const int bluePin = 9;
const int relayPin = 6;    // NEW: Relay Control Pin

const int SAFE_LIMIT = 350;
const int WARNING_LIMIT = 600;

int gasValue = 0;

void setup() {
  Serial.begin(9600);

  pinMode(gasSensorPin, INPUT);
  pinMode(buzzerPin, OUTPUT);
  pinMode(redPin, OUTPUT);
  pinMode(greenPin, OUTPUT);
  pinMode(bluePin, OUTPUT);
  pinMode(relayPin, OUTPUT); // NEW: Set Relay as output

  // Start with Relay OFF (Active LOW usually means LOW = ON, but let's
  ↪  assume HIGH = OFF)
  digitalWrite(relayPin, HIGH);

  // Initialize OLED
  if(!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println(F("SSD1306 allocation failed"));
    for(;;);
  }

  display.clearDisplay();
  display.setTextSize(1);
  display.setTextColor(WHITE);
  display.setCursor(0,0);
  display.println("Sys Initialize...");
  display.println("Exhaust: OFF");
  display.display();
  delay(2000);

  setColor(0, 255, 0);
```

```
  digitalWrite(buzzerPin, LOW);
}

void loop() {
  gasValue = analogRead(gasSensorPin);

  display.clearDisplay();
  display.setCursor(0,0);

  // Header
  display.setTextSize(1);
  display.println("Gas Safety Unit");
  display.println("----------------");
  display.print("Gas Val: ");
  display.print(gasValue);
  display.println("   ");

  if (gasValue >= WARNING_LIMIT) {
    // --- DANGER: ACTIVATE SAFETY SYSTEMS ---

    display.setTextColor(BLACK, WHITE);
    display.println(" ALERT: CRITICAL");
    display.setTextColor(WHITE);

    // SAFETY ACTION: Turn Relay ON (Active LOW logic used here)
    digitalWrite(relayPin, LOW);
    display.println(">> EXHAUST: ON <<"); // Fan is spinning

    setColor(255, 0, 0);

    // Alarm
    tone(buzzerPin, 1000);
    delay(200);
    noTone(buzzerPin);
    delay(200);

  } else if (gasValue >= SAFE_LIMIT) {
    // --- WARNING ---

    display.println(" ALERT: WARNING");

    // Keep Exhaust OFF to save power (or ON if you prefer constant
     ↪  ventilation)
    digitalWrite(relayPin, HIGH);
    display.println("Exhaust: STANDBY");

    setColor(255, 255, 0);
```

```
    tone(buzzerPin, 800);
    delay(150);
    noTone(buzzerPin);
    delay(850);

  } else {
    // --- SAFE ---

    display.println(" STATUS: SAFE");

    // SAFETY ACTION: Turn Relay OFF
    digitalWrite(relayPin, HIGH);
    display.println("Exhaust: OFF");

    setColor(0, 255, 0);

    digitalWrite(buzzerPin, LOW);
    delay(500);
  }

  display.display();
}

void setColor(int redValue, int greenValue, int blueValue) {
  analogWrite(redPin, 255 - redValue);
  analogWrite(greenPin, 255 - greenValue);
  analogWrite(bluePin, 255 - blueValue);
}
```

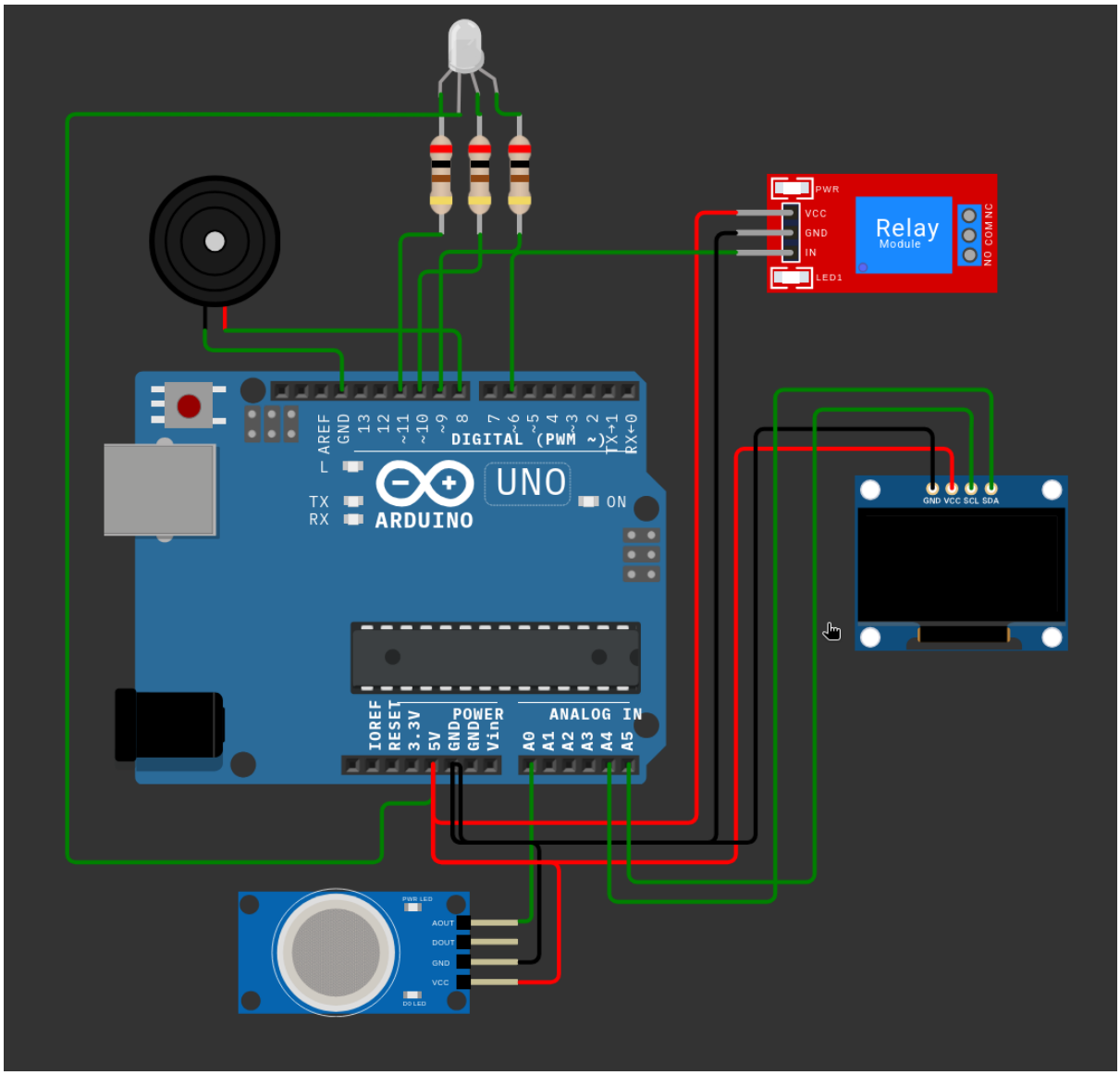
JSON

```
{
  "version": 1,
  "author": "Shree Kottes",
  "editor": "wokwi",
  "parts": [
    { "type": "wokwi-arduino-uno", "id": "uno", "top": 77.4, "left":
     ↪  -10.2, "attrs": {} },
    { "type": "wokwi-gas-sensor", "id": "gas1", "top": 329.1, "left": 55,
     ↪  "attrs": {} },
    { "type": "wokwi-rgb-led", "id": "rgb1", "top": -101.6, "left":
     ↪  145.1, "attrs": {} },
    {
      "type": "wokwi-buzzer",
      "id": "bz1",
      "top": -26.4,
      "left": 11.4,
```

```
      "attrs": { "volume": "0.1" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r1",
      "top": -24,
      "left": 162.65,
      "rotate": 90,
      "attrs": { "value": "200" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r2",
      "top": -24,
      "left": 124.25,
      "rotate": 90,
      "attrs": { "value": "200" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r3",
      "top": -24,
      "left": 143.45,
      "rotate": 90,
      "attrs": { "value": "200" }
    },
    { "type": "wokwi-relay-module", "id": "relay1", "top": -19, "left":
     ↪  297.6, "attrs": {} },
    {
      "type": "board-ssd1306",
      "id": "oled1",
      "top": 127.94,
      "left": 355.43,
      "attrs": { "i2cAddress": "0x3c" }
    }
  ],
  "connections": [
    [ "gas1:AOUT", "uno:A0", "green", [ "h0" ] ],
    [ "gas1:VCC", "uno:5V", "red", [ "h19.2", "v-58.5", "h-57.6" ] ],
    [ "gas1:GND", "uno:GND.2", "black", [ "h9.6", "v-58.4", "h-38.4" ] ],
    [ "bz1:1", "uno:GND.1", "green", [ "v9.6", "h57.6" ] ],
    [ "bz1:2", "uno:8", "green", [ "v0", "h114.8" ] ],
    [ "rgb1:R", "r2:1", "green", [ "v0" ] ],
    [ "rgb1:G", "r3:1", "green", [ "v0" ] ],
    [ "rgb1:B", "r1:1", "green", [ "v0" ] ],
    [ "r2:2", "uno:11", "green", [ "h0" ] ],
    [ "r3:2", "uno:10", "green", [ "v18", "h-29.5" ] ],
    [ "r1:2", "uno:9", "green", [ "h0", "v8.4", "h-38.4", "v67.2" ] ],
```

```
    [
      "rgb1:COM",
      "uno:5V",
      "green",
      [ "v-0.4", "h-191.9", "v364.8", "h153.6", "v-28.8", "h19.2" ]
    ],
    [ "relay1:VCC", "uno:5V", "red", [ "h-19.2", "v297.6", "h-124.8" ] ],
    [ "relay1:IN", "uno:6", "green", [ "h0" ] ],
    [ "relay1:GND", "uno:GND.2", "black", [ "h-9.6", "v297.2", "h-124.8"
     ↪  ] ],
    [ "oled1:VCC", "uno:5V", "red", [ "v-19.2", "h-105.45", "v201.6",
     ↪  "h-153.6" ] ],
    [ "oled1:GND", "uno:GND.2", "black", [ "v-28.8", "h-86.4", "v201.6",
     ↪  "h-144", "v-9.6" ] ],
    [
      "oled1:SCL",
      "uno:A5",
      "green",
      [ "v-28.8", "h0.3", "v-9.6", "h-76.8", "v230.4", "h-86.4" ]
    ],
    [ "oled1:SDA", "uno:A4", "green", [ "v-48", "h-105.53", "v249.6",
     ↪  "h-86.4" ] ]
  ],
  "dependencies": {}
}
```

PIN CONFIGURATION



EX.NO: 7	MQTT CLIENT SERVER COMMUNICATION
20/02/2026	

AIM

To demonstrate the client/server communication for sending and receiving a message using MQTT Broker.

APPARATUS REQUIRED

- Espressif ESP 32 dev module
- Mosquitto MQTT Server

PRE-CONDITIONS

POST-CONDITIONS

CODE

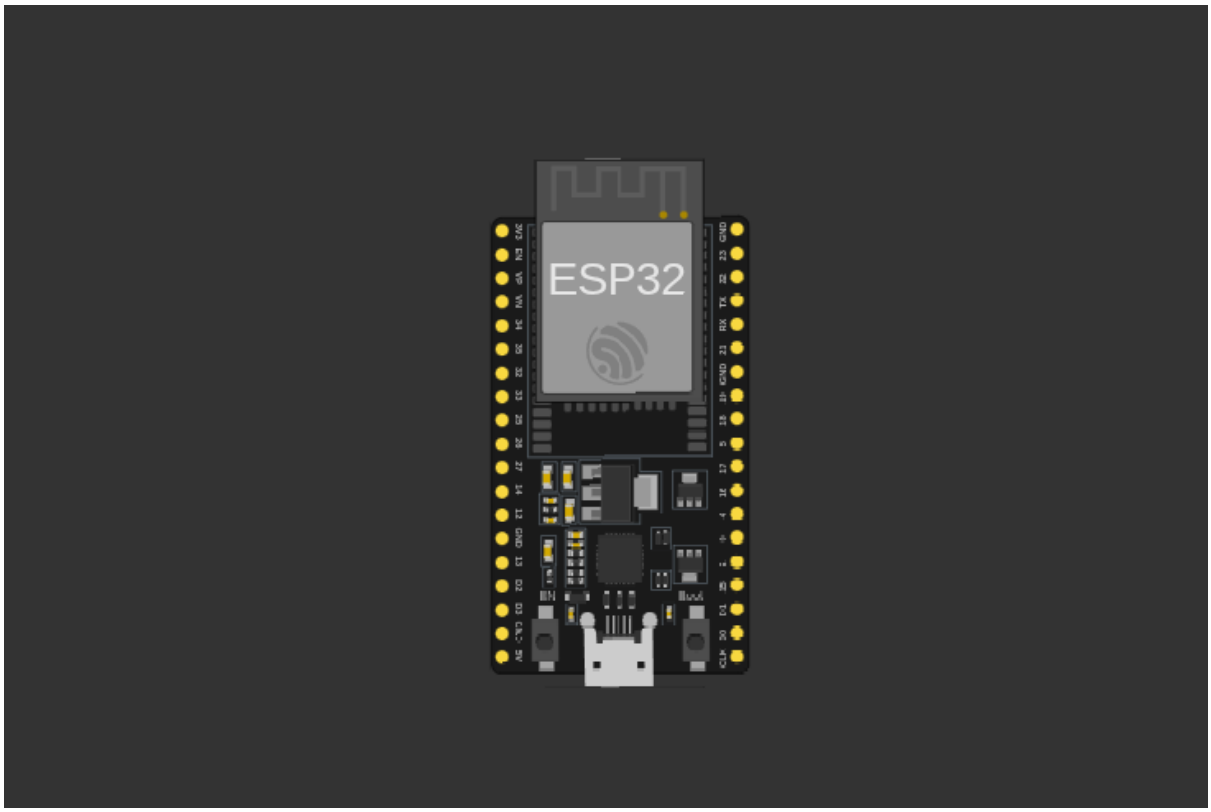
```
#include <PubSubClient.h>
#include <WiFi.h>

const char* MQTT_SERVER = "10.151.23.86";
const uint16_t MQTT_PORT = 1883;
const char* MQTT_CLIENT_ID = "wokwi-esp-client";

void onMsg(const char* topic, const byte* payload, unsigned int length) {
    Serial.print("[");
    Serial.print(topic);
    Serial.print("]: ");
    for (unsigned int i = 0; i < length; i++) {
        Serial.print((char)payload[i]);
    }
    Serial.println();
}

WiFiClient espClient;
PubSubClient mqtt_client(
    MQTT_SERVER,
    MQTT_PORT,
    onMsg,
    espClient
);

void connectWifi() {
    WiFi.begin("Wokwi-GUEST", "");
```

```
    while (WiFi.status() != WL_CONNECTED) {
        Serial.print(".");
        delay(500);
    }
    Serial.print("\nWifi Connected!");  // added semicolon
}

void connectMqtt() {
    while (!mqtt_client.connected()) {
        if (mqtt_client.connect(MQTT_CLIENT_ID)) {
            mqtt_client.subscribe("/home/krill");
            Serial.println("MQTT Server connected!");
        } else {
            Serial.print("Connection failed. RC: ");
            Serial.println(mqtt_client.state());
        }
    }
}

void setup() {
    Serial.begin(115200);
    connectWifi();
}

void loop() {
    if (!mqtt_client.connected())
        connectMqtt();
    mqtt_client.loop();
    mqtt_client.publish("/home/krill", "Krill is here.");  // added
 ↪  semicolon
    delay(10);
}
```

JSON

```
{
  "version": 1,
  "author": "Shree Kottes",
  "editor": "wokwi",
  "parts": [ { "type": "board-esp32-devkit-c-v4", "id": "esp", "top": 0,
   ↪  "left": 0, "attrs": {} } ],
  "connections": [ [ "esp:TX", "$serialMonitor:RX", "", [] ], [ "esp:RX",
   ↪  "$serialMonitor:TX", "", [] ] ],
  "dependencies": {}
}
```

PIN CONFIGURATION



EX.NO: 8	MQTT ARITHMETIC OPERATIONS
27/02/2026	

AIM

To perform simple arithmetic operations for two integer numbers using MQTT Broker.

APPARATUS REQUIRED

- Espressif ESP 32 Dev Module
- ILI9341 Cap Touch Display

PRE-CONDITIONS

POST-CONDITIONS

CODE

SERVER

```
#include <WiFi.h>
#include <PubSubClient.h>

const char *WIFI_SSID = "Wokwi-GUEST";
const char *WIFI_PASS = "";
const char *MQTT_BROKER = "100.64.0.2";
const int MQTT_PORT = 1883;
const char *MQTT_CLIENT = "esp32-calc";
const char *TOPIC_INPUT = "calc/input";
const char *TOPIC_RESULT = "calc/result";

WiFiClient wifiClient;
PubSubClient mqtt(wifiClient);

static const char *expr_ptr = nullptr; // current position in expression
 ↪  string

long parseExpr();
long parseTerm();
long parseFactor();
long compute(const char *expression, bool *error);
void parser(const char *raw, char *resultBuf, size_t bufLen);

/*
 * Grammar (handles precedence & parentheses):
 *   Expr   = Term   { ('+'|'-') Term   }
```

```
 *   Term   = Factor { ('*'|'/') Factor }
 *   Factor = '(' Expr ')' | ['-'] Number
 */

static void skipSpaces()
{
    while (*expr_ptr == ' ')
        expr_ptr++;
}

/* Lowest pecedence: addition & subtraction */
long parseExpr()
{
    long result = parseTerm();
    skipSpaces();
    while (*expr_ptr == '+' || *expr_ptr == '-')
    {
        char op = *expr_ptr++;
        long rhs = parseTerm();
        result = (op == '+') ? result + rhs : result - rhs;
        skipSpaces();
    }
    return result;
}

/* Middle precedence: multiplication & division */
long parseTerm()
{
    long result = parseFactor();
    skipSpaces();
    while (*expr_ptr == '*' || *expr_ptr == '/')
    {
        char op = *expr_ptr++;
        long rhs = parseFactor();
        if (op == '/' && rhs == 0)
        {
            // Signal divide-by-zero by poisoning the pointer
            expr_ptr = nullptr;
            return 0;
        }
        result = (op == '*') ? result * rhs : result / rhs;
        skipSpaces();
    }
    return result;
}

/* Highest precedence: parentheses & integer literals */
long parseFactor()
```

```
{
    skipSpaces();
    if (expr_ptr == nullptr)
        return 0;

    // Parenthesised sub-expression
    if (*expr_ptr == '(')
    {
        expr_ptr++; // consume '('
        long result = parseExpr();
        skipSpaces();
        if (*expr_ptr == ')')
            expr_ptr++; // consume ')'
        return result;
    }

    // Unary minus
    int sign = 1;
    if (*expr_ptr == '-')
    {
        sign = -1;
        expr_ptr++;
    }

    // Integer literal
    long num = 0;
    bool hasDigit = false;
    while (*expr_ptr >= '0' && *expr_ptr <= '9')
    {
        num = num * 10 + (*expr_ptr++ - '0');
        hasDigit = true;
    }

    if (!hasDigit)
        expr_ptr = nullptr; // unexpected character → error
    return sign * num;
}

/*
 * Entry point for evaluation.
 * Sets *error = true and returns 0 on any parse/math fault.
 */
long compute(const char *expression, bool *error)
{
    expr_ptr = expression;
    *error = false;

    long result = parseExpr();
```

```
    // Leftover characters or a poisoned pointer = malformed expression
    if (expr_ptr == nullptr || (*expr_ptr != '\0' && *expr_ptr != '\n'))
    {
        *error = true;
        return 0;
    }
    return result;
}

/*
 * Strips whitespace from raw MQTT payload, drives compute(),
 * and serialises the outcome into resultBuf.
 */
void parser(const char *raw, char *resultBuf, size_t bufLen)
{
    // Compact: copy non-space chars into a clean buffer
    char clean[128] = {};
    size_t j = 0;
    for (size_t i = 0; raw[i] && j < sizeof(clean) - 1; i++)
    {
        if (raw[i] != ' ')
            clean[j++] = raw[i];
    }

    bool error = false;
    long result = compute(clean, &error);

    if (error)
    {
        snprintf(resultBuf, bufLen, "ERROR: invalid expression [%s]",
 ↪  raw);
    }
    else
    {
        snprintf(resultBuf, bufLen, "%s = %ld", raw, result);
    }
}

void onMessage(char *topic, byte *payload, unsigned int length)
{
    // Null-terminate payload
    char msg[128] = {};
    size_t len = min((unsigned int)(sizeof(msg) - 1), length);
    memcpy(msg, payload, len);

    Serial.printf("[MQTT] RX on '%s': %s\n", topic, msg);
```

```
    char resultBuf[160] = {};
    parser(msg, resultBuf, sizeof(resultBuf));

    Serial.printf("[MQTT] TX → %s\n", resultBuf);

    mqtt.publish(TOPIC_RESULT, resultBuf);
}

void connectWiFi()
{
    Serial.printf("Connecting to WiFi '%s'", WIFI_SSID);
    WiFi.begin(WIFI_SSID, WIFI_PASS);
    while (WiFi.status() != WL_CONNECTED)
    {
        delay(500);
        Serial.print(".");
    }
    Serial.printf("\nWiFi OK - IP: %s\n",
↪   WiFi.localIP().toString().c_str());
}

void connectMQTT()
{
    while (!mqtt.connected())
    {
        Serial.print("Connecting to MQTT broker… ");
        if (mqtt.connect(MQTT_CLIENT))
        {
            Serial.println("connected");
            mqtt.subscribe(TOPIC_INPUT);
            Serial.printf("Subscribed to '%s'\n", TOPIC_INPUT);
        }
        else
        {
            Serial.printf("failed (rc=%d), retry in 3 s\n",
↪   mqtt.state());
            delay(3000);
        }
    }
}

//   Arduino entry points
void setup()
{
    Serial.begin(115200);
    connectWiFi();
    mqtt.setServer(MQTT_BROKER, MQTT_PORT);
    mqtt.setCallback(onMessage);
```

```
    connectMQTT();
}

void loop()
{
    if (!mqtt.connected())
        connectMQTT();
    mqtt.loop();
}
```

CLIENT

```
#include <SPI.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_ILI9341.h>
#include <Adafruit_FT6206.h>
#include <WiFi.h>
#include <PubSubClient.h>

#define TFT_CS  15
#define TFT_DC   2

const char* WIFI_SSID = "Wokwi-GUEST";
const char* WIFI_PASS = "";

const char* MQTT_HOST    = "100.64.0.2";
const int   MQTT_PORT    = 1883;
const char* TOPIC_INPUT  = "calc/input";
const char* TOPIC_RESULT = "calc/result";

#define EXPR_Y    0
#define EXPR_H   60
#define RESULT_Y 60
#define RESULT_H 50
#define KEY_Y   114
#define KEY_ROWS  4
#define KEY_COLS  5
#define KEY_W   (240 / KEY_COLS)              // 48 px
#define KEY_H   ((320 - KEY_Y) / KEY_ROWS)    // 51 px

#define C_BG       0x0841
#define C_PANEL    0x10A2
#define C_BORDER   0x2945
#define C_EXPR_TXT 0xFFE0   // yellow
#define C_RES_TXT  0x07EF   // cyan
#define C_DIM      0x7BCF   // muted grey
```

```
#define C_NUM      0x2945   // dark slate
#define C_OP       0x0219   // teal
#define C_PAR      0x2104   // plum
#define C_SEND     0x0480   // green
#define C_DEL      0x7800   // red
#define C_CLR      0x3186   // indigo
#define C_FLASH    0xFFFF   // white press-flash
#define C_WHITE    0xFFFF
#define C_BLACK    0x0000

struct Key { const char* label; uint8_t col, row; uint16_t bg, fg; };

const Key KEYS[] = {
  // Row 0
  { "7",    0, 0, C_NUM,  C_WHITE    },
  { "8",    1, 0, C_NUM,  C_WHITE    },
  { "9",    2, 0, C_NUM,  C_WHITE    },
  { "/",    3, 0, C_OP,   C_EXPR_TXT },
  { "(",    4, 0, C_PAR,  C_RES_TXT  },
  // Row 1
  { "4",    0, 1, C_NUM,  C_WHITE    },
  { "5",    1, 1, C_NUM,  C_WHITE    },
  { "6",    2, 1, C_NUM,  C_WHITE    },
  { "*",    3, 1, C_OP,   C_EXPR_TXT },
  { ")",    4, 1, C_PAR,  C_RES_TXT  },
  // Row 2
  { "1",    0, 2, C_NUM,  C_WHITE    },
  { "2",    1, 2, C_NUM,  C_WHITE    },
  { "3",    2, 2, C_NUM,  C_WHITE    },
  { "-",    3, 2, C_OP,   C_EXPR_TXT },
  { "DEL",  4, 2, C_DEL,  C_WHITE    },
  // Row 3
  { "0",    0, 3, C_NUM,  C_WHITE    },
  { ".",    1, 3, C_NUM,  C_WHITE    },
  { "+",    2, 3, C_OP,   C_EXPR_TXT },
  { "CLR",  3, 3, C_CLR,  C_WHITE    },
  { "SEND", 4, 3, C_SEND, C_WHITE    },
};
constexpr int NUM_KEYS = sizeof(KEYS) / sizeof(KEYS[0]);

//   Globals
Adafruit_ILI9341 tft(TFT_CS, TFT_DC);
Adafruit_FT6206  ctp;                // I²C - no CS needed
WiFiClient       wifiClient;
PubSubClient     mqtt(wifiClient);

String expression  = "";
String resultText  = "";
```

```
bool   exprDirty   = false;
bool   resultDirty = false;

uint32_t lastTouchMs  = 0;
const uint32_t DEBOUNCE = 220;   // ms

//   Forward Declarations
void drawUI();
void drawExprPanel();
void drawResultPanel();
void drawAllKeys();
void drawKey(const Key& k, bool pressed = false);
void flashKey(const Key& k);
void handleKey(const char* label);
void statusLine(const char* msg, uint16_t col);
void connectWiFi();
void connectMQTT();
void mqttCallback(char* topic, byte* payload, unsigned int len);

//
//  SETUP
//
void setup() {
  Serial.begin(115200);

  //   Display
  tft.begin();
  tft.setRotation(0);           // portrait, USB at bottom
  tft.fillScreen(C_BG);

  //   FT6206 capacitive touch (I²C)
  // Default Wire uses GPIO21=SDA, GPIO22=SCL on ESP32 DevKit C v4.
  // If your board routes I²C elsewhere, call Wire.begin(SDA_PIN,
  ↪  SCL_PIN)
  // here before ctp.begin().
  Wire.begin();                 // SDA=21, SCL=22 (ESP32 DevKit C v4
↪  defaults)
  if (!ctp.begin(40)) {         // 40 = sensitivity coefficient (matches
  ↪  reference)
    Serial.println("[touch] FT6206 not found - check SDA/SCL wiring");
    // Non-fatal in Wokwi - sim may not enforce I²C probe
  }

  //   Splash
  tft.setTextColor(C_RES_TXT);
  tft.setTextSize(2);
  tft.setCursor(18, 80);
  tft.print("MQTT Calc v2.0");
```

```
  tft.setTextColor(C_DIM);
  tft.setTextSize(1);
  tft.setCursor(38, 108);
  tft.print("ESP32 + ILI9341-CAP");

  //   Connect WiFi
  statusLine("Connecting WiFi...", C_EXPR_TXT);
  connectWiFi();
  statusLine("WiFi OK!", 0x07E0);
  delay(350);

  //   Connect MQTT
  statusLine("Connecting MQTT...", C_EXPR_TXT);
  connectMQTT();
  statusLine("MQTT Ready!", 0x07E0);
  delay(350);

  //   Main UI
  drawUI();
}

//
//  LOOP
//
void loop() {
  // Keep MQTT alive
  if (!mqtt.connected()) connectMQTT();
  mqtt.loop();

  // Redraw only dirty panels
  if (exprDirty) { drawExprPanel();  exprDirty   = false; }
  if (resultDirty){ drawResultPanel(); resultDirty = false; }

  //   Touch Handling
  // FT6206 pattern from reference code: poll ctp.touched(),
   ↪  getPoint(),
  // then mirror both axes exactly as in the reference sketch.
  if (!ctp.touched()) return;

  uint32_t now = millis();
  if (now - lastTouchMs < DEBOUNCE) return;
  lastTouchMs = now;

  TS_Point p = ctp.getPoint();

  // Mirror axes to match screen orientation (portrait, rotation=0)
  // Identical to: p.x = map(p.x, 0, 240, 240, 0)  in the reference
   ↪  code.
```

```
  int sx = map(p.x, 0, 240, 239, 0);
  int sy = map(p.y, 0, 320, 319, 0);
  sx = constrain(sx, 0, 239);
  sy = constrain(sy, 0, 319);

  Serial.printf("[touch] raw(%d,%d) → screen(%d,%d)\n", p.x, p.y, sx,
↪  sy);

  // Only handle touches inside the keypad area
  if (sy < KEY_Y) return;

  int col = constrain(sx / KEY_W, 0, KEY_COLS - 1);
  int row = constrain((sy - KEY_Y) / KEY_H, 0, KEY_ROWS - 1);

  for (int i = 0; i < NUM_KEYS; i++) {
    if (KEYS[i].col == col && KEYS[i].row == row) {
      flashKey(KEYS[i]);
      handleKey(KEYS[i].label);
      return;
    }
  }
}

//
//  KEY HANDLING
//
void handleKey(const char* label) {
  if (strcmp(label, "DEL") == 0) {
    if (expression.length() > 0) {
      expression.remove(expression.length() - 1);
      exprDirty = true;
    }
  } else if (strcmp(label, "CLR") == 0) {
    expression  = "";
    resultText  = "";
    exprDirty   = true;
    resultDirty = true;
  } else if (strcmp(label, "SEND") == 0) {
    if (expression.length() > 0) {
      resultText  = "Waiting...";
      resultDirty = true;
      mqtt.publish(TOPIC_INPUT, expression.c_str());
      Serial.print("[mqtt] published → ");
      Serial.println(expression);
    }
  } else {
    if (expression.length() < 40) {
      expression += String(label);
```

```
      exprDirty = true;
    }
  }
}

//
//  MQTT CALLBACK
//
void mqttCallback(char* topic, byte* payload, unsigned int len) {
  resultText = "";
  for (unsigned int i = 0; i < len; i++) resultText += (char)payload[i];
  resultDirty = true;
  Serial.print("[mqtt] result ← ");
  Serial.println(resultText);
}

//
//  DISPLAY
//
void drawUI() {
  tft.fillScreen(C_BG);
  drawExprPanel();
  drawResultPanel();
  tft.fillRect(0, KEY_Y - 4, 240, 4, C_BORDER);   // thick divider
  drawAllKeys();
}

void drawExprPanel() {
  tft.fillRect(0, EXPR_Y, 240, EXPR_H, C_PANEL);
  tft.drawRect(0, EXPR_Y, 240, EXPR_H, C_BORDER);

  tft.setTextSize(1);
  tft.setTextColor(C_DIM);
  tft.setCursor(5, EXPR_Y + 4);
  tft.print("EXPRESSION");

  char cntBuf[6];
  snprintf(cntBuf, sizeof(cntBuf), "%2dc", (int)expression.length());
  tft.setCursor(205, EXPR_Y + 4);
  tft.print(cntBuf);

  // Two lines, 18 chars each at size-2 (12 px/char → 216 px max)
  const int MAX_CHARS = 18;
  String disp = expression;
  if ((int)disp.length() > MAX_CHARS * 2)
    disp = disp.substring(disp.length() - MAX_CHARS * 2);

  String l1 = disp.length() <= MAX_CHARS ? disp          :
↪   disp.substring(0, MAX_CHARS);
```

```
  String l2 = disp.length() <= MAX_CHARS ? String("")     :
↪  disp.substring(MAX_CHARS);

  while ((int)l1.length() < MAX_CHARS) l1 += ' ';
  while ((int)l2.length() < MAX_CHARS) l2 += ' ';

  tft.setTextColor(C_EXPR_TXT);
  tft.setTextSize(2);
  tft.setCursor(5, EXPR_Y + 16);
  tft.print(l1);
  tft.setCursor(5, EXPR_Y + 38);
  tft.print(l2);
}

void drawResultPanel() {
  tft.fillRect(0, RESULT_Y, 240, RESULT_H, C_PANEL);
  tft.drawRect(0, RESULT_Y, 240, RESULT_H, C_BORDER);

  tft.setTextSize(1);
  tft.setTextColor(C_DIM);
  tft.setCursor(5, RESULT_Y + 4);
  tft.print("RESULT");

  String disp = resultText;
  disp.trim();
  if ((int)disp.length() > 19) disp = disp.substring(0, 19);
  while ((int)disp.length() < 19) disp += ' ';

  tft.setTextColor(C_RES_TXT);
  tft.setTextSize(2);
  tft.setCursor(5, RESULT_Y + 22);
  tft.print(disp);
}

void drawAllKeys() {
  for (int i = 0; i < NUM_KEYS; i++) drawKey(KEYS[i]);
}

void drawKey(const Key& k, bool pressed) {
  int x = k.col * KEY_W;
  int y = KEY_Y + k.row * KEY_H;
  int w = KEY_W;
  int h = KEY_H;

  uint16_t bg = pressed ? C_FLASH : k.bg;
  uint16_t fg = pressed ? C_BLACK : k.fg;

  tft.fillRoundRect(x + 2, y + 2, w - 4, h - 4, 5, bg);
```

```
  tft.drawRoundRect(x + 2, y + 2, w - 4, h - 4, 5, C_BORDER);

  int len = strlen(k.label);
  if (len == 1) {
    tft.setTextSize(2);
    tft.setTextColor(fg);
    tft.setCursor(x + w / 2 - 6, y + h / 2 - 8);
    tft.print(k.label);
  } else {
    tft.setTextSize(1);
    tft.setTextColor(fg);
    tft.setCursor(x + w / 2 - len * 3, y + h / 2 - 4);
    tft.print(k.label);
  }
}

void flashKey(const Key& k) {
  drawKey(k, true);
  delay(70);
  drawKey(k, false);
}

//
void statusLine(const char* msg, uint16_t col) {
  tft.fillRect(0, 152, 240, 14, C_BG);
  tft.setTextSize(1);
  tft.setTextColor(col);
  int16_t x1, y1; uint16_t w, h;
  tft.getTextBounds(msg, 0, 0, &x1, &y1, &w, &h);
  tft.setCursor((240 - (int)w) / 2, 154);
  tft.print(msg);
}

//
//  WiFi
//
void connectWiFi() {
  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASS);
  int tries = 0;
  while (WiFi.status() != WL_CONNECTED && tries < 40) {
    delay(250);
    Serial.print('.');
    tries++;
  }
  if (WiFi.status() == WL_CONNECTED)
    Serial.printf("\n[wifi] IP %s\n",
↪   WiFi.localIP().toString().c_str());
```

```
  else
    Serial.println("\n[wifi] failed - continuing anyway");
}

//
//  MQTT
//
void connectMQTT() {
  mqtt.setServer(MQTT_HOST, MQTT_PORT);
  mqtt.setCallback(mqttCallback);

  int tries = 0;
  while (!mqtt.connected() && tries < 8) {
    String id = "ESP32Calc-" + String(esp_random() & 0xFFFF, HEX);
    Serial.printf("[mqtt] connecting as %s ... ", id.c_str());
    if (mqtt.connect(id.c_str())) {
      Serial.println("OK");
      mqtt.subscribe(TOPIC_RESULT);
      Serial.printf("[mqtt] subscribed → %s\n", TOPIC_RESULT);
    } else {
      Serial.printf("fail (rc=%d), retry\n", mqtt.state());
      delay(1500);
    }
    tries++;
  }
}
```

JSON

SERVER

```
{
  "version": 1,
  "author": "Shree Kottes",
  "editor": "wokwi",
  "parts": [ { "type": "board-esp32-devkit-c-v4", "id": "esp", "top": 0,
   ↪  "left": 0, "attrs": {} } ],
  "connections": [ [ "esp:TX", "$serialMonitor:RX", "", [] ], [ "esp:RX",
   ↪  "$serialMonitor:TX", "", [] ] ],
  "dependencies": {}
}
```

CLIENT

```
{
  "version": 1,
  "author": "ESP32 MQTT Calculator (cap-touch)",
```

```
  "editor": "wokwi",
  "parts": [
    { "type": "board-esp32-devkit-c-v4", "id": "esp", "top": 0, "left":
     ↪  -4.76, "attrs": {} },
    {
      "type": "board-ili9341-cap-touch",
      "id": "lcd1",
      "top": -47.24,
      "left": 172.42,
      "attrs": {}
    }
  ],
  "connections": [
    [ "esp:TX", "$serialMonitor:RX", "", [] ],
    [ "esp:RX", "$serialMonitor:TX", "", [] ],
    [ "esp:3V3", "lcd1:VCC", "red", [ "v19.2", "h-86.25", "v192" ] ],
    [ "esp:GND.3", "lcd1:GND", "black", [ "h57.6", "v182.4", "h67.2" ] ],
    [ "esp:15", "lcd1:CS", "purple", [ "v76.8", "h67.2", "h67.2" ] ],
    [ "esp:2", "lcd1:D/C", "green", [ "v96", "h105.6", "v-19.2" ] ],
    [ "esp:23", "lcd1:MOSI", "blue", [ "h19.2", "v249.6", "h144" ] ],
    [ "esp:18", "lcd1:SCK", "cyan", [ "v172.8", "h144", "v-38.4" ] ],
    [ "esp:19", "lcd1:MISO", "yellow", [ "h38.4", "v211.2", "h153.6" ] ],
    [ "esp:21", "lcd1:SDA", "orange", [ "h28.8", "v220.8", "h182.4" ] ],
    [ "esp:22", "lcd1:SCL", "white", [ "h38.4", "v220.8", "v19.2",
     ↪  "h163.2" ] ]
  ],
  "dependencies": {}
}
```

PIN CONFIGURATION

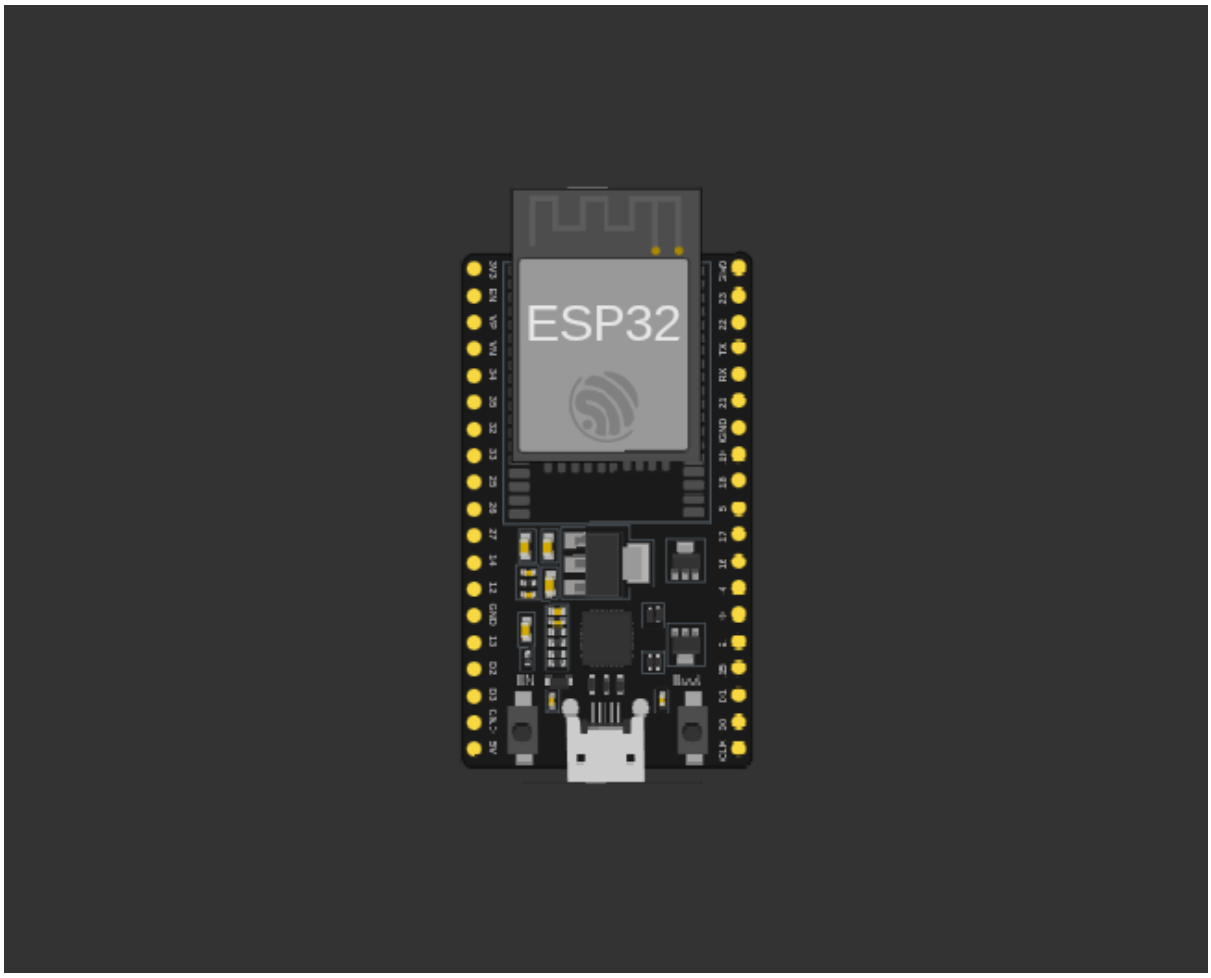


Figure 8.1: Server

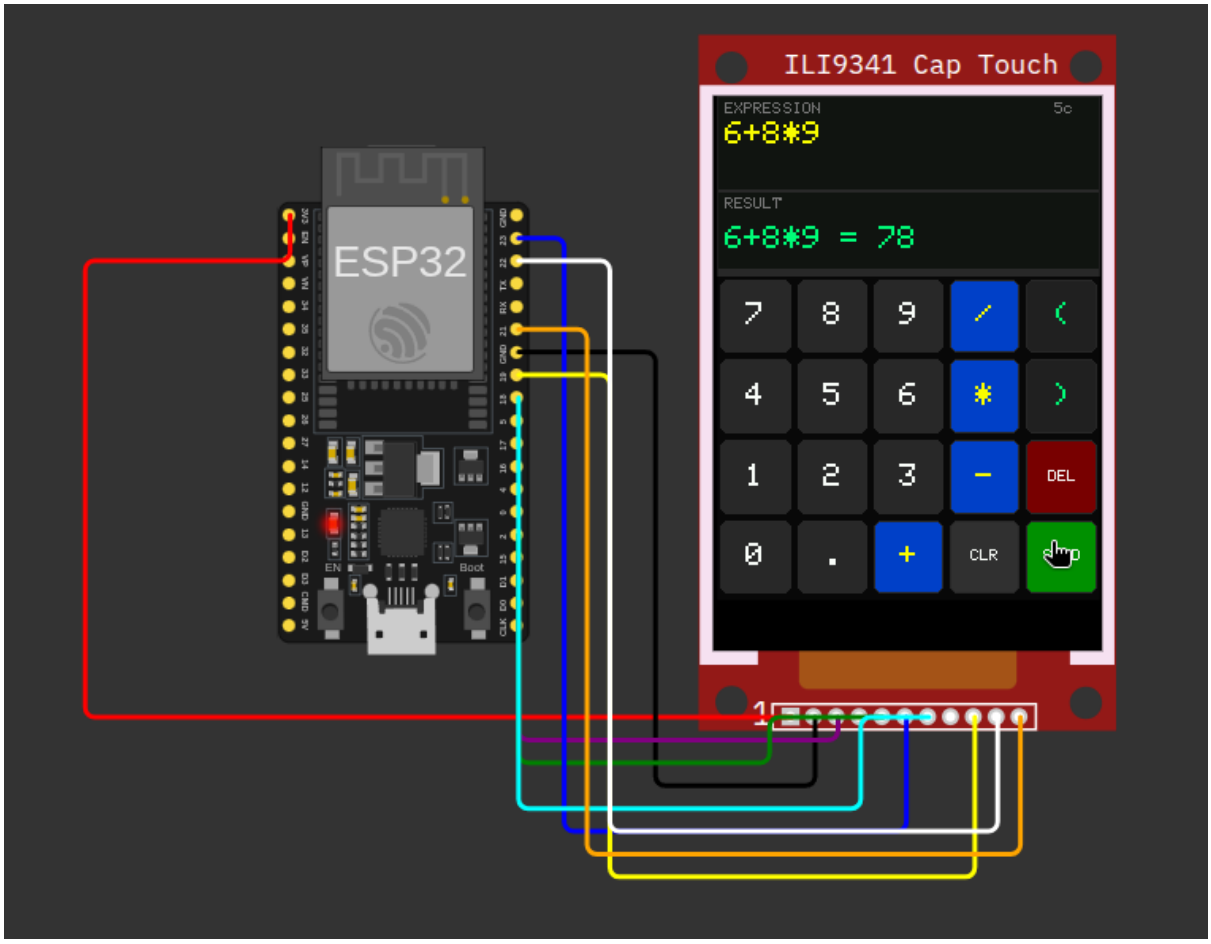


Figure 8.2: Client

EX.NO: 9	SMART TEMPERATURE AND HUMIDITY MONITORING
27/02/2026	

AIM

To simulate smart temperature and humidity monitoring using MQTT Broker.

APPARATUS REQUIRED

- Espressif ESP32 Dev Module.
- DHT22 (Temperature and Humidity Sensor).

PRE-CONDITIONS

POST-CONDITIONS

CODE

SERVER

```
#include <Arduino.h>
#include <WiFi.h>
#include <PubSubClient.h>
#include <DHT.h>

//    Config
#define DHT_PIN          4
#define DHT_TYPE         DHT22
#define PUBLISH_INTERVAL 2000 // ms

#define WIFI_SSID "Wokwi-GUEST"
#define WIFI_PASS ""

#define MQTT_BROKER  "100.64.0.2"
#define MQTT_PORT    1883
#define CLIENT_ID    "ESP32-TempMonitor"

// Topics
#define TOPIC_TEMP    "/home/temp"
#define TOPIC_HUMID   "/home/humid"
#define TOPIC_ALERT   "/home/alerts"
#define TOPIC_STATUS  "/home/status"

// Thresholds
#define TEMP_MAX  35.0
#define HUMID_MAX 80.0
```

```
//    Globals
DHT          dht(DHT_PIN, DHT_TYPE);
WiFiClient   wifiClient;
PubSubClient mqtt(wifiClient);

//    WiFi
void connectWiFi() {
  WiFi.begin(WIFI_SSID, WIFI_PASS);
  Serial.print("[WiFi] Connecting");
  while (WiFi.status() != WL_CONNECTED) {
    delay(400);
    Serial.print(".");
  }
  Serial.printf("\n[WiFi] IP: %s\n", WiFi.localIP().toString().c_str());
}

//    MQTT
void onMessage(char* topic, byte* payload, unsigned int len) {
  String msg;
  for (unsigned int i = 0; i < len; i++) msg += (char)payload[i];
  Serial.printf("[MQTT] Received [%s]: %s\n", topic, msg.c_str());
}

void connectMQTT() {
  while (!mqtt.connected()) {
    Serial.print("[MQTT] Connecting...");
    if (mqtt.connect(CLIENT_ID)) {
      Serial.println(" connected");
      mqtt.publish(TOPIC_STATUS, "online", true);  // retained
    } else {
      Serial.printf(" failed (rc=%d), retry 5s\n", mqtt.state());
      delay(5000);
    }
  }
}

void ensureConnected() {
  if (WiFi.status() != WL_CONNECTED) connectWiFi();
  if (!mqtt.connected())              connectMQTT();
}

//    Sensor
struct Reading {
  float temp;
  float humid;
  bool  valid;
};
```

```
Reading readDHT() {
  Reading r;
  r.temp  = dht.readTemperature();
  r.humid = dht.readHumidity();
  r.valid = !isnan(r.temp) && !isnan(r.humid);
  return r;
}

//    Publish
void publishReading(const Reading& r) {
  char tBuf[8], hBuf[8];
  dtostrf(r.temp,  4, 2, tBuf);
  dtostrf(r.humid, 4, 2, hBuf);

  mqtt.publish(TOPIC_TEMP,  tBuf);
  mqtt.publish(TOPIC_HUMID, hBuf);

  Serial.printf("[DATA] Temp: %s°C | Humidity: %s%%\n", tBuf, hBuf);
}

void checkThresholds(const Reading& r) {
  if (r.temp > TEMP_MAX) {
    char msg[40];
    snprintf(msg, sizeof(msg), "HIGH_TEMP:%.2f", r.temp);
    mqtt.publish(TOPIC_ALERT, msg);
    Serial.printf("[ALERT] %s\n", msg);
  }
  if (r.humid > HUMID_MAX) {
    char msg[40];
    snprintf(msg, sizeof(msg), "HIGH_HUMID:%.2f", r.humid);
    mqtt.publish(TOPIC_ALERT, msg);
    Serial.printf("[ALERT] %s\n", msg);
  }
}

//    Main
void setup() {
  Serial.begin(115200);
  dht.begin();
  connectWiFi();
  mqtt.setServer(MQTT_BROKER, MQTT_PORT);
  mqtt.setCallback(onMessage);
  connectMQTT();
}

void loop() {
  ensureConnected();
```

```
  mqtt.loop();

  static unsigned long lastPublish = 0;
  if (millis() - lastPublish >= PUBLISH_INTERVAL) {
    lastPublish = millis();
    Reading r = readDHT();
    if (r.valid) {
      publishReading(r);
      checkThresholds(r);
    } else {
      Serial.println("[ERR] Sensor read failed");
    }
  }
}
```

CLIENT

```
#include <SPI.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_ILI9341.h>
#include <Adafruit_FT6206.h>
#include <WiFi.h>
#include <PubSubClient.h>

#define TFT_CS 15
#define TFT_DC 2

const char *WIFI_SSID = "Wokwi-GUEST";
const char *WIFI_PASS = "";
const char *MQTT_HOST = "100.64.0.2";
const int   MQTT_PORT = 1883;

#define TOPIC_TEMP   "/home/temp"
#define TOPIC_HUMID  "/home/humid"
#define TOPIC_ALERT  "/home/alerts"
#define TOPIC_STATUS "/home/status"

#define C_BG     0x0821
#define C_CARD   0x1082
#define C_BORDER 0x294A
#define C_WHITE  0xFFFF
#define C_GRAY   0x7BEF
#define C_DKGRAY 0x2965
#define C_GREEN  0x07E0
#define C_RED    0xF800
#define C_ORANGE 0xFC60
```

```
#define C_YELLOW 0xFFE0
#define C_CYAN   0x07FF

#define HDR_Y  0
#define HDR_H  30
#define TEMP_Y 30
#define TEMP_H 70
#define HUM_Y  100
#define HUM_H  70
#define ALT_Y  170
#define ALT_H  90
#define LOG_Y  260
#define LOG_H  60

#define ALERT_LINES 3  // number of alert history lines shown

Adafruit_ILI9341 tft(TFT_CS, TFT_DC);
Adafruit_FT6206  ctp;
WiFiClient       wifiClient;
PubSubClient     mqtt(wifiClient);

//   Live values
String valTemp   = "--";
String valHumid  = "--";
String valStatus = "Waiting...";
bool   mqttOK    = false;

// Alert ring buffer - newest at index [ALERT_LINES-1]
String alertHistory[ALERT_LINES];
int    alertCount = 0;   // total alerts received (caps at ALERT_LINES
 ↪  for display)

// Log ring buffer
String logLines[3];

//   Shadow values - track what is currently drawn
String  drawnTemp;
String  drawnHumid;
String  drawnStatus;
bool    drawnMqttOK     = false;
String  drawnAlerts[ALERT_LINES];
String  drawnLog[3];
bool    drawnAlertFlash = false;

//   Dirty flags (set when we KNOW a redraw is needed regardless of
 ↪  shadow)
bool dirtyHdr   = false;
bool dirtyTemp  = false;
```

```
bool dirtyHumid = false;
bool dirtyAlert = false;
bool dirtyLog   = false;
bool fullRedraw = true;

//   Alert flash
bool     alertFlashing    = false;
uint32_t alertFlashStart  = 0;
uint8_t  alertPhase       = 0;

uint32_t lastTouchMs = 0;

//
//  Helpers
//

void drawCardBg(int y, int h, uint16_t bg, uint16_t border) {
  tft.fillRect(0, y, 240, h, C_BG);
  tft.fillRoundRect(2, y + 2, 236, h - 4, 7, bg);
  tft.drawRoundRect(2, y + 2, 236, h - 4, 7, border);
}

void cardLabel(int y, const char *label, uint16_t col) {
  tft.setTextSize(1);
  tft.setTextColor(col);
  tft.setCursor(10, y + 9);
  tft.print(label);
}

void cardValue(int y, int h, const char *val, uint16_t col) {
  int len  = strlen(val);
  int txtW = len * 18;
  int cx   = max(8, (240 - txtW) / 2);
  int cy   = y + (h / 2) - 4;
  tft.setTextSize(3);
  tft.setTextColor(col);
  tft.setCursor(cx, cy);
  tft.print(val);
}

void pushLog(const char *topic, const char *val) {
  logLines[0] = logLines[1];
  logLines[1] = logLines[2];
  char buf[36];
  snprintf(buf, sizeof(buf), "%-6s: %.22s", topic, val);
  logLines[2] = String(buf);
}
```

```
void pushAlert(const String &msg) {
  alertHistory[0] = alertHistory[1];
  alertHistory[1] = alertHistory[2];
  alertHistory[2] = msg;
  alertCount++;
}

//
//  Draw functions
//

void drawHeader() {
  drawCardBg(HDR_Y, HDR_H, C_CARD, C_BORDER);
  tft.setTextColor(C_WHITE); tft.setTextSize(1);
  tft.setCursor(10, HDR_Y + 7);
  tft.print("HOME MONITOR");
  tft.setTextColor(C_GRAY);
  tft.setCursor(10, HDR_Y + 19);
  String s = valStatus;
  if (s.length() > 16) s = s.substring(0, 16);
  tft.print(s);
  tft.fillCircle(225, HDR_Y + 15, 5, mqttOK ? C_GREEN : C_RED);
  tft.drawCircle(225, HDR_Y + 15, 5, C_DKGRAY);

  drawnStatus  = valStatus;
  drawnMqttOK  = mqttOK;
}

void drawTempCard() {
  drawCardBg(TEMP_Y, TEMP_H, C_CARD, C_BORDER);
  cardLabel(TEMP_Y, "TEMPERATURE  (C)", C_GRAY);
  tft.setTextSize(1); tft.setTextColor(C_DKGRAY);
  tft.setCursor(195, TEMP_Y + 9);
  tft.print("deg C");
  String disp = valTemp;
  if (disp.length() > 8) disp = disp.substring(0, 8);
  cardValue(TEMP_Y, TEMP_H, disp.c_str(), C_CYAN);

  drawnTemp = valTemp;
}

void drawHumidCard() {
  drawCardBg(HUM_Y, HUM_H, C_CARD, C_BORDER);
  cardLabel(HUM_Y, "HUMIDITY  (%)", C_GRAY);
  tft.setTextSize(1); tft.setTextColor(C_DKGRAY);
  tft.setCursor(210, HUM_Y + 9);
  tft.print("%");
  String disp = valHumid;
```

```
  if (disp.length() > 8) disp = disp.substring(0, 8);
  cardValue(HUM_Y, HUM_H, disp.c_str(), C_GREEN);

  drawnHumid = valHumid;
}

// Alert card shows the last ALERT_LINES messages, newest at bottom
void drawAlertCard(bool flashOn) {
  uint16_t bg     = flashOn ? 0x8000  : C_CARD;
  uint16_t brd    = flashOn ? C_ORANGE : C_BORDER;
  uint16_t hdrCol = flashOn ? C_YELLOW : C_GRAY;
  uint16_t txtCol = flashOn ? C_WHITE  : C_ORANGE;

  drawCardBg(ALT_Y, ALT_H, bg, brd);

  tft.setTextSize(1); tft.setTextColor(hdrCol);
  tft.setCursor(10, ALT_Y + 9);
  tft.print(flashOn ? "!! ALERT !!" : "ALERT");

  // Count how many entries are non-empty
  int first = ALERT_LINES;
  for (int i = 0; i < ALERT_LINES; i++)
    if (alertHistory[i].length() > 0) { first = i; break; }

  if (first == ALERT_LINES) {
    // Nothing received yet
    tft.setTextColor(C_DKGRAY);
    tft.setCursor(10, ALT_Y + 28);
    tft.print("No alerts");
  } else {
    int lineH  = 17;
    int startY = ALT_Y + 24;
    for (int i = first; i < ALERT_LINES; i++) {
      // Newest line is brightest
      uint16_t lc = (i == ALERT_LINES - 1) ? txtCol : C_DKGRAY;
      tft.setTextColor(lc);
      tft.setCursor(10, startY + (i - first) * lineH);
      // Truncate to 28 chars to fit card width
      String line = alertHistory[i];
      if (line.length() > 28) line = line.substring(0, 28);
      tft.print(line);
    }
  }

  if (!flashOn && alertCount > 0) {
    tft.setTextColor(C_DKGRAY);
    tft.setCursor(140, ALT_Y + ALT_H - 14);
    tft.print("tap to clear");
```

```
  }

  drawnAlertFlash = flashOn;
  for (int i = 0; i < ALERT_LINES; i++) drawnAlerts[i] =
   ↪  alertHistory[i];
}

void drawLogCard() {
  drawCardBg(LOG_Y, LOG_H, C_BG, C_BORDER);
  tft.setTextSize(1); tft.setTextColor(C_DKGRAY);
  tft.setCursor(10, LOG_Y + 6);
  tft.print("Last received:");
  for (int i = 0; i < 3; i++) {
    tft.setTextColor(i == 2 ? C_GRAY : C_DKGRAY);
    tft.setCursor(10, LOG_Y + 18 + i * 13);
    tft.print(logLines[i]);
  }
  for (int i = 0; i < 3; i++) drawnLog[i] = logLines[i];
}

void drawAll() {
  tft.fillScreen(C_BG);
  drawHeader();
  drawTempCard();
  drawHumidCard();
  drawAlertCard(alertPhase);
  drawLogCard();
}

//
//  Selective update - compare shadow, repaint only what changed
//

void updateDisplay() {
  // Header: MQTT status dot or status string changed
  if (dirtyHdr || valStatus != drawnStatus || mqttOK != drawnMqttOK) {
    drawHeader();
    dirtyHdr = false;
  }

  if (dirtyTemp || valTemp != drawnTemp) {
    drawTempCard();
    dirtyTemp = false;
  }

  if (dirtyHumid || valHumid != drawnHumid) {
    drawHumidCard();
    dirtyHumid = false;
```

```
  }

  // Alert: flash phase changed, or history content changed
  bool alertsDiffer = false;
  for (int i = 0; i < ALERT_LINES; i++)
    if (alertHistory[i] != drawnAlerts[i]) { alertsDiffer = true; break;
     ↪  }

  if (dirtyAlert || alertsDiffer || (alertPhase != drawnAlertFlash)) {
    drawAlertCard(alertPhase);
    dirtyAlert = false;
  }

  // Log: any line changed
  bool logDiffer = false;
  for (int i = 0; i < 3; i++)
    if (logLines[i] != drawnLog[i]) { logDiffer = true; break; }

  if (dirtyLog || logDiffer) {
    drawLogCard();
    dirtyLog = false;
  }
}

//
//  MQTT callback
//

void mqttCallback(char *topic, byte *payload, unsigned int len) {
  char buf[128];
  int n = min((int)len, 127);
  memcpy(buf, payload, n);
  buf[n] = '\0';
  String msg = String(buf);
  msg.trim();

  if (strcmp(topic, TOPIC_TEMP) == 0) {
    valTemp = msg;
    pushLog("TEMP", msg.c_str());
  } else if (strcmp(topic, TOPIC_HUMID) == 0) {
    valHumid = msg;
    pushLog("HUMID", msg.c_str());
  } else if (strcmp(topic, TOPIC_ALERT) == 0) {
    pushAlert(msg);
    alertFlashing   = true;
    alertFlashStart = millis();
    alertPhase      = 1;
    pushLog("ALERT", msg.c_str());
```

```
  } else if (strcmp(topic, TOPIC_STATUS) == 0) {
    valStatus = msg;
    pushLog("STATUS", msg.c_str());
  }
  // Shadow comparison in updateDisplay() handles whether a repaint is
   ↪ needed
}

//
//  WiFi / MQTT connection
//

void connectWiFi() {
  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASS);
  int t = 0;
  while (WiFi.status() != WL_CONNECTED && t++ < 40) delay(250);
}

void connectMQTT() {
  mqtt.setServer(MQTT_HOST, MQTT_PORT);
  mqtt.setCallback(mqttCallback);
  for (int t = 0; t < 8 && !mqtt.connected(); t++) {
    String id = "ESP32Dash-" + String(esp_random() & 0xFFFF, HEX);
    if (mqtt.connect(id.c_str())) {
      mqtt.subscribe(TOPIC_TEMP);
      mqtt.subscribe(TOPIC_HUMID);
      mqtt.subscribe(TOPIC_ALERT);
      mqtt.subscribe(TOPIC_STATUS);
      mqttOK = true;
    } else {
      delay(1500);
    }
  }
}

//
//  Setup / Loop
//

void setup() {
  Serial.begin(115200);
  tft.begin();
  tft.setRotation(0);
  tft.fillScreen(C_BG);
  Wire.begin();
  ctp.begin(40);
```

```
  tft.setTextColor(C_CYAN); tft.setTextSize(2);
  tft.setCursor(20, 100);
  tft.print("Home Monitor");
  tft.setTextColor(C_GRAY); tft.setTextSize(1);
  tft.setCursor(60, 130);
  tft.print("Connecting...");

  connectWiFi();
  connectMQTT();
  drawAll();
}

void loop() {
  if (!mqtt.connected()) {
    mqttOK = false;
    connectMQTT();
  }
  mqtt.loop();

  // Alert flash - 250 ms toggle for 3 seconds
  if (alertFlashing) {
    uint32_t elapsed = millis() - alertFlashStart;
    if (elapsed < 3000) {
      uint8_t phase = (elapsed / 250) % 2;
      if (phase != alertPhase) alertPhase = phase;
    } else {
      alertFlashing = false;
      alertPhase    = 0;
    }
  }

  updateDisplay();

  // Touch - tap alert card to clear history
  if (ctp.touched()) {
    uint32_t now = millis();
    if (now - lastTouchMs > 300) {
      lastTouchMs = now;
      TS_Point p  = ctp.getPoint();
      int sy = map(p.y, 0, 320, 319, 0);
      if (sy >= ALT_Y && sy < ALT_Y + ALT_H) {
        alertFlashing = false;
        alertPhase    = 0;
        alertCount    = 0;
        for (int i = 0; i < ALERT_LINES; i++) alertHistory[i] = "";
      }
    }
  }
```

```
  delay(16);
}
```

JSON

SERVER

```
{
  "version": 1,
  "author": "Shree Kottes",
  "editor": "wokwi",
  "parts": [
    { "type": "board-esp32-devkit-c-v4", "id": "esp", "top": 19.2,
     ↪  "left": -23.96, "attrs": {} },
    { "type": "wokwi-dht22", "id": "dht1", "top": -114.9, "left": -5.4,
     ↪  "attrs": {} }
  ],
  "connections": [
    [ "esp:TX", "$serialMonitor:RX", "", [] ],
    [ "esp:RX", "$serialMonitor:TX", "", [] ],
    [ "dht1:GND", "esp:GND.2", "black", [ "v9.6", "h57.6", "v38.4" ] ],
    [ "dht1:VCC", "esp:3V3", "red", [ "v9.6", "h-48", "v38.4" ] ],
    [ "dht1:SDA", "esp:4", "green", [ "v19.2", "h86.5", "v144" ] ]
  ],
  "dependencies": {}
}
```

CLIENT

```
{
  "version": 1,
  "author": "ESP32 MQTT Calculator (cap-touch)",
  "editor": "wokwi",
  "parts": [
    { "type": "board-esp32-devkit-c-v4", "id": "esp", "top": 0, "left":
     ↪  -4.76, "attrs": {} },
    {
      "type": "board-ili9341-cap-touch",
      "id": "lcd1",
      "top": -47.24,
      "left": 172.42,
      "attrs": {}
    }
  ],
  "connections": [
    [ "esp:TX", "$serialMonitor:RX", "", [] ],
```

```
    [ "esp:RX", "$serialMonitor:TX", "", [] ],
    [ "esp:3V3", "lcd1:VCC", "red", [ "v19.2", "h-86.25", "v192" ] ],
    [ "esp:GND.3", "lcd1:GND", "black", [ "h57.6", "v182.4", "h67.2" ] ],
    [ "esp:15", "lcd1:CS", "purple", [ "v76.8", "h67.2", "h67.2" ] ],
    [ "esp:2", "lcd1:D/C", "green", [ "v96", "h105.6", "v-19.2" ] ],
    [ "esp:23", "lcd1:MOSI", "blue", [ "h19.2", "v249.6", "h144" ] ],
    [ "esp:18", "lcd1:SCK", "cyan", [ "v172.8", "h144", "v-38.4" ] ],
    [ "esp:19", "lcd1:MISO", "yellow", [ "h38.4", "v211.2", "h153.6" ] ],
    [ "esp:21", "lcd1:SDA", "orange", [ "h28.8", "v220.8", "h182.4" ] ],
    [ "esp:22", "lcd1:SCL", "white", [ "h38.4", "v220.8", "v19.2",
     ↪  "h163.2" ] ]
  ],
  "dependencies": {}
}
```

PIN CONFIGURATION

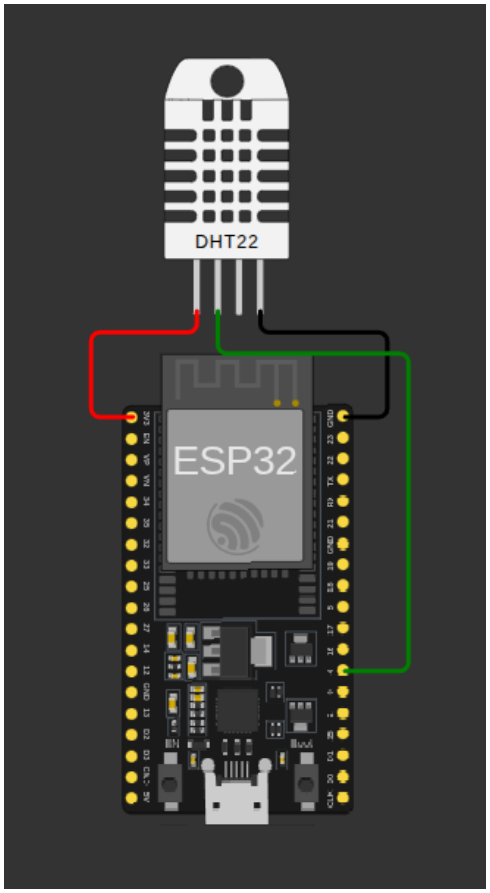


Figure 1: Server

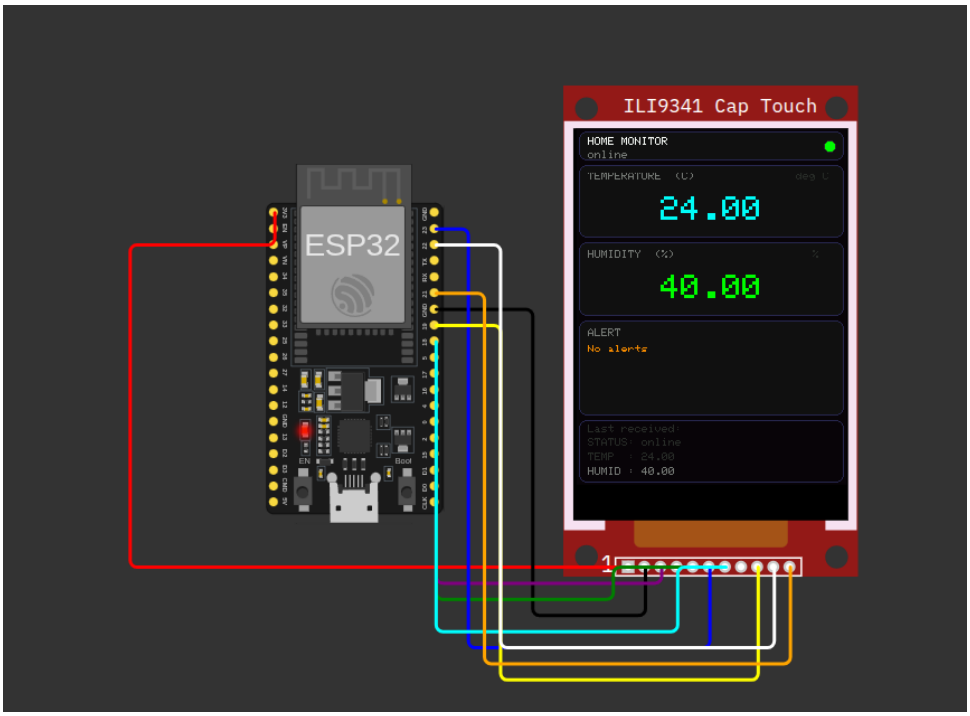


Figure 2: Client

EX.NO: 10	SMART FOUR WHEELER VEHICLE PARKING
20/03/2026	

AIM

To build a Smart Four Wheeler Vehicle Parking System using MQTT Broker.

APPARATUS REQUIRED

- Espressif ESP32 Dev Module
- HC-SR04 Proximity Sensor
- ILI9341 Cap Touch LCD Display

PRE-CONDITIONS

POST-CONDITIONS

CODE

SERVER

```
#include <WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>

const char *WIFI_SSID = "Wokwi-GUEST"; // change for real hardware
const char *WIFI_PASS = "";
const char *BROKER = "100.64.0.2";
const int BROKER_PORT = 1883;

const char *TOPIC_STATUS = "smartparking/status";
const char *TOPIC_CMD = "smartparking/cmd";
const char *TOPIC_ACK = "smartparking/ack";

#define NUM_SLOTS 6
#define THRESHOLD_CM 20
#define RESERVE_TIMEOUT_MS (30 * 1000)//(5UL * 60000UL)
#define STATUS_INTERVAL_MS 1000

// Slot 1→6 TRIG / ECHO pin pairs
const int TRIG_PINS[NUM_SLOTS] = {12, 13, 25, 26, 27, 33};
const int ECHO_PINS[NUM_SLOTS] = {14, 35, 34, 32, 4, 17};

struct Slot
{
  bool occupied, reserved;
```

```
  float dist;
  unsigned long resAt;
};
Slot slots[NUM_SLOTS];

WiFiClient wifiClient;
PubSubClient mqtt(wifiClient);
unsigned long lastStatus = 0;

// --- Sensor ---

float measureDist(int trig, int echo)
{
  digitalWrite(trig, LOW);
  delayMicroseconds(2);
  digitalWrite(trig, HIGH);
  delayMicroseconds(10);
  digitalWrite(trig, LOW);
  long dur = pulseIn(echo, HIGH, 30000);
  return (dur == 0) ? 999.0f : dur * 0.0343f / 2.0f;
}

void updateSlots()
{
  for (int i = 0; i < NUM_SLOTS; i++)
  {
    float d = measureDist(TRIG_PINS[i], ECHO_PINS[i]);
    slots[i].dist = d;
    if (d < THRESHOLD_CM)
    {
      slots[i].occupied = true;
      slots[i].reserved = false; // car arrived, consume reservation
    }
    else
    {
      slots[i].occupied = false;
      if (slots[i].reserved &&
          millis() - slots[i].resAt > RESERVE_TIMEOUT_MS)
      {
        slots[i].reserved = false; // timed-out reservation
      }
    }
  }
}

// --- MQTT publish ---

void publishStatus()
```

```
{
  StaticJsonDocument<512> doc;
  doc["total"] = NUM_SLOTS;
  int freeCount = 0;
  JsonArray arr = doc.createNestedArray("slots");
  for (int i = 0; i < NUM_SLOTS; i++)
  {
    bool avail = !slots[i].occupied && !slots[i].reserved;
    if (avail)
      freeCount++;
    JsonObject s = arr.createNestedObject();
    s["id"] = i + 1;
    s["occupied"] = slots[i].occupied;
    s["reserved"] = slots[i].reserved;
    s["available"] = avail;
    s["dist"] = (int)slots[i].dist;
  }
  doc["free"] = freeCount;
  char buf[512];
  serializeJson(doc, buf, sizeof(buf));
  mqtt.publish(TOPIC_STATUS, buf, true); // retained
}

// --- MQTT callback ---

void onMessage(char *topic, byte *payload, unsigned int len)
{
  StaticJsonDocument<128> req;
  if (deserializeJson(req, payload, len) != DeserializationError::Ok)
    return;

  const char *action = req["action"] | "";
  int sid = req["slot"] | 0;
  if (sid < 1 || sid > NUM_SLOTS)
    return;
  int idx = sid - 1;

  char ack[96];

  if (strcmp(action, "reserve") == 0)
  {
    if (slots[idx].occupied || slots[idx].reserved)
    {
      snprintf(ack, sizeof(ack),

                  ↪  "{\"slot\":%d,\"success\":false,\"msg\":\"unavailable\"}",
                  ↪  sid);
    }
```

```
    else
    {
      slots[idx].reserved = true;
      slots[idx].resAt = millis();
      snprintf(ack, sizeof(ack),
               "{\"slot\":%d,\"success\":true,\"msg\":\"reserved\"}",
               ↪  sid);
      publishStatus();
    }
  }
  else if (strcmp(action, "release") == 0)
  {
    if (!slots[idx].reserved)
    {
      snprintf(ack, sizeof(ack),
               "{\"slot\":%d,\"success\":false,\"msg\":\"not
               ↪  reserved\"}", sid);
    }
    else
    {
      slots[idx].reserved = false;
      snprintf(ack, sizeof(ack),
               "{\"slot\":%d,\"success\":true,\"msg\":\"released\"}",
               ↪  sid);
      publishStatus();
    }
  }
  else
  {
    return;
  }

  mqtt.publish(TOPIC_ACK, ack);
}

// --- Connection helpers ---

void connectWiFi()
{
  WiFi.begin(WIFI_SSID, WIFI_PASS);
  Serial.print("WiFi");
  while (WiFi.status() != WL_CONNECTED)
  {
    delay(500);
    Serial.print(".");
  }
  Serial.println(" OK");
}
```

```
void connectMQTT()
{
  String cid = "ParkSrv-" + String((uint32_t)ESP.getEfuseMac(), HEX);
  while (!mqtt.connected())
  {
    Serial.print("MQTT...");
    if (mqtt.connect(cid.c_str()))
    {
      mqtt.subscribe(TOPIC_CMD);
      Serial.println("OK");
    }
    else
    {
      delay(2000);
    }
  }
}

// --- Setup / Loop ---

void setup()
{
  Serial.begin(115200);
  for (int i = 0; i < NUM_SLOTS; i++)
  {
    pinMode(TRIG_PINS[i], OUTPUT);
    pinMode(ECHO_PINS[i], INPUT);
    slots[i] = {false, false, 999.0f, 0};
  }
  connectWiFi();
  mqtt.setServer(BROKER, BROKER_PORT);
  mqtt.setBufferSize(1024);
  mqtt.setCallback(onMessage);
  connectMQTT();
  publishStatus();
}

void loop()
{
  if (!mqtt.connected())
    connectMQTT();
  mqtt.loop();
  updateSlots();
  if (millis() - lastStatus >= STATUS_INTERVAL_MS)
  {
    lastStatus = millis();
    publishStatus();
```

```
  }
}
```

CLIENT

```
#include "SPI.h"
#include "Adafruit_GFX.h"
#include "Adafruit_ILI9341.h"
#include <Adafruit_FT6206.h>
#include <WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>

const char *WIFI_SSID = "Wokwi-GUEST";
const char *WIFI_PASS = "";
const char *BROKER = "100.64.0.2";
const int BROKER_PORT = 1883;

const char *TOPIC_STATUS = "smartparking/status";
const char *TOPIC_CMD = "smartparking/cmd";
const char *TOPIC_ACK = "smartparking/ack";

#define TFT_CS 5
#define TFT_DC 15

Adafruit_ILI9341 tft = Adafruit_ILI9341(TFT_CS, TFT_DC);
Adafruit_FT6206 ctp = Adafruit_FT6206();

#define C_BG ILI9341_BLACK
#define C_HEADER 0x1082
#define C_FREE 0x07E0
#define C_OCCUPIED 0xF800
#define C_RESERVED 0xFFE0
#define C_SELECT 0x001F
#define C_DK ILI9341_BLACK
#define C_LT ILI9341_WHITE
#define C_FOOTER 0x2124

#define SCREEN_W 240
#define SCREEN_H 320
#define HEADER_H 40
#define FOOTER_H 44
#define FOOTER_Y (SCREEN_H - FOOTER_H)
#define SLOT_COLS 2
#define SLOT_ROWS 3
#define SLOT_W (SCREEN_W / SLOT_COLS)
#define SLOT_H ((SCREEN_H - HEADER_H - FOOTER_H) / SLOT_ROWS)
```

```
#define PAD 4
#define NUM_SLOTS 6

struct SlotInfo
{
  bool occupied, reserved, available;
  int dist;
};

SlotInfo slots[NUM_SLOTS];
SlotInfo prevSlots[NUM_SLOTS]; // shadow copy for dirty detection

int selectedSlot = -1;
int prevSelected = -2; // -2 forces initial draw
int myReservedSlot = -1;
int prevReserved = -2;
bool wifiWasConn = false;

WiFiClient wifiClient;
PubSubClient mqtt(wifiClient);

//   Geometry helper

void slotRect(int idx, int &x, int &y, int &w, int &h)
{
  x = (idx % SLOT_COLS) * SLOT_W + PAD;
  y = HEADER_H + (idx / SLOT_COLS) * SLOT_H + PAD;
  w = SLOT_W - PAD * 2;
  h = SLOT_H - PAD * 2;
}

//   Draw one slot

void drawSlot(int idx)
{
  int x, y, w, h;
  slotRect(idx, x, y, w, h);

  uint16_t bg;
  const char *label;
  if (slots[idx].occupied)
  {
    bg = C_OCCUPIED;
    label = "OCCUPIED";
  }
  else if (slots[idx].reserved)
  {
    bg = C_RESERVED;
```

```
    label = "RESERVED";
  }
  else
  {
    bg = C_FREE;
    label = "FREE";
  }

  // Clear the selection-border zone before drawing
  tft.fillRoundRect(x - 3, y - 3, w + 6, h + 6, 6, C_BG);

  if (idx == selectedSlot)
    tft.fillRoundRect(x - 3, y - 3, w + 6, h + 6, 6, C_SELECT);

  tft.fillRoundRect(x, y, w, h, 4, bg);

  tft.setTextColor(C_DK);
  tft.setTextSize(3);
  tft.setCursor(x + w / 2 - 9, y + 8);
  tft.print(idx + 1);

  tft.setTextSize(1);
  int lw = strlen(label) * 6;
  tft.setCursor(x + w / 2 - lw / 2, y + h - 16);
  tft.print(label);

  if (!slots[idx].occupied && slots[idx].dist < 500)
  {
    char buf[8];
    snprintf(buf, sizeof(buf), "%dcm", slots[idx].dist);
    int dw = strlen(buf) * 6;
    tft.setCursor(x + w / 2 - dw / 2, y + h - 27);
    tft.print(buf);
  }
}

//   Draw footer

void drawFooter()
{
  tft.fillRect(0, FOOTER_Y, SCREEN_W, FOOTER_H, C_FOOTER);

  if (selectedSlot == -1)
  {
    tft.setTextColor(0x8410);
    tft.setTextSize(1);
    tft.setCursor(28, FOOTER_Y + 16);
    tft.print("Tap a slot to select");
```

```
    return;
  }
  if (slots[selectedSlot].occupied)
  {
    tft.setTextColor(C_OCCUPIED);
    tft.setTextSize(1);
    tft.setCursor(20, FOOTER_Y + 16);
    tft.print("Slot ");
    tft.print(selectedSlot + 1);
    tft.print(" is occupied");
    return;
  }
  if (selectedSlot == myReservedSlot)
  {
    tft.fillRoundRect(16, FOOTER_Y + 8, 208, 28, 5, C_OCCUPIED);
    tft.setTextColor(C_LT);
    tft.setTextSize(1);
    tft.setCursor(44, FOOTER_Y + 18);
    tft.print("CANCEL RESERVATION");
  }
  else if (slots[selectedSlot].reserved)
  {
    tft.setTextColor(C_RESERVED);
    tft.setTextSize(1);
    tft.setCursor(20, FOOTER_Y + 16);
    tft.print("Slot ");
    tft.print(selectedSlot + 1);
    tft.print(" already reserved");
  }
  else
  {
    tft.fillRoundRect(16, FOOTER_Y + 8, 208, 28, 5, C_FREE);
    tft.setTextColor(C_DK);
    tft.setTextSize(1);
    tft.setCursor(48, FOOTER_Y + 18);
    tft.print("RESERVE SLOT ");
    tft.print(selectedSlot + 1);
  }
}

//   Draw header

void drawHeader()
{
  tft.fillRect(0, 0, SCREEN_W, HEADER_H, C_HEADER);
  tft.setTextColor(C_LT);
  tft.setTextSize(2);
  tft.setCursor(8, 12);
```

```
  tft.print("SMART PARKING");
  tft.fillCircle(SCREEN_W - 14, 20, 6,
                 (WiFi.status() == WL_CONNECTED) ? C_FREE : C_OCCUPIED);
}

//   WiFi dot only

void updateWifiDot()
{
  bool conn = (WiFi.status() == WL_CONNECTED);
  if (conn != wifiWasConn)
  {
    tft.fillCircle(SCREEN_W - 14, 20, 6, conn ? C_FREE : C_OCCUPIED);
    wifiWasConn = conn;
  }
}

//   Full initial paint

void drawAll()
{
  tft.fillScreen(C_BG);
  drawHeader();
  for (int i = 0; i < NUM_SLOTS; i++)
    drawSlot(i);
  drawFooter();
  memcpy(prevSlots, slots, sizeof(slots));
  prevSelected = selectedSlot;
  prevReserved = myReservedSlot;
  wifiWasConn = (WiFi.status() == WL_CONNECTED);
}

//   Partial update - only dirty regions

void updateDisplay()
{
  bool footerDirty = (selectedSlot != prevSelected) ||
                     (myReservedSlot != prevReserved);

  for (int i = 0; i < NUM_SLOTS; i++)
  {
    bool dataDiff = (slots[i].occupied != prevSlots[i].occupied ||
                     slots[i].reserved != prevSlots[i].reserved ||
                     slots[i].dist != prevSlots[i].dist);
    bool selDiff = (i == selectedSlot) != (i == prevSelected);

    if (dataDiff || selDiff)
    {
```

```
      drawSlot(i);
      if (i == selectedSlot || i == prevSelected)
        footerDirty = true;
    }
    if (dataDiff && i == selectedSlot)
      footerDirty = true;
  }

  // Redraw the previously selected slot to remove its highlight ring
  if (selectedSlot != prevSelected && prevSelected >= 0)
  {
    drawSlot(prevSelected);
  }

  if (footerDirty)
    drawFooter();
  updateWifiDot();

  memcpy(prevSlots, slots, sizeof(slots));
  prevSelected = selectedSlot;
  prevReserved = myReservedSlot;
}

//   MQTT

void publishCmd(const char *action, int slotId)
{
  char buf[64];
  snprintf(buf, sizeof(buf), "{\"action\":\"%s\",\"slot\":%d}", action,
 ↪  slotId);
  mqtt.publish(TOPIC_CMD, buf);
}

void onMessage(char *topic, byte *payload, unsigned int len)
{
  if (strcmp(topic, TOPIC_STATUS) != 0)
    return;

  StaticJsonDocument<512> doc;
  if (deserializeJson(doc, payload, len) != DeserializationError::Ok)
    return;

  JsonArray arr = doc["slots"].as<JsonArray>();
  int i = 0;
  for (JsonObject s : arr)
  {
    if (i >= NUM_SLOTS)
      break;
```

```
    slots[i].occupied = s["occupied"].as<bool>();
    slots[i].reserved = s["reserved"].as<bool>();
    slots[i].available = s["available"].as<bool>();
    slots[i].dist = s["dist"].as<int>();
    i++;
  }

  if (myReservedSlot >= 0 && slots[myReservedSlot].occupied)
  {
    myReservedSlot = -1;
    selectedSlot = -1;
  }

  updateDisplay();
}

//   Touch

void handleTouch()
{
  if (!ctp.touched())
    return;
  delay(10);
  if (!ctp.touched())
    return;

  TS_Point p = ctp.getPoint();
  p.x = map(p.x, 0, 240, 240, 0);
  p.y = map(p.y, 0, 320, 320, 0);

  if (p.y >= FOOTER_Y && selectedSlot != -1)
  {
    if (!slots[selectedSlot].occupied)
    {
      if (selectedSlot == myReservedSlot)
      {
        publishCmd("release", selectedSlot + 1);
        myReservedSlot = -1;
        selectedSlot = -1;
      }
      else if (!slots[selectedSlot].reserved)
      {
        if (myReservedSlot >= 0)
          publishCmd("release", myReservedSlot + 1);
        publishCmd("reserve", selectedSlot + 1);
        myReservedSlot = selectedSlot;
      }
    }
```

```
    updateDisplay();
    return;
  }

  if (p.y >= HEADER_H && p.y < FOOTER_Y)
  {
    int idx = (p.y - HEADER_H) / SLOT_H * SLOT_COLS + p.x / SLOT_W;
    if (idx >= 0 && idx < NUM_SLOTS)
    {
      selectedSlot = (selectedSlot == idx) ? -1 : idx;
      updateDisplay();
    }
  }
}

//   Connections

void connectWiFi()
{
  WiFi.begin(WIFI_SSID, WIFI_PASS);
  tft.setTextColor(0x8410);
  tft.setTextSize(1);
  tft.setCursor(10, 140);
  tft.print("Connecting WiFi...");
  while (WiFi.status() != WL_CONNECTED)
    delay(500);
}

void connectMQTT()
{
  String cid = "ParkClient-" + String((uint32_t)ESP.getEfuseMac(), HEX);
  while (!mqtt.connected())
  {
    if (mqtt.connect(cid.c_str()))
    {
      mqtt.subscribe(TOPIC_STATUS);
      mqtt.subscribe(TOPIC_ACK);
    }
    else
    {
      delay(2000);
    }
  }
}

//   Setup / Loop

void setup()
```

```
{
  Serial.begin(115200);
  tft.begin();
  tft.setRotation(0);
  tft.fillScreen(C_BG);
  tft.setTextColor(C_LT);
  tft.setTextSize(2);
  tft.setCursor(20, 110);
  tft.print("Smart Parking");

  if (!ctp.begin(40))
  {
    tft.fillScreen(C_OCCUPIED);
    tft.setCursor(10, 140);
    tft.setTextSize(2);
    tft.print("Touch FAIL");
    while (1)
      ;
  }

  connectWiFi();
  mqtt.setServer(BROKER, BROKER_PORT);
  mqtt.setBufferSize(1024);
  mqtt.setCallback(onMessage);
  connectMQTT();

  tft.fillScreen(C_BG);
  tft.setTextColor(C_FREE);
  tft.setTextSize(2);
  tft.setCursor(30, 150);
  tft.print("Connected!");
  delay(800);

  memset(slots, 0, sizeof(slots));
  memset(prevSlots, 0, sizeof(prevSlots));
  drawAll();
}

void loop()
{
  if (!mqtt.connected())
    connectMQTT();
  mqtt.loop();
  handleTouch();
  updateWifiDot();
}
```

JSON

SERVER

```
{
  "version": 1,
  "author": "SmartParking",
  "editor": "wokwi",
  "parts": [
    { "type": "board-esp32-devkit-c-v4", "id": "esp1", "top": 460,
     ↪  "left": -220, "attrs": {} },
    { "type": "wokwi-hc-sr04", "id": "sr1", "top":   0, "left": -320,
     ↪  "attrs": { "distance": "150" } },
    { "type": "wokwi-hc-sr04", "id": "sr2", "top":   0, "left": -100,
     ↪  "attrs": { "distance": "10"  } },
    { "type": "wokwi-hc-sr04", "id": "sr3", "top": 130, "left": -320,
     ↪  "attrs": { "distance": "180" } },
    { "type": "wokwi-hc-sr04", "id": "sr4", "top": 130, "left": -100,
     ↪  "attrs": { "distance": "200" } },
    { "type": "wokwi-hc-sr04", "id": "sr5", "top": 260, "left": -320,
     ↪  "attrs": { "distance": "160" } },
    { "type": "wokwi-hc-sr04", "id": "sr6", "top": 260, "left": -100,
     ↪  "attrs": { "distance": "8"   } }
  ],
  "connections": [
    [ "sr1:VCC", "esp1:3V3", "red", [] ], [ "sr1:GND", "esp1:GND.1",
     ↪  "black", [] ],
    [ "sr1:TRIG", "esp1:12", "green", [] ], [ "sr1:ECHO", "esp1:14",
     ↪  "blue", [] ],
    [ "sr2:VCC", "esp1:3V3", "red", [] ], [ "sr2:GND", "esp1:GND.1",
     ↪  "black", [] ],
    [ "sr2:TRIG", "esp1:13", "green", [] ], [ "sr2:ECHO", "esp1:35",
     ↪  "blue", [] ],
    [ "sr3:VCC", "esp1:3V3", "red", [] ], [ "sr3:GND", "esp1:GND.1",
     ↪  "black", [] ],
    [ "sr3:TRIG", "esp1:25", "green", [] ], [ "sr3:ECHO", "esp1:34",
     ↪  "blue", [] ],
    [ "sr4:VCC", "esp1:3V3", "red", [] ], [ "sr4:GND", "esp1:GND.2",
     ↪  "black", [] ],
    [ "sr4:TRIG", "esp1:26", "green", [] ], [ "sr4:ECHO", "esp1:32",
     ↪  "blue", [] ],
    [ "sr5:VCC", "esp1:3V3", "red", [] ], [ "sr5:GND", "esp1:GND.2",
     ↪  "black", [] ],
    [ "sr5:TRIG", "esp1:27", "green", [] ], [ "sr5:ECHO", "esp1:4",
     ↪  "blue", [] ],
    [ "sr6:VCC", "esp1:3V3", "red", [] ], [ "sr6:GND", "esp1:GND.2",
     ↪  "black", [] ],
    [ "sr6:TRIG", "esp1:33", "green", [] ], [ "sr6:ECHO", "esp1:17",
     ↪  "blue", [] ]
```

```
  ],
  "dependencies": {}
}
```

CLIENT

```
{
  "version": 1,
  "author": "SmartParking",
  "editor": "wokwi",
  "parts": [
    {
      "type": "board-esp32-devkit-c-v4",
      "id": "esp1",
      "top": 64,
      "left": -264,
      "attrs": {}
    },
    {
      "type": "board-ili9341-cap-touch",
      "id": "lcd1",
      "top": -146,
      "left": -48,
      "attrs": {}
    }
  ],
  "connections": [
    ["lcd1:VCC",  "esp1:3V3",   "red",     ["v19.2", "h-57.6", "v-105.6",
     ↪  "h-124.95"]],
    ["lcd1:GND",  "esp1:GND.2", "black",   ["v28.8", "h-76.8",
     ↪  "v-76.8"]],
    ["lcd1:CS",   "esp1:5",     "orange",  ["v0"]],
    ["lcd1:D/C",  "esp1:15",    "gold",    ["v0"]],
    ["lcd1:MOSI", "esp1:23",    "violet",  ["v105.6", "h-124.8",
     ↪  "v-144"]],
    ["lcd1:SCK",  "esp1:18",    "white",   ["v115.2", "h-144",
     ↪  "v-86.4"]],
    ["esp1:19",   "lcd1:MISO",  "magenta", ["h57.6", "v105.6",
     ↪  "h172.8"]],
    ["esp1:22",   "lcd1:SCL",   "yellow",  ["h48", "v163.2", "h192"]],
    ["esp1:21",   "lcd1:SDA",   "cyan",    ["h38.4", "v144", "h211.2"]]
  ],
  "dependencies": {}
}
```

PIN CONFIGURATION

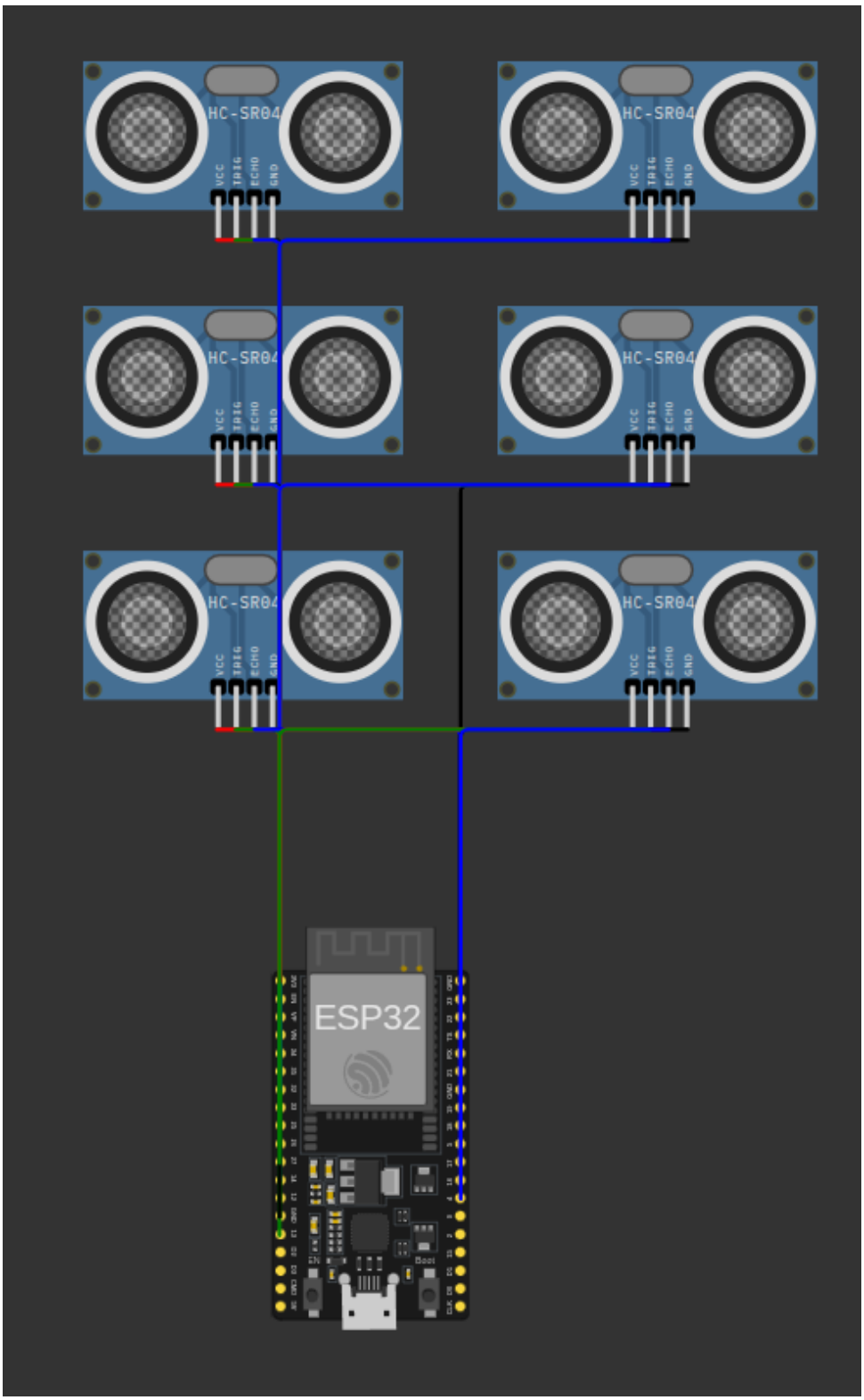


Figure 1: Server

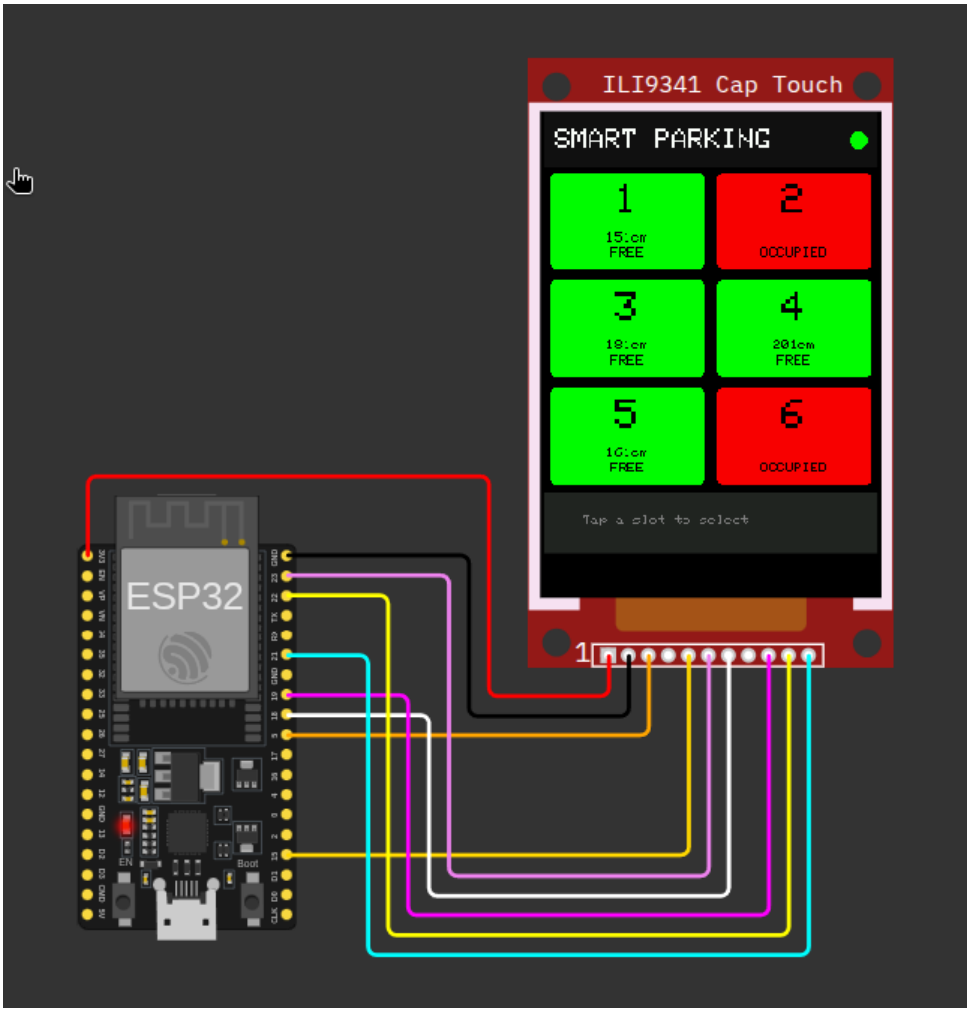


Figure 2: Client

EX.NO: 11	ELECTRONIC ROAD PASSING SYSTEM
20/03/2026	

AIM

To simulate electronic road passing system and toll collection without human intervention and send SMS to the vehicle owner using MQTT Broker.

APPARATUS REQUIRED

- Espressif ESP32 Dev Module
- MFRC522 RFID Scanner
- SSD1306 OLED 128X64
- Servo Motor
- 3xLED
- Buzzer

PRE-CONDITIONS

POST-CONDITIONS

CODE

```
#include <SPI.h>
#include <MFRC522.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <ESP32Servo.h>
#include <WiFi.h>
#include <WebServer.h>
#include <PubSubClient.h>
#include <qrcode.h>

#define RFID_SS_PIN  5
#define RFID_RST_PIN 4
#define SERVO_PIN    13
#define LED_GREEN    25
#define LED_YELLOW   26
#define LED_RED      27
#define BUZZER_PIN   14

const char *WIFI_SSID      = "Wokwi-GUEST";
const char *WIFI_PASSWORD  = "";
const char *MQTT_BROKER    = "100.64.0.2";
const int   MQTT_PORT      = 1883;
```

```
const char *MQTT_CLIENT_ID = "ZOOP_ESP32_001";
const char *TOPIC_STATUS   = "zoop/status/entry";

#define TOLL_AMOUNT       65.0f
#define LOW_BAL_THRESHOLD 100.0f
#define PENALTY_FACTOR    2
#define BARRIER_OPEN_MS   3000
#define OLED_WIDTH        128
#define OLED_HEIGHT       64
#define OLED_I2C_ADDR     0x3C

struct Vehicle {
  const char *uid;
  const char *vehicleNo;
  const char *phone;
  float       balance;
};

Vehicle db[] = {
  { "01020304", "TN01AB1234", "9900000001", 500.0f },
  { "11223344", "KA03CD5678", "9900000002",  80.0f },
  { "55667788", "MH12EF9999", "9900000003",   0.0f }
};
const int DB_SIZE = sizeof(db) / sizeof(db[0]);

MFRC522          rfid(RFID_SS_PIN, RFID_RST_PIN);
Adafruit_SSD1306 oled(OLED_WIDTH, OLED_HEIGHT, &Wire, -1);
Servo            barrier;
WiFiClient       wifiClient;
PubSubClient     mqtt(wifiClient);
WebServer        server(80);

String pendingUID       = "";
String pendingVehicleNo = "";
float  pendingAmount    = 0;
bool   paymentPending   = false;
bool   isProcessing     = false;

void connectWiFi();
void connectMQTT();
void handleRFID();
void processSufficient(Vehicle &v);
void processLowBalanceAlert(Vehicle &v);
void processZeroBalance(Vehicle &v);
void openBarrier();
void closeBarrier();
void clearLEDs();
void buzzerBeep(int times, int durationMs);
```

```
void oledIdle();
void oledStatus(const char *line1, const char *line2,
                const char *line3, const char *line4);
void oledQR(const char *url);
void publishMQTT(const char *topic, const char *payload);
String   uidToString(MFRC522::Uid &uid);
Vehicle *findVehicle(const String &uid);

void handlePayPage()
{
  if (!paymentPending) {
    server.send(404, "text/plain", "No pending payment.");
    return;
  }

  String html = R"rawhtml(
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>ZOOP Payment</title>
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; }
    body {
      font-family: Arial, sans-serif;
      background: #0f0f0f;
      color: #f0f0f0;
      display: flex;
      justify-content: center;
      align-items: center;
      min-height: 100vh;
      padding: 20px;
    }
    .card {
      background: #1c1c1e;
      border-radius: 16px;
      padding: 30px;
      max-width: 380px;
      width: 100%;
      box-shadow: 0 8px 30px rgba(0,0,0,0.5);
    }
    .logo {
      font-size: 28px;
      font-weight: bold;
      color: #f5a623;
      text-align: center;
      margin-bottom: 6px;
```

```
}
.subtitle {
  text-align: center;
  color: #888;
  font-size: 13px;
  margin-bottom: 24px;
}
.badge {
  background: #ff3b30;
  color: white;
  border-radius: 8px;
  padding: 10px 14px;
  font-size: 13px;
  margin-bottom: 20px;
  text-align: center;
}
.row {
  display: flex;
  justify-content: space-between;
  padding: 10px 0;
  border-bottom: 1px solid #2c2c2e;
  font-size: 14px;
}
.row .label { color: #888; }
.row .value { font-weight: bold; color: #f0f0f0; }
.total-row {
  display: flex;
  justify-content: space-between;
  padding: 16px 0 10px;
  font-size: 17px;
  font-weight: bold;
}
.total-row .value { color: #f5a623; }
.pay-btn {
  width: 100%;
  padding: 16px;
  background: #f5a623;
  color: #000;
  border: none;
  border-radius: 12px;
  font-size: 17px;
  font-weight: bold;
  cursor: pointer;
  margin-top: 22px;
  transition: background 0.2s;
}
.pay-btn:hover { background: #e6941a; }
.note {
```

```
      font-size: 11px;
      color: #555;
      text-align: center;
      margin-top: 14px;
    }
  </style>
</head>
<body>
<div class="card">
  <div class="logo">&#128663; ZOOP</div>
  <div class="subtitle">Toll Payment Portal</div>
  <div class="badge">&#9888;&#65039; Zero Balance &mdash; Entry
   ↪  Blocked</div>
  <div class="row">
    <span class="label">Vehicle</span>
    <span class="value">)rawhtml";
  html += pendingVehicleNo;
  html += R"rawhtml(</span>
  </div>
  <div class="row">
    <span class="label">Toll Charge</span>
    <span class="value">&#8377;65</span>
  </div>
  <div class="row">
    <span class="label">Zero Balance Penalty</span>
    <span class="value">&#8377;65</span>
  </div>
  <div class="total-row">
    <span class="label">Total Payable</span>
    <span class="value">&#8377;)rawhtml";
  html += String((int)pendingAmount);
  html += R"rawhtml(</span>
  </div>
  <form method="POST" action="/confirm">
    <input type="hidden" name="uid" value=")rawhtml";
  html += pendingUID;
  html += R"rawhtml(">
    <button class="pay-btn" type="submit">&#128179; PAY &amp;
     ↪  ENTER</button>
  </form>
  <div class="note">Simulation: clicking confirms payment and opens
   ↪  barrier.</div>
</div>
</body>
</html>
)rawhtml";

  server.send(200, "text/html", html);
```

```
}

void handleConfirm()
{
  if (!paymentPending) {
    server.send(400, "text/plain", "No pending payment.");
    return;
  }

  String uid = server.arg("uid");
  if (uid != pendingUID) {
    server.send(400, "text/plain", "UID mismatch.");
    return;
  }

  paymentPending = false;
  clearLEDs();
  digitalWrite(LED_GREEN, HIGH);
  buzzerBeep(1, 200);
  oledStatus("  ZOOP PAYMENT", "  CONFIRMED!",
             "ENTRY GRANTED", pendingVehicleNo.c_str());
  openBarrier();

  char payload[200];
  snprintf(payload, sizeof(payload),

     ↪  "{\"vehicle\":\"%s\",\"status\":\"ENTRY_GRANTED_POST_PAYMENT\",\"amount\":%.0
    pendingVehicleNo.c_str(), pendingAmount);
  publishMQTT(TOPIC_STATUS, payload);

  char topic[64];
  snprintf(topic, sizeof(topic), "zoop/%s", pendingVehicleNo.c_str());
  char userPayload[256];
  snprintf(userPayload, sizeof(userPayload),
    "{\"status\":\"PAYMENT_CONFIRMED\",\"vehicle\":\"%s\","
    "\"amount\":%.0f,\"message\":\"Payment successful. Entry granted with
     ↪  penalty waived.\"}",
    pendingVehicleNo.c_str(), pendingAmount);
  publishMQTT(topic, userPayload);

  delay(BARRIER_OPEN_MS);
  closeBarrier();
  clearLEDs();
  oledIdle();
  isProcessing = false;

  server.sendHeader("Location", "/thankyou", true);
  server.send(302, "text/plain", "");
```

```
}

void handleThankYou()
{
  String html = R"rawhtml(
<!DOCTYPE html><html><head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width,initial-scale=1">
<title>ZOOP</title>
<style>
body{background:#0f0f0f;color:#f0f0f0;font-family:Arial,sans-serif;

 ↪  display:flex;justify-content:center;align-items:center;min-height:100vh;}
.box{text-align:center;padding:40px;background:#1c1c1e;border-radius:16px;}
.icon{font-size:60px;margin-bottom:16px;}
h2{color:#34c759;margin-bottom:8px;}
p{color:#888;font-size:14px;}
</style></head><body>
<div class="box">
  <div class="icon">&#9989;</div>
  <h2>Entry Granted!</h2>
  <p>Barrier is now open. Safe travels.</p>
  <p
   ↪  style="margin-top:20px;color:#f5a623;font-size:20px;font-weight:bold;">&#128663
   ↪  ZOOP</p>
</div></body></html>
)rawhtml";
  server.send(200, "text/html", html);
}

void setup()
{
  Serial.begin(115200);
  Serial.println("\n=== ZOOP Initializing ===");

  pinMode(LED_GREEN,  OUTPUT);
  pinMode(LED_YELLOW, OUTPUT);
  pinMode(LED_RED,    OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  clearLEDs();
  digitalWrite(BUZZER_PIN, LOW);

  barrier.attach(SERVO_PIN);
  closeBarrier();

  Wire.begin(21, 22);
  if (!oled.begin(SSD1306_SWITCHCAPVCC, OLED_I2C_ADDR)) {
    Serial.println("OLED init failed!");
```

```
    while (true);
  }
  oled.clearDisplay();
  oled.setTextColor(WHITE);

  SPI.begin();
  rfid.PCD_Init();
  Serial.println("MFRC522 ready.");

  connectWiFi();

  mqtt.setServer(MQTT_BROKER, MQTT_PORT);
  connectMQTT();

  server.on("/pay",      HTTP_GET,  handlePayPage);
  server.on("/confirm",  HTTP_POST, handleConfirm);
  server.on("/thankyou", HTTP_GET,  handleThankYou);
  server.begin();
  Serial.println("Web server started.");
  Serial.print("ESP32 IP: ");
  Serial.println(WiFi.localIP());

  oledIdle();
  Serial.println("=== ZOOP Ready ===\n");
}

void loop()
{
  if (!mqtt.connected()) connectMQTT();
  mqtt.loop();
  server.handleClient();
  handleRFID();
}

void handleRFID()
{
  if (isProcessing) return;
  if (!rfid.PICC_IsNewCardPresent()) return;
  if (!rfid.PICC_ReadCardSerial())   return;

  isProcessing = true;

  String uid = uidToString(rfid.uid);
  Serial.println("Tag UID: " + uid);

  Vehicle *v = findVehicle(uid);

  if (v == nullptr) {
```

```
    clearLEDs();
    digitalWrite(LED_RED, HIGH);
    buzzerBeep(3, 100);
    oledStatus("  UNREGISTERED", "  VEHICLE", uid.c_str(), "Contact
↪   ZOOP");
    delay(2500);
    clearLEDs();
    oledIdle();
    isProcessing = false;
  }
  else if (v->balance >= TOLL_AMOUNT) {
    processSufficient(*v);
    isProcessing = false;
  }
  else {
    processZeroBalance(*v);
  }

  rfid.PICC_HaltA();
  rfid.PCD_StopCrypto1();
}

void processSufficient(Vehicle &v)
{
  v.balance -= TOLL_AMOUNT;

  clearLEDs();
  digitalWrite(LED_GREEN, HIGH);
  buzzerBeep(1, 150);

  char line2[32], line3[32], line4[32];
  snprintf(line2, sizeof(line2), "Deducted:  Rs.%.0f", TOLL_AMOUNT);
  snprintf(line3, sizeof(line3), "Balance:   Rs.%.0f", v.balance);
  snprintf(line4, sizeof(line4), "%s", v.vehicleNo);
  oledStatus("  ENTRY GRANTED", line2, line3, line4);

  openBarrier();

  char payload[200];
  snprintf(payload, sizeof(payload),

     ↪  "{\"vehicle\":\"%s\",\"status\":\"ENTRY_GRANTED\",\"deducted\":%.0f,\"balance
    v.vehicleNo, TOLL_AMOUNT, v.balance);
  publishMQTT(TOPIC_STATUS, payload);

  char topic[64];
  snprintf(topic, sizeof(topic), "zoop/%s", v.vehicleNo);
  char userPayload[256];
```

```
  snprintf(userPayload, sizeof(userPayload),
    "{\"status\":\"TOLL_DEDUCTED\",\"vehicle\":\"%s\","
    "\"deducted\":%.0f,\"balance\":%.0f,\"message\":\"Toll paid
     ↪  successfully. Entry granted.\"}",
    v.vehicleNo, TOLL_AMOUNT, v.balance);
  publishMQTT(topic, userPayload);

  if (v.balance <= LOW_BAL_THRESHOLD && v.balance > 0) {
    delay(1500);
    processLowBalanceAlert(v);
  }

  delay(BARRIER_OPEN_MS);
  closeBarrier();
  clearLEDs();
  oledIdle();
}

void processLowBalanceAlert(Vehicle &v)
{
  clearLEDs();
  digitalWrite(LED_YELLOW, HIGH);
  buzzerBeep(3, 200);

  char line2[32];
  snprintf(line2, sizeof(line2), "Balance: Rs.%.0f", v.balance);
  oledStatus("  LOW BALANCE", line2, "MQTT Alert Sent", v.vehicleNo);

  char topic[64];
  snprintf(topic, sizeof(topic), "zoop/%s", v.vehicleNo);

  char payload[256];
  snprintf(payload, sizeof(payload),
    "{\"status\":\"LOW_BALANCE\",\"vehicle\":\"%s\","
    "\"balance\":%.0f,\"message\":\"Recharge your ZOOP wallet soon!\"}",
    v.vehicleNo, v.balance);

  publishMQTT(topic, payload);
  Serial.printf("MQTT Alert sent to topic: %s\n", topic);

  delay(2000);
  clearLEDs();
}

void processZeroBalance(Vehicle &v)
{
  clearLEDs();
  digitalWrite(LED_RED, HIGH);
```

```
  buzzerBeep(2, 500);

  pendingUID       = String(v.uid);
  pendingVehicleNo = String(v.vehicleNo);
  pendingAmount    = TOLL_AMOUNT * PENALTY_FACTOR;
  paymentPending   = true;

  char url[128];
  snprintf(url, sizeof(url), "http://100.64.0.2:8888/pay?uid=%s",
↪  v.uid);
  Serial.println("Payment URL: " + String(url));

  oledQR(url);

  char topic[64];
  snprintf(topic, sizeof(topic), "zoop/%s", v.vehicleNo);

  char payload[300];
  snprintf(payload, sizeof(payload),
    "{\"status\":\"ZERO_BALANCE\",\"vehicle\":\"%s\","
    "\"payable\":%.0f,\"message\":\"Scan QR at toll to pay and
     ↪  enter.\"}",
    v.vehicleNo, pendingAmount);

  publishMQTT(topic, payload);
}

void openBarrier()
{
  barrier.write(90);
  Serial.println("Barrier OPEN");
}

void closeBarrier()
{
  barrier.write(0);
  Serial.println("Barrier CLOSED");
}

void clearLEDs()
{
  digitalWrite(LED_GREEN,  LOW);
  digitalWrite(LED_YELLOW, LOW);
  digitalWrite(LED_RED,    LOW);
}

void buzzerBeep(int times, int durationMs)
{
```

```
  for (int i = 0; i < times; i++) {
    digitalWrite(BUZZER_PIN, HIGH);
    delay(durationMs);
    digitalWrite(BUZZER_PIN, LOW);
    if (i < times - 1) delay(100);
  }
}

void oledIdle()
{
  oled.clearDisplay();
  oled.setTextSize(2);
  oled.setCursor(20, 10);
  oled.println("  ZOOP");
  oled.setTextSize(1);
  oled.setCursor(15, 38);
  oled.println("Place Your RFID Tag");
  oled.setCursor(28, 52);
  oled.println("to Enter Toll");
  oled.display();
}

void oledStatus(const char *line1, const char *line2,
                const char *line3, const char *line4)
{
  oled.clearDisplay();
  oled.setTextSize(1);
  oled.setCursor(0,  0); oled.println(line1);
  oled.drawLine(0, 10, 127, 10, WHITE);
  oled.setCursor(0, 16); oled.println(line2);
  oled.setCursor(0, 30); oled.println(line3);
  oled.setCursor(0, 44); oled.println(line4);
  oled.display();
}

void oledQR(const char *url)
{
  oled.clearDisplay();
  oled.setTextSize(1);
  oled.setCursor(0, 0);
  oled.println("SCAN TO PAY & ENTER");
  oled.drawLine(0, 10, 127, 10, WHITE);

  QRCode qrcode;
  uint8_t qrcodeData[qrcode_getBufferSize(3)];
  qrcode_initText(&qrcode, qrcodeData, 3, ECC_LOW, url);

  int qrSize   = qrcode.size;
```

```
  int scale    = 52 / qrSize;
  if (scale < 1) scale = 1;

  int offsetX  = (128 - qrSize * scale) / 2;
  int offsetY  = 12 + (52 - qrSize * scale) / 2;

  for (int y = 0; y < qrSize; y++) {
    for (int x = 0; x < qrSize; x++) {
      if (qrcode_getModule(&qrcode, x, y)) {
        for (int dy = 0; dy < scale; dy++)
          for (int dx = 0; dx < scale; dx++)
            oled.drawPixel(offsetX + x*scale + dx,
                           offsetY + y*scale + dy, WHITE);
      }
    }
  }

  oled.display();
  Serial.println("QR Code displayed on OLED.");
}

void connectWiFi()
{
  Serial.print("Connecting to WiFi");
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  int attempts = 0;
  while (WiFi.status() != WL_CONNECTED && attempts < 20) {
    delay(500);
    Serial.print(".");
    attempts++;
  }
  if (WiFi.status() == WL_CONNECTED)
    Serial.println("\nWiFi connected. IP: " + WiFi.localIP().toString());
  else
    Serial.println("\nWiFi failed.");
}

void connectMQTT()
{
  int attempts = 0;
  while (!mqtt.connected() && attempts < 5) {
    Serial.print("Connecting to MQTT broker...");
    if (mqtt.connect(MQTT_CLIENT_ID)) {
      Serial.println(" connected.");
      return;
    } else {
      Serial.printf(" failed (rc=%d). Retry in 2s.\n", mqtt.state());
      delay(2000);
```

```
      attempts++;
    }
  }
  if (!mqtt.connected()) {
    Serial.println("MQTT connection failed - continuing without MQTT");
  }
}

void publishMQTT(const char *topic, const char *payload)
{
  if (!mqtt.connected()) {
    Serial.println("MQTT not connected - skipping publish");
    return;
  }
  if (mqtt.publish(topic, payload))
    Serial.printf("MQTT Published -> [%s] %s\n", topic, payload);
  else
    Serial.println("MQTT publish failed.");
}

String uidToString(MFRC522::Uid &uid)
{
  String s = "";
  for (byte i = 0; i < uid.size; i++) {
    if (uid.uidByte[i] < 0x10) s += "0";
    s += String(uid.uidByte[i], HEX);
  }
  s.toUpperCase();
  return s;
}

Vehicle *findVehicle(const String &uid)
{
  for (int i = 0; i < DB_SIZE; i++)
    if (String(db[i].uid) == uid) return &db[i];
  return nullptr;
}
```

JSON

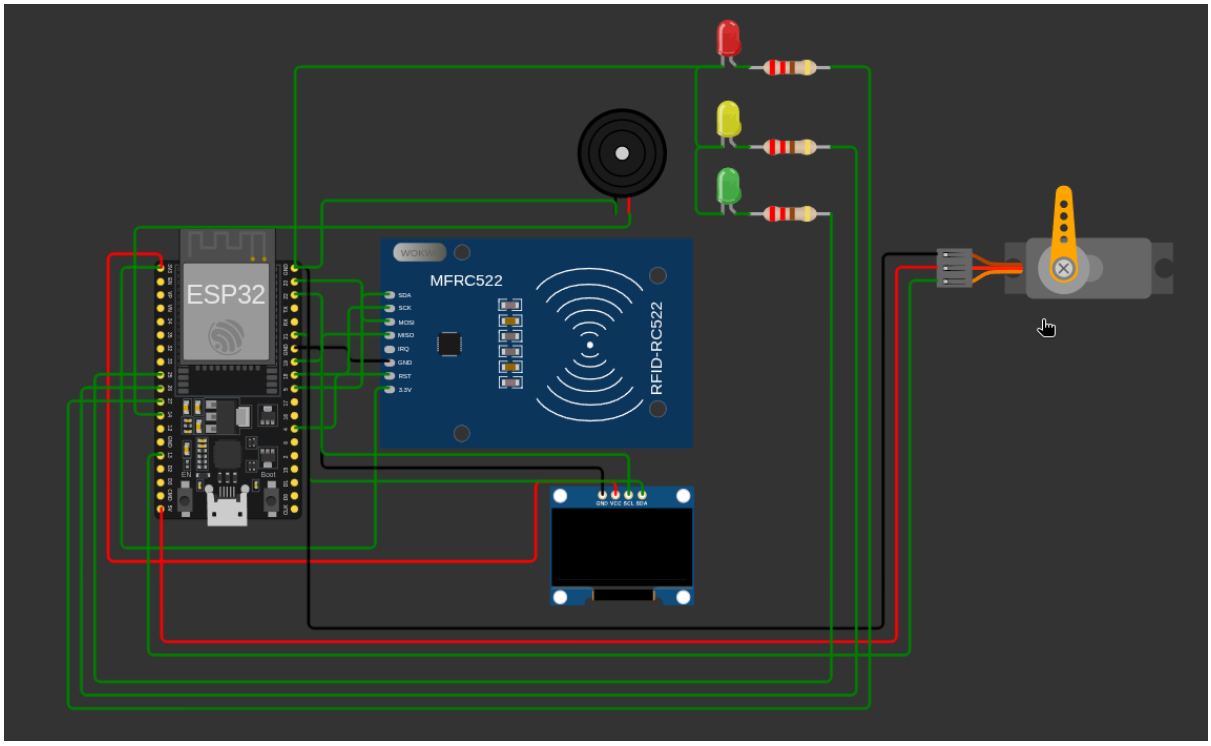
```
{
  "version": 1,
  "author": "Shree Kottes",
  "editor": "wokwi",
  "parts": [
    {
      "type": "board-esp32-devkit-c-v4",
```

```
      "id": "esp",
      "top": -182.4,
      "left": -388.76,
      "attrs": {}
    },
    { "type": "board-mfrc522", "id": "rfid1", "top": -175.45, "left":
     ↪  -226.96, "attrs": {} },
    {
      "type": "board-ssd1306",
      "id": "oled1",
      "top": 3.14,
      "left": -105.37,
      "attrs": { "i2cAddress": "0x3c" }
    },
    {
      "type": "wokwi-servo",
      "id": "servo1",
      "top": -213.2,
      "left": 172.8,
      "attrs": { "hornColor": "orange" }
    },
    { "type": "wokwi-led", "id": "led1", "top": -339.6, "left": 3.8,
     ↪  "attrs": { "color": "red" } },
    {
      "type": "wokwi-led",
      "id": "led2",
      "top": -282,
      "left": 3.8,
      "attrs": { "color": "yellow" }
    },
    {
      "type": "wokwi-led",
      "id": "led3",
      "top": -234,
      "left": 3.8,
      "attrs": { "color": "limegreen" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r1",
      "top": -303.25,
      "left": 38.4,
      "attrs": { "value": "220" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r2",
      "top": -245.65,
```

```
      "left": 38.4,
      "attrs": { "value": "220" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r3",
      "top": -197.65,
      "left": 38.4,
      "attrs": { "value": "220" }
    },
    {
      "type": "wokwi-buzzer",
      "id": "bz1",
      "top": -276,
      "left": -84.6,
      "attrs": { "volume": "0.1" }
    }
  ],
  "connections": [
    [ "esp:TX", "$serialMonitor:RX", "", [] ],
    [ "esp:RX", "$serialMonitor:TX", "", [] ],
    [ "led1:A", "r1:1", "green", [ "v0" ] ],
    [ "led2:A", "r2:1", "green", [ "v0" ] ],
    [ "led3:A", "r3:1", "green", [ "v0" ] ],
    [
      "rfid1:3.3V",
      "esp:3V3",
      "green",
      [ "h-9.6", "v105.6", "h0", "v9.6", "h-182.4", "v-201.6" ]
    ],
    [ "rfid1:GND", "esp:GND.3", "black", [ "h-28.8", "v-9.6" ] ],
    [ "rfid1:SDA", "esp:5", "green", [ "h-19.2", "v67.2" ] ],
    [ "rfid1:SCK", "esp:18", "green", [ "h-28.8", "v48" ] ],
    [ "esp:23", "rfid1:MOSI", "green", [ "h48", "v28.8" ] ],
    [ "esp:19", "rfid1:MISO", "green", [ "h19.2", "v-19.2" ] ],
    [ "esp:4", "rfid1:RST", "green", [ "h28.8", "v-38.4" ] ],
    [
      "oled1:VCC",
      "esp:3V3",
      "red",
      [ "v-9.6", "h-57.45", "v57.6", "h-307.2", "v-220.8", "h47.85" ]
    ],
    [ "oled1:GND", "esp:GND.3", "black", [ "v-19.2", "h-201.6", "v-86.4"
     ↪  ] ],
    [ "esp:22", "oled1:SCL", "green", [ "h19.2", "v115.2", "h38.4" ] ],
    [ "oled1:SDA", "esp:21", "green", [ "v-9.6", "h-239.93", "v-105.6" ]
     ↪  ],
    [ "esp:13", "servo1:PWM", "green", [ "h-9.45", "v144", "h547.2",
     ↪  "v-4.8" ] ],
```

```
    [ "esp:5V", "servo1:V+", "red", [ "h0.15", "v96", "h528", "v-268.9" ]
     ↪  ],
    [ "servo1:GND", "esp:GND.2", "black", [ "h-38.4", "v268.8",
     ↪  "h-412.8", "v-259.2" ] ],
    [ "led1:C", "led2:C", "green", [ "v0", "h-18.8", "v57.6" ] ],
    [ "led2:C", "led3:C", "green", [ "v0", "h-18.8", "v48" ] ],
    [ "led1:C", "esp:GND.2", "green", [ "v0", "h-297.2" ] ],
    [ "r3:2", "esp:25", "green", [ "v336", "h-529.2", "v-220.8" ] ],
    [ "esp:26", "r2:2", "green", [ "h-57.45", "v220.8", "h556.8",
     ↪  "v-393.6" ] ],
    [ "r1:2", "esp:27", "green", [ "h27.6", "v460.8", "h-576", "v-220.8"
     ↪  ] ],
    [ "esp:14", "bz1:2", "green", [ "h-19.05", "v-134.4", "h355.6" ] ],
    [ "bz1:1", "esp:GND.2", "green", [ "v-9.6", "h-211.2", "v48" ] ]
  ],
  "dependencies": {}
}
```


PIN CONFIGURATION



EX.NO: 12	SMART GAS LEAKAGE DETECTION SYSTEM
20/03/2026	

AIM

To simulate Smart Gas Leakage Detection System and Sending alert notification to the owner using MQTT Broker.

APPARATUS REQUIRED

- Espressif ESP32 Dev Module
- Relay Module
- 2x LED
- MQ2 Gas Sensor
- OLED 1306
- Buzzer

PRE-CONDITIONS

POST-CONDITIONS

CODE

```
#include <WiFi.h>
#include <PubSubClient.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <ArduinoJson.h>
#include <ESP32Servo.h>

// WiFi Configuration
const char *WIFI_SSID = "Wokwi-GUEST";
const char *WIFI_PASSWORD = "";

// MQTT Configuration
const char *MQTT_BROKER = "100.64.0.1";
const int MQTT_PORT = 1883;
const char *MQTT_CLIENT = "ESP32-GasDetector";

const char *TOPIC_LEVEL = "home/gas/level";
const char *TOPIC_STATUS = "home/gas/status";
const char *TOPIC_ALERT = "home/gas/alert";
const char *TOPIC_VALVE = "home/gas/valve";
const char *TOPIC_FAN = "home/gas/fan";
const char *TOPIC_HEALTH = "home/gas/system/health";
```

```
const char *TOPIC_CONTROL = "home/gas/control";

// Pin Definitions
#define MQ2_ANALOG_PIN 34
#define MQ2_DIGITAL_PIN 35
#define BUZZER_PIN 26
#define LED_RED_PIN 27
#define LED_GREEN_PIN 25
#define RELAY_PIN 32
#define SERVO_PIN 33

// OLED Display
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

// Thresholds
int THRESHOLD_WARNING = 300;
int THRESHOLD_DANGER = 600;

// System State
enum GasState
{
  SAFE,
  WARNING_LEVEL,
  DANGER_LEVEL
};
GasState currentState = SAFE;
GasState previousState = SAFE;

bool valveOpen = true;
bool buzzerSilenced = false;
bool autoRecovery = false;
int servoAngle = 0;

bool testMode = false;
unsigned long testModeStart = 0;
unsigned long lastSensorRead = 0;
unsigned long lastHeartbeat = 0;
unsigned long lastBlink = 0;
bool blinkState = false;

WiFiClient wifiClient;
PubSubClient mqtt(wifiClient);
Servo ventServo;

// Function Prototypes
```

```
void connectWiFi();
void connectMQTT();
void mqttCallback(char *, byte *, unsigned int);
void handleGasLevel(int ppm);
void handleCommand(const String &cmd, const JsonDocument &doc);
void setValve(bool open, const char *reason);
void setFan(int angle, const char *reason, int ppm);
void systemReset();
void systemStatus(int ppm);
void updateOLED(int ppm);

void publishLevel(int ppm);
void publishStatus(int ppm);
void publishAlert(int ppm);
void publishValve(const char *reason);
void publishFan(const char *reason, int ppm);
void publishHealth();

void setup()
{
  Serial.begin(115200);
  Serial.println("\n=== Gas Detector Boot ===");

  pinMode(LED_RED_PIN, OUTPUT);
  pinMode(LED_GREEN_PIN, OUTPUT);
  pinMode(RELAY_PIN, OUTPUT);
  pinMode(MQ2_DIGITAL_PIN, INPUT);

  // Initialize to safe state
  digitalWrite(RELAY_PIN, LOW);
  noTone(BUZZER_PIN);
  digitalWrite(LED_RED_PIN, LOW);
  digitalWrite(LED_GREEN_PIN, HIGH);

  ventServo.attach(SERVO_PIN);
  ventServo.write(0);

  Wire.begin(21, 22);
  if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C))
  {
    Serial.println("OLED init failed!");
  }
  display.clearDisplay();
  display.setTextColor(SSD1306_WHITE);
  display.setTextSize(1);
  display.setCursor(0, 0);
  display.println("Smart Gas Detector");
  display.println("Booting...");
```

```
  display.display();
  delay(1000);

  connectWiFi();
  mqtt.setServer(MQTT_BROKER, MQTT_PORT);
  mqtt.setCallback(mqttCallback);
  connectMQTT();

  Serial.println("System ready.");
}

void loop()
{
  if (!mqtt.connected())
    connectMQTT();
  mqtt.loop();

  unsigned long now = millis();

  // Test mode auto-expire after 10 seconds
  if (testMode && (now - testModeStart >= 10000))
  {
    testMode = false;
    Serial.println("Test mode expired - resetting.");
    systemReset();
  }

  // Read sensor every 2 seconds
  if (now - lastSensorRead >= 2000)
  {
    lastSensorRead = now;

    // Digital pin: LOW = gas detected, HIGH = no gas
    int digitalValue = digitalRead(MQ2_DIGITAL_PIN);
    int ppm = (digitalValue == LOW) ? 1000 : 0;

    if (!testMode)
      handleGasLevel(ppm);

    publishLevel(ppm);
    updateOLED(ppm);
  }

  // Heartbeat every 30 seconds
  if (now - lastHeartbeat >= 30000)
  {
    lastHeartbeat = now;
    publishHealth();
```

```
  }

  // Blink LEDs and buzzer in WARNING state
  if (currentState == WARNING_LEVEL && (now - lastBlink >= 500))
  {
    lastBlink = now;
    blinkState = !blinkState;
    digitalWrite(LED_RED_PIN, blinkState);
    digitalWrite(LED_GREEN_PIN, !blinkState);
    if (!buzzerSilenced)
    {
      if (blinkState)
        tone(BUZZER_PIN, 1000);
      else
        noTone(BUZZER_PIN);
    }
  }
}

void connectWiFi()
{
  Serial.print("Connecting WiFi");
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  while (WiFi.status() != WL_CONNECTED)
  {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWiFi OK: " + WiFi.localIP().toString());
}

void connectMQTT()
{
  while (!mqtt.connected())
  {
    Serial.print("Connecting MQTT @ ");
    Serial.print(MQTT_BROKER);
    Serial.print("...");
    if (mqtt.connect(MQTT_CLIENT))
    {
      Serial.println("connected.");
      mqtt.subscribe(TOPIC_CONTROL);
      publishHealth();
    }
    else
    {
      Serial.print("failed (rc=");
      Serial.print(mqtt.state());
```

```
      Serial.println("). Retry in 3 s...");
      delay(3000);
    }
  }
}

void mqttCallback(char *topic, byte *payload, unsigned int length)
{
  String msg;
  for (unsigned int i = 0; i < length; i++)
    msg += (char)payload[i];
  Serial.println("[MQTT IN] " + String(topic) + " → " + msg);

  StaticJsonDocument<256> doc;
  if (deserializeJson(doc, msg))
  {
    Serial.println("JSON parse error - ignoring.");
    return;
  }
  handleCommand(doc["command"].as<String>(), doc);
}

void handleCommand(const String &cmd, const JsonDocument &doc)
{
  int ppm = analogRead(MQ2_ANALOG_PIN);
  Serial.println("[CMD] " + cmd);

  if (cmd == "VALVE_OPEN")
    setValve(true, "MANUAL");
  else if (cmd == "VALVE_CLOSE")
    setValve(false, "MANUAL");
  else if (cmd == "FAN_OFF")
    setFan(0, "MANUAL", ppm);
  else if (cmd == "FAN_PARTIAL")
    setFan(90, "MANUAL", ppm);
  else if (cmd == "FAN_ON")
    setFan(180, "MANUAL", ppm);
  else if (cmd == "BUZZER_SILENCE")
  {
    buzzerSilenced = true;
    noTone(BUZZER_PIN);
    Serial.println("Buzzer silenced.");
  }
  else if (cmd == "BUZZER_ENABLE")
  {
    buzzerSilenced = false;
    Serial.println("Buzzer re-enabled.");
  }
```

```
else if (cmd == "BUZZER_TEST")
{
  if (!buzzerSilenced)
  {
    tone(BUZZER_PIN, 2000);
    delay(2000);
    noTone(BUZZER_PIN);
  }
}
else if (cmd == "LED_TEST")
{
  for (int i = 0; i < 3; i++)
  {
    digitalWrite(LED_RED_PIN, HIGH);
    digitalWrite(LED_GREEN_PIN, HIGH);
    delay(400);
    digitalWrite(LED_RED_PIN, LOW);
    digitalWrite(LED_GREEN_PIN, LOW);
    delay(400);
  }
  digitalWrite(LED_GREEN_PIN, currentState == SAFE ? HIGH : LOW);
  digitalWrite(LED_RED_PIN, currentState == DANGER_LEVEL ? HIGH : LOW);
}
else if (cmd == "SYSTEM_RESET")
  systemReset();
else if (cmd == "SYSTEM_STATUS")
  systemStatus(ppm);
else if (cmd == "SYSTEM_REBOOT")
{
  Serial.println("Rebooting...");
  delay(500);
  ESP.restart();
}
else if (cmd == "TEST_ALARM")
{
  testMode = true;
  testModeStart = millis();
  currentState = DANGER_LEVEL;
  setValve(false, "TEST");
  setFan(180, "TEST", 999);
  if (!buzzerSilenced)
    tone(BUZZER_PIN, 2000);
  digitalWrite(LED_RED_PIN, HIGH);
  digitalWrite(LED_GREEN_PIN, LOW);
  publishAlert(999);
  Serial.println("TEST_ALARM → 10 s simulation.");
}
else if (cmd == "TEST_WARNING")
```

```
  {
    testMode = true;
    testModeStart = millis();
    currentState = WARNING_LEVEL;
    setFan(90, "TEST", 450);
    Serial.println("TEST_WARNING → 10 s simulation.");
  }
  else if (cmd == "SENSOR_CALIBRATE")
  {
    long sum = 0;
    for (int i = 0; i < 20; i++)
    {
      sum += analogRead(MQ2_ANALOG_PIN);
      delay(50);
    }
    Serial.println("Calibration baseline ADC: " + String(sum / 20));
  }
  else if (cmd == "HEALTH_PING")
    publishHealth();
  else if (cmd == "SET_THRESHOLD")
  {
    if (doc.containsKey("warning"))
      THRESHOLD_WARNING = doc["warning"].as<int>();
    if (doc.containsKey("danger"))
      THRESHOLD_DANGER = doc["danger"].as<int>();
    Serial.println("Thresholds → WARNING:" + String(THRESHOLD_WARNING) +
                   " DANGER:" + String(THRESHOLD_DANGER));
  }
  else if (cmd == "AUTO_RECOVERY_ON")
  {
    autoRecovery = true;
    Serial.println("Auto-recovery ON.");
  }
  else if (cmd == "AUTO_RECOVERY_OFF")
  {
    autoRecovery = false;
    Serial.println("Auto-recovery OFF.");
  }
  else
    Serial.println("Unknown command: " + cmd);
}

void handleGasLevel(int ppm)
{
  previousState = currentState;

  if (ppm >= THRESHOLD_DANGER)
    currentState = DANGER_LEVEL;
```

```
  else if (ppm >= THRESHOLD_WARNING)
    currentState = WARNING_LEVEL;
  else
    currentState = SAFE;

  switch (currentState)
  {
  case DANGER_LEVEL:
    digitalWrite(LED_RED_PIN, HIGH);
    digitalWrite(LED_GREEN_PIN, LOW);
    if (!buzzerSilenced)
      tone(BUZZER_PIN, 2000);
    setValve(false, "DANGER_AUTO");
    setFan(180, "DANGER_AUTO", ppm);
    if (previousState != DANGER_LEVEL)
      publishAlert(ppm);
    break;

  case WARNING_LEVEL:
    setFan(90, "WARNING_AUTO", ppm);
    break;

  case SAFE:
    digitalWrite(LED_RED_PIN, LOW);
    digitalWrite(LED_GREEN_PIN, HIGH);
    if (!buzzerSilenced)
      noTone(BUZZER_PIN);
    if (autoRecovery)
      setValve(true, "AUTO_RECOVERY");
    setFan(0, "SAFE", ppm);
    break;
  }

  if (previousState != currentState)
    publishStatus(ppm);
}

void setValve(bool open, const char *reason)
{
  valveOpen = open;
  digitalWrite(RELAY_PIN, open ? LOW : HIGH);
  publishValve(reason);
  Serial.print("Valve → ");
  Serial.println(open ? "OPEN" : "CLOSED");
}

void setFan(int angle, const char *reason, int ppm)
{
```

```
  servoAngle = angle;
  ventServo.write(angle);
  publishFan(reason, ppm);
  Serial.print("Fan/Servo → ");
  Serial.print(angle);
  Serial.println("°");
}

void systemReset()
{
  currentState = SAFE;
  buzzerSilenced = false;
  testMode = false;
  blinkState = false;
  noTone(BUZZER_PIN);
  digitalWrite(LED_RED_PIN, LOW);
  digitalWrite(LED_GREEN_PIN, HIGH);
  setValve(true, "SYSTEM_RESET");
  setFan(0, "SYSTEM_RESET", 0);
  Serial.println("System reset complete.");
}

void systemStatus(int ppm)
{
  publishLevel(ppm);
  publishStatus(ppm);
  publishValve("STATUS_REQUEST");
  publishFan("STATUS_REQUEST", ppm);
  publishHealth();
}

void updateOLED(int ppm)
{
  const char *stateStr =
      currentState == DANGER_LEVEL ? "!! DANGER !!" : currentState ==
 ↪  WARNING_LEVEL ? "  WARNING   "





  display.clearDisplay();

  display.fillRect(0, 0, 128, 10, SSD1306_WHITE);
  display.setTextColor(SSD1306_BLACK);
  display.setTextSize(1);
  display.setCursor(10, 1);
```

```
  display.print("GAS DETECTOR");

  display.setTextColor(SSD1306_WHITE);
  display.setTextSize(2);
  display.setCursor(10, 20);
  display.print(stateStr);

  display.setTextSize(1);
  display.setCursor(0, 45);
  display.print("Valve:");
  display.print(valveOpen ? "OPEN " : "SHUT ");
  display.print("Fan:");
  display.print(servoAngle == 0 ? "OFF" : servoAngle == 90 ? "MID"
                                                            : "MAX");

  display.setCursor(0, 56);
  display.print("AutoRec:");
  display.print(autoRecovery ? "ON " : "OFF");
  display.print(" Mute:");
  display.print(buzzerSilenced ? "Y" : "N");

  display.display();
}

void publishLevel(int ppm)
{
  StaticJsonDocument<128> doc;
  doc["ppm"] = ppm;
  doc["unit"] = "ppm";
  doc["uptime_s"] = millis() / 1000;
  char buf[128];
  serializeJson(doc, buf);
  mqtt.publish(TOPIC_LEVEL, buf);
}

void publishStatus(int ppm)
{
  const char *s = currentState == DANGER_LEVEL ? "DANGER" : currentState
  ↪  == WARNING_LEVEL ? "WARNING"



  StaticJsonDocument<128> doc;
  doc["status"] = s;
  doc["ppm"] = ppm;
  char buf[128];
  serializeJson(doc, buf);
  mqtt.publish(TOPIC_STATUS, buf);
```

```
}

void publishAlert(int ppm)
{
  StaticJsonDocument<256> doc;
  doc["alert"] = true;
  doc["level"] = "DANGER";
  doc["ppm"] = ppm;
  doc["message"] = "Gas leakage detected! Evacuate immediately.";
  char buf[256];
  serializeJson(doc, buf);
  mqtt.publish(TOPIC_ALERT, buf, true);
}

void publishValve(const char *reason)
{
  StaticJsonDocument<128> doc;
  doc["state"] = valveOpen ? "OPEN" : "CLOSED";
  doc["reason"] = reason;
  char buf[128];
  serializeJson(doc, buf);
  mqtt.publish(TOPIC_VALVE, buf);
}

void publishFan(const char *reason, int ppm)
{
  const char *s = servoAngle == 0 ? "OFF" : servoAngle == 90 ? "PARTIAL"
                                                              : "ON";
  StaticJsonDocument<128> doc;
  doc["state"] = s;
  doc["angle"] = servoAngle;
  doc["reason"] = reason;
  doc["ppm"] = ppm;
  char buf[128];
  serializeJson(doc, buf);
  mqtt.publish(TOPIC_FAN, buf);
}

void publishHealth()
{
  StaticJsonDocument<256> doc;
  doc["device"] = MQTT_CLIENT;
  doc["status"] = "online";
  doc["uptime_s"] = millis() / 1000;
  doc["ip"] = WiFi.localIP().toString();
  doc["valve"] = valveOpen ? "OPEN" : "CLOSED";
  doc["fan_angle"] = servoAngle;
  doc["auto_recovery"] = autoRecovery;
```

```
  doc["buzzer_silenced"] = buzzerSilenced;
  doc["threshold_warning"] = THRESHOLD_WARNING;
  doc["threshold_danger"] = THRESHOLD_DANGER;
  char buf[256];
  serializeJson(doc, buf);
  mqtt.publish(TOPIC_HEALTH, buf);
}
```

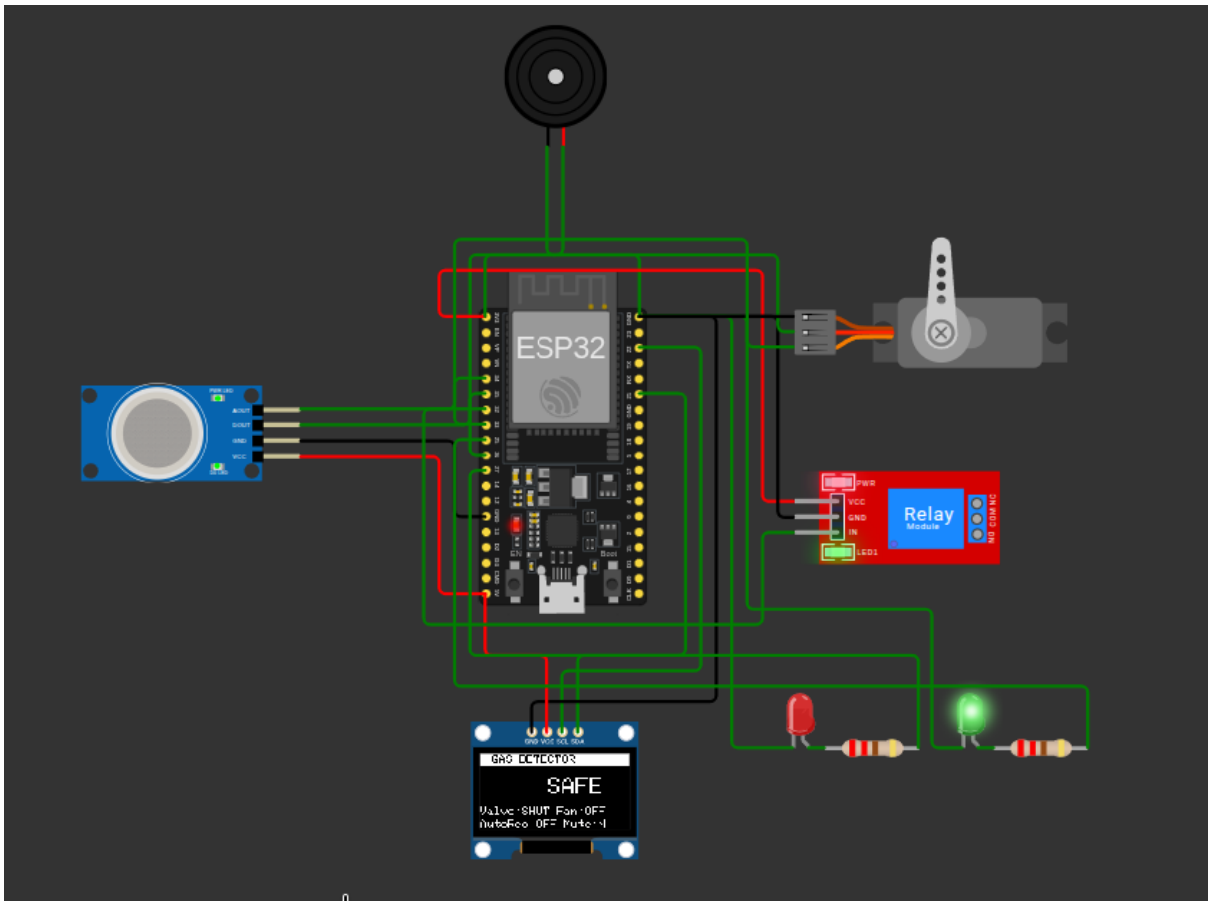
JSON

```
{
  "version": 1,
  "author": "Shree Kottes",
  "editor": "wokwi",
  "parts": [
    { "type": "board-esp32-devkit-c-v4", "id": "esp", "top": 28.8,
     ↪  "left": 100.84, "attrs": {} },
    { "type": "wokwi-gas-sensor", "id": "gas1", "top": 98.7, "left":
     ↪  -146.6, "attrs": {} },
    { "type": "wokwi-servo", "id": "servo1", "top": 7.6, "left": 297.6,
     ↪  "attrs": {} },
    {
      "type": "board-ssd1306",
      "id": "oled1",
      "top": 310.34,
      "left": 96.23,
      "attrs": { "i2cAddress": "0x3c" }
    },
    {
      "type": "wokwi-buzzer",
      "id": "bz1",
      "top": -132,
      "left": 117,
      "attrs": { "volume": "1.0" }
    },
    {
      "type": "wokwi-led",
      "id": "led1",
      "top": 284.4,
      "left": 282.2,
      "attrs": { "color": "red" }
    },
    {
      "type": "wokwi-led",
      "id": "led2",
      "top": 284.4,
      "left": 387.8,
```

```
      "attrs": { "color": "green" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r1",
      "top": 320.75,
      "left": 316.8,
      "attrs": { "value": "220" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r2",
      "top": 320.75,
      "left": 422.4,
      "attrs": { "value": "220" }
    },
    { "type": "wokwi-relay-module", "id": "relay1", "top": 153.8, "left":
     ↪  297.6, "attrs": {} }
  ],
  "connections": [
    [ "esp:TX", "$serialMonitor:RX", "", [] ],
    [ "esp:RX", "$serialMonitor:TX", "", [] ],
    [ "led1:A", "r1:1", "green", [ "v0" ] ],
    [ "led2:A", "r2:1", "green", [ "v0" ] ],
    [ "gas1:AOUT", "esp:34", "green", [ "h96", "v-19.2" ] ],
    [ "gas1:DOUT", "esp:35", "green", [ "h105.6", "v-19.5" ] ],
    [ "gas1:GND", "esp:GND.1", "black", [ "h96", "v47.2" ] ],
    [ "gas1:VCC", "esp:5V", "red", [ "h86.4", "v85.5" ] ],
    [ "oled1:SCL", "esp:22", "green", [ "v-38.4", "h86.7", "v-182.4" ] ],
    [ "oled1:SDA", "esp:21", "green", [ "v-48", "h67.27", "v-144" ] ],
    [ "oled1:GND", "esp:GND.2", "black", [ "v-19.2", "h115.2", "v-220.8"
     ↪  ] ],
    [ "oled1:VCC", "esp:5V", "red", [ "v-48", "h-38.25" ] ],
    [ "bz1:1", "esp:GND.2", "green", [ "v67.2", "h57.6" ] ],
    [ "bz1:2", "esp:26", "green", [ "v67.2", "h-58", "v124.8" ] ],
    [ "led1:C", "esp:GND.2", "green", [ "v0", "h-38", "v-268.8" ] ],
    [ "led2:C", "esp:GND.2", "green", [ "h-18.8", "v-86.4", "h-115.2",
     ↪  "v-182.4" ] ],
    [ "r1:2", "esp:27", "green", [ "v-57.6", "h-279.6", "v-115.2" ] ],
    [ "esp:25", "r2:2", "green", [ "h-19.05", "v153.6", "h394.8" ] ],
    [ "relay1:IN", "esp:32", "green", [ "h-19.2", "v57.4", "h-211.2",
     ↪  "v-144" ] ],
    [ "relay1:VCC", "esp:3V3", "red", [ "h-19.2", "v-144", "h-201.6",
     ↪  "v-19.2" ] ],
    [ "relay1:GND", "esp:GND.2", "black", [ "h-9.6", "v-125.2" ] ],
    [ "servo1:V+", "esp:3V3", "green", [ "h-9.6", "v-47.9", "h-211.2" ]
     ↪  ],
    [ "servo1:GND", "esp:GND.2", "black", [ "h0" ] ],
```

```
    [ "servo1:PWM", "esp:33", "green", [ "h-28.8", "v-67", "h-182.4",
     ↪  "v115.2" ] ]
  ],
  "dependencies": {}
}
```

PIN CONFIGURATION



EX.NO: 13	BANK LOCKER ACCESS SYSTEM
20/03/2026	

AIM

To simulate Bank Locker Access System and send the SMS notification to the Locker-Hirer and give the alarm for detecting the unauthorized person using MQTT Broker.

APPARATUS REQUIRED

- Espressif ESP32 Dev Module
- Breadboard
- 4x LED
- 2x Servo Motor
- Keypad
- LCD I2C Display
- Buzzer

PRE-CONDITIONS

POST-CONDITIONS

CODE

```
/*
 * BANK LOCKER ACCESS SYSTEM - DUAL LOCKER
 * Two-Phase Auth + Dynamic OTP (MQTT) | Wokwi / ESP32
 *
 * PHASE 1 → Authority Master PIN (4 digits, shared)
 * PHASE 2 → Hirer PIN (4 dig) + MQTT OTP (2 dig) = 6 digits
 * OTP rotates every 60s → published to bank/locker/XX/otp
 *
 * Keypad:
 *   [1][2] - select locker
 *   [*]    - backspace
 *   [#]    - system reset
 *   [D]    - manual lock
 */

#include <WiFi.h>
#include <PubSubClient.h>
#include <Keypad.h>
#include <LiquidCrystal_I2C.h>
#include <ESP32Servo.h>

const char* WIFI_SSID  = "Wokwi-GUEST";
```

```
const char* WIFI_PASS  = "";
const char* MQTT_HOST  = "100.64.0.1";
const int   MQTT_PORT  = 1883;
const char* MQTT_CID   = "BankLocker_ESP32_A1";

const String MASTER_PIN   = "1234";
const String HIRER_PIN[3] = {"", "5678", "9012"};

const uint32_t OTP_INTERVAL    = 60000UL;   // 60s
const uint32_t UNLOCK_DURATION = 300000UL;  // 5 mins
const uint32_t ALARM_HOLD      = 10000UL;   // 10s
const uint32_t LCD_REFRESH     = 1000UL;

// GPIO
const byte ROWS = 4, COLS = 4;
byte rowPins[ROWS] = {13, 12, 14, 27};
byte colPins[COLS] = {26, 25, 33, 32};
char keys[ROWS][COLS] = {
  {'1','2','3','A'},
  {'4','5','6','B'},
  {'7','8','9','C'},
  {'*','0','#','D'}
};

const uint8_t SERVO_PIN[3] = {0, 18,  5};
const uint8_t GREEN_PIN[3] = {0,  2,  4};
const uint8_t RED_PIN[3]   = {0, 15, 16};
const uint8_t BUZZER_PIN   = 19;

// State
enum State { SEL_LOCKER, PHASE1, PHASE2, UNLOCKED, ALARM };
State   sysState  = SEL_LOCKER;
int     activeLk  = 0;
int     failCnt   = 0;
String  inputBuf  = "";

String   liveOTP[3]  = {"", "", ""};
uint32_t lastOtpMs   = 0;
uint32_t unlockMs    = 0;
uint32_t alarmMs     = 0;
uint32_t lcdMs       = 0;

// Objects
WiFiClient        wifiCli;
PubSubClient      mqtt(wifiCli);
Keypad            pad   = Keypad(makeKeymap(keys), rowPins, colPins,
 ↪  ROWS, COLS);
LiquidCrystal_I2C lcd(0x27, 16, 2);
```

```
Servo             srv[3];

// LCD
void lcdRow(uint8_t row, const char* text) {
  char buf[17];
  memset(buf, ' ', 16); buf[16] = '\0';
  strncpy(buf, text, strlen(text) < 16 ? strlen(text) : 16);
  lcd.setCursor(0, row);
  lcd.print(buf);
}

void lcdShow(const char* l1, const char* l2 = "") {
  lcd.clear();
  lcd.setCursor(0, 0); lcd.print(l1);
  lcd.setCursor(0, 1); lcd.print(l2);
}

// MQTT
void pub(const char* topic, String msg) {
  if (mqtt.connected()) {
    mqtt.publish(topic, msg.c_str(), true);
  }
  Serial.printf("[MQTT] %-38s  %s\n", topic, msg.c_str());
}

void mkTopic(char* buf, uint8_t sz, int locker, const char* evt) {
  snprintf(buf, sz, "bank/locker/%02d/%s", locker, evt);
}

// Network
void wifiConnect() {
  lcdShow("Connecting WiFi", "Please wait...");
  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASS);
  Serial.print("WiFi");
  uint8_t tries = 0;
  while (WiFi.status() != WL_CONNECTED && tries++ < 40) {
    delay(400); Serial.print('.');
  }
  if (WiFi.status() == WL_CONNECTED) {
    Serial.println(" OK  " + WiFi.localIP().toString());
    lcdShow("WiFi Connected!", WiFi.localIP().toString().c_str());
    delay(900);
  } else {
    lcdShow("WiFi FAILED", "Check SSID/Pass");
    delay(2000);
  }
}
```

```
void mqttConnect() {
  while (!mqtt.connected()) {
    Serial.print("MQTT connect... ");
    if (sysState == SEL_LOCKER) {
      lcdShow("MQTT Connecting", "...");
    }
    if (mqtt.connect(MQTT_CID)) {
      Serial.println("OK");
      pub("bank/locker/system", "BankLockerSystem ONLINE | ESP32 ready");
      if (sysState == SEL_LOCKER) {
        lcdShow("Select Locker:", "  Press [1] [2] ");
      }
    } else {
      Serial.printf("fail (rc=%d) retrying\n", mqtt.state());
      delay(3000);
    }
  }
}

// OTP
void rotateOTPs() {
  char topic[40];
  for (int l = 1; l <= 2; l++) {
    liveOTP[l] = String(random(10, 100));
    mkTopic(topic, sizeof(topic), l, "otp");
    pub(topic,
        "SMS to Hirer-L#" + String(l) +
        ": Your OTP = [" + liveOTP[l] +
        "] | Append to your 4-digit PIN | Valid 60s");
  }
  lastOtpMs = millis();
  Serial.printf(">>> OTPs rotated  L1=%s  L2=%s\n",
                liveOTP[1].c_str(), liveOTP[2].c_str());
}

// Locker control
void openLocker(int l) {
  srv[l].write(90);
  digitalWrite(GREEN_PIN[l], HIGH);
  digitalWrite(RED_PIN[l],   LOW);

  char topic[40], line1[17];
  snprintf(line1, sizeof(line1), "Locker #%d  OPEN!", l);
  lcdShow(line1, "[D]Lock 5m-auto");

  mkTopic(topic, sizeof(topic), l, "sms");
  pub(topic, "SMS: Locker #" + String(l) + " OPENED successfully.
  ↪ Auto-lock in 5 mins.");
```

```
  mkTopic(topic, sizeof(topic), l, "status");
  pub(topic, "UNLOCKED");

  sysState = UNLOCKED;
  unlockMs = millis();
}

void closeLocker(int l) {
  srv[l].write(0);
  digitalWrite(GREEN_PIN[l], LOW);

  char topic[40];
  mkTopic(topic, sizeof(topic), l, "status");
  pub(topic, "SECURED");
  mkTopic(topic, sizeof(topic), l, "sms");
  pub(topic, "SMS: Locker #" + String(l) + " has been SECURED.");
}

void triggerAlarm(int l, const char* reason) {
  sysState = ALARM;
  alarmMs  = millis();

  digitalWrite(RED_PIN[l], HIGH);

  char line1[17], topic[40];
  snprintf(line1, sizeof(line1), " !! ALARM  L#%d !!", l);
  lcdShow(line1, reason);

  mkTopic(topic, sizeof(topic), l, "alarm");
  pub(topic, "ALERT on Locker #" + String(l) + ": " + String(reason));

  tone(BUZZER_PIN, 2000);
}

void resetSystem() {
  for (int l = 1; l <= 2; l++) {
    srv[l].write(0);
    digitalWrite(GREEN_PIN[l], LOW);
    digitalWrite(RED_PIN[l],   LOW);
  }
  noTone(BUZZER_PIN);
  inputBuf  = "";
  failCnt   = 0;
  activeLk  = 0;
  sysState  = SEL_LOCKER;
  pub("bank/locker/system", "System RESET by operator");
  lcdShow("System Reset OK", "Select: [1]  [2]");
}
```

```
// Input handling
String maskStr(const String& s) {
  String m = "";
  for (size_t i = 0; i < s.length(); i++) m += '*';
  return m;
}

void handleKey(char key) {
  if (key == '#') { resetSystem(); return; }

  if (key == 'D' && sysState == UNLOCKED) {
    char topic[40];
    mkTopic(topic, sizeof(topic), activeLk, "sms");
    pub(topic, "SMS: Locker #" + String(activeLk) + " MANUALLY LOCKED by
 ↪  user.");
    closeLocker(activeLk);
    sysState = SEL_LOCKER;
    lcdShow("Manual Lock OK", "Select: [1] [2]");
    return;
  }

  if (key == '*') {
    if ((sysState == PHASE1 || sysState == PHASE2) && inputBuf.length()
     ↪  > 0) {
      inputBuf.remove(inputBuf.length() - 1);
      lcdRow(1, maskStr(inputBuf).c_str());
    }
    return;
  }

  switch (sysState) {
    case SEL_LOCKER:
      if (key == '1' || key == '2') {
        activeLk = key - '0';
        failCnt  = 0;
        inputBuf = "";
        sysState = PHASE1;
        char hdr[17];
        snprintf(hdr, sizeof(hdr), "L#%d | Phase-1:", activeLk);
        lcdShow(hdr, "Authority PIN: _");
      }
      break;

    case PHASE1:
      if (inputBuf.length() < 4) {
        inputBuf += key;
        lcdRow(1, maskStr(inputBuf).c_str());
```

```
    }
    if (inputBuf.length() == 4) {
      delay(150);
      if (inputBuf == MASTER_PIN) {
        char topic[40], hdr[17];
        mkTopic(topic, sizeof(topic), activeLk, "auth");
        pub(topic, "Phase-1 PASSED - Authority verified for Locker #" +
↪ String(activeLk));
        failCnt  = 0;
        inputBuf = "";
        sysState = PHASE2;
        snprintf(hdr, sizeof(hdr), "L#%d | Phase-2:", activeLk);
        lcdShow(hdr, "PIN+OTP (6 dig):");
        lcdMs = millis();
      } else {
        failCnt++;
        inputBuf = "";
        if (failCnt >= 3) {
          triggerAlarm(activeLk, "3x Auth1 fails");
        } else {
          char buf[17];
          snprintf(buf, sizeof(buf), "Wrong! (%d left) ", 3 -
↪ failCnt);
          lcdShow(buf, "Authority PIN: _");
        }
      }
    }
    break;

  case PHASE2:
    if (inputBuf.length() < 6) {
      inputBuf += key;
      lcdRow(1, maskStr(inputBuf).c_str());
    }
    if (inputBuf.length() == 6) {
      delay(150);
      String expected = HIRER_PIN[activeLk] + liveOTP[activeLk];
      if (inputBuf == expected) {
        openLocker(activeLk);
      } else {
        failCnt++;
        inputBuf = "";
        if (failCnt >= 3) {
          triggerAlarm(activeLk, "3x Auth2 fails");
        } else {
          char buf[17];
          snprintf(buf, sizeof(buf), "Wrong! (%d left) ", 3 -
↪ failCnt);
```

```
            lcdShow(buf, "PIN+OTP (6 dig):");
          }
        }
      }
      break;

    default: break;
  }
}

void setup() {
  Serial.begin(115200);
  delay(300);
  Serial.println("\n\n=== BANK LOCKER SYSTEM BOOT ===");

  // GPIO setup
  for (int l = 1; l <= 2; l++) {
    pinMode(GREEN_PIN[l], OUTPUT);
    pinMode(RED_PIN[l], OUTPUT);
    digitalWrite(GREEN_PIN[l], LOW);
    digitalWrite(RED_PIN[l], LOW);
  }

  noTone(BUZZER_PIN);

  // Servos
  srv[1].attach(SERVO_PIN[1]); srv[1].write(0);
  srv[2].attach(SERVO_PIN[2]); srv[2].write(0);

  lcd.init();
  lcd.backlight();
  lcdShow(" BANK LOCKER SYS", "  Initializing..");
  delay(800);

  wifiConnect();
  mqtt.setServer(MQTT_HOST, MQTT_PORT);
  mqttConnect();

  randomSeed(analogRead(34) ^ millis());
  rotateOTPs();

  lcdShow("Select Locker:", "  Press [1] [2] ");
  sysState = SEL_LOCKER;
  Serial.println("=== SYSTEM READY ===\n");
}

void loop() {
  if (!mqtt.connected()) mqttConnect();
```

```
  mqtt.loop();

  uint32_t now = millis();

  if (now - lastOtpMs >= OTP_INTERVAL) {
    rotateOTPs();
    if (sysState == PHASE2) {
      inputBuf = "";
      lcdRow(0, "OTP expired!Retry");
      lcdRow(1, "PIN+OTP (6 dig):");
      delay(1200);
      char hdr[17];
      snprintf(hdr, sizeof(hdr), "L#%d | Phase-2:", activeLk);
      lcdRow(0, hdr);
      lcdRow(1, "PIN+OTP (6 dig):");
    }
  }

  if (sysState == UNLOCKED && now - unlockMs >= UNLOCK_DURATION) {
    closeLocker(activeLk);
    sysState = SEL_LOCKER;
    lcdShow("Locker Secured.", "Select: [1] [2]");
  }

  if (sysState == ALARM && now - alarmMs >= ALARM_HOLD) {
    resetSystem();
  }

  if (sysState == PHASE2 && now - lcdMs >= LCD_REFRESH) {
    lcdMs = now;
    uint32_t elapsed   = (now - lastOtpMs) / 1000;
    uint32_t remaining = (elapsed < 60) ? (60 - elapsed) : 0;
    char buf[17];
    snprintf(buf, sizeof(buf), "L#%d Ph2 exp:%2lus", activeLk, (unsigned
↪   long)remaining);
    lcdRow(0, buf);
  }

  char key = pad.getKey();
  if (key) handleKey(key);
}
```

JSON

```
{
  "version": 1,
  "author": "Shree Kottes",
```

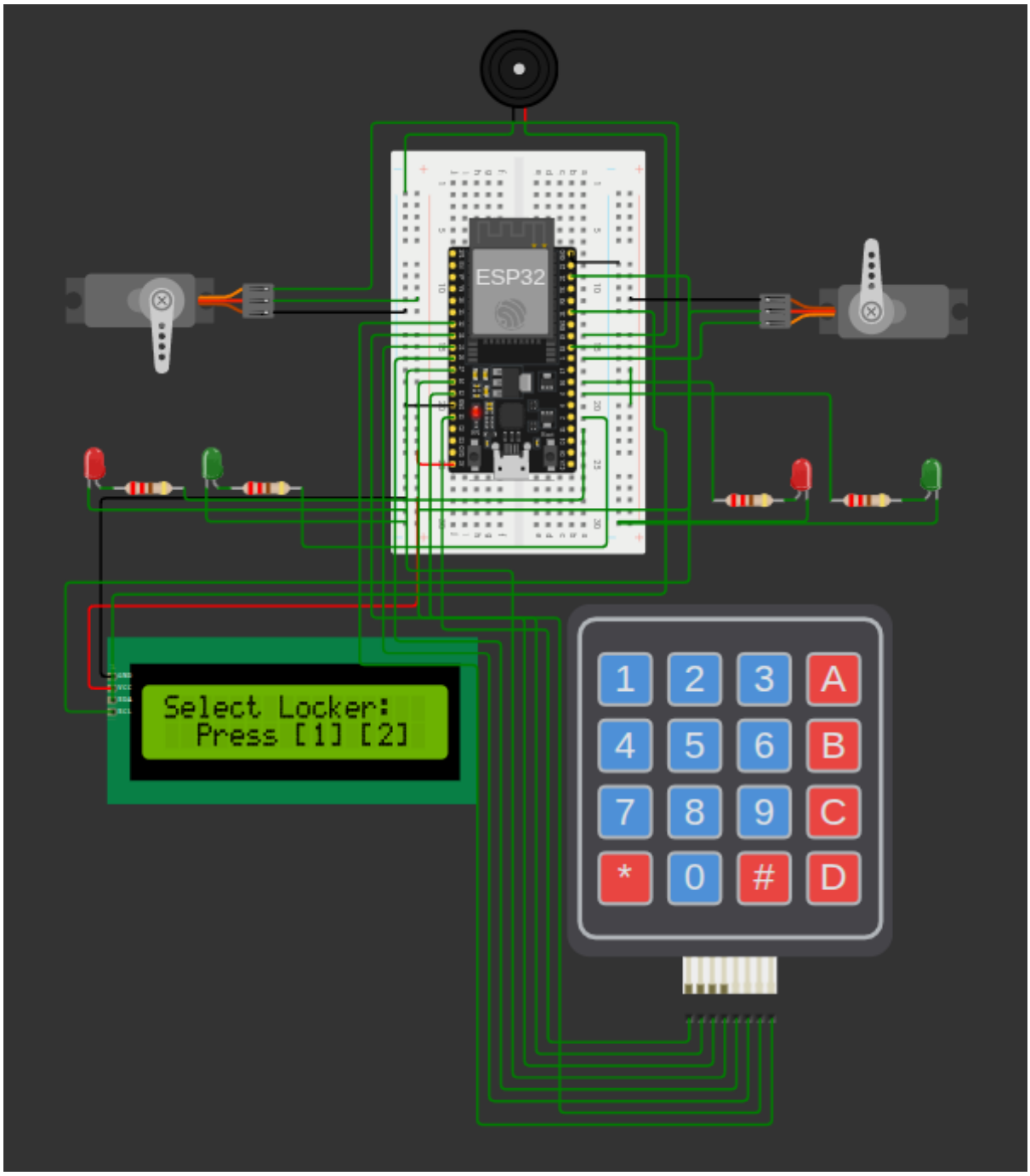
```
"editor": "wokwi",
"parts": [
  {
    "type": "wokwi-breadboard-half",
    "id": "bb1",
    "top": -90.3,
    "left": 348.9,
    "rotate": 90,
    "attrs": {}
  },
  { "type": "board-esp32-devkit-c-v4", "id": "esp", "top": -96, "left":
   ↪  456.04, "attrs": {} },
  { "type": "wokwi-membrane-keypad", "id": "keypad1", "top": 218.8,
   ↪  "left": 552.8, "attrs": {} },
  {
    "type": "wokwi-lcd1602",
    "id": "lcd1",
    "top": 246.4,
    "left": 178.4,
    "attrs": { "pins": "i2c" }
  },
  {
    "type": "wokwi-servo",
    "id": "servo1",
    "top": -88.4,
    "left": 144,
    "rotate": 180,
    "attrs": {}
  },
  {
    "type": "wokwi-buzzer",
    "id": "bz1",
    "top": -256.8,
    "left": 481.8,
    "attrs": { "volume": "0.5" }
  },
  { "type": "wokwi-led", "id": "led1", "top": 82.8, "left": 147.8,
   ↪  "attrs": { "color": "red" } },
  {
    "type": "wokwi-led",
    "id": "led2",
    "top": 82.8,
    "left": 243.8,
    "attrs": { "color": "green" }
  },
  {
    "type": "wokwi-resistor",
    "id": "r1",
```

```
      "top": 119.15,
      "left": 182.4,
      "attrs": { "value": "220" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r2",
      "top": 119.15,
      "left": 278.4,
      "attrs": { "value": "220" }
    },
    { "type": "wokwi-servo", "id": "servo2", "top": -78.8, "left": 710.4,
     ↪  "attrs": {} },
    {
      "type": "wokwi-led",
      "id": "led3",
      "top": 92.4,
      "left": 724.2,
      "attrs": { "color": "red", "flip": "1" }
    },
    {
      "type": "wokwi-led",
      "id": "led4",
      "top": 92.4,
      "left": 829.8,
      "attrs": { "color": "green", "flip": "1" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r3",
      "top": 128.75,
      "left": 672,
      "attrs": { "value": "220" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r4",
      "top": 128.75,
      "left": 768,
      "attrs": { "value": "220" }
    }
  ],
  "connections": [
    [ "esp:TX", "$serialMonitor:RX", "", [] ],
    [ "esp:RX", "$serialMonitor:TX", "", [] ],
    [ "led1:A", "r1:1", "green", [ "v0" ] ],
    [ "r2:1", "led2:A", "green", [ "v0" ] ],
    [ "led3:A", "r3:2", "green", [ "v0" ] ],
```

```
    [ "r4:2", "led4:A", "green", [ "v0" ] ],
    [ "esp:5V", "bb1:bp.20", "red", [ "h0" ] ],
    [ "esp:GND.2", "bb1:tn.6", "black", [ "v0" ] ],
    [ "bz1:2", "bb1:14t.a", "green", [ "v9.6", "h114.8", "v163.2" ] ],
    [ "esp:GND.1", "bb1:bn.16", "black", [ "h0" ] ],
    [ "bz1:1", "bb1:bn.1", "green", [ "v9.6", "h-86.4" ] ],
    [ "servo2:PWM", "bb1:16t.a", "green", [ "h-48", "v29" ] ],
    [ "servo1:PWM", "esp:18", "green", [ "h79.6", "v-135.4", "h249.6",
     ↪  "v182.4" ] ],
    [ "servo1:V+", "bb1:bp.9", "green", [ "h0" ] ],
    [ "servo1:GND", "bb1:bn.10", "black", [ "h0" ] ],
    [ "servo2:GND", "bb1:tp.9", "black", [ "h0" ] ],
    [ "servo2:V+", "bb1:bp.24", "green", [ "h-57.6", "v182.5" ] ],
    [ "lcd1:GND", "bb1:bn.23", "black", [ "h-9.6", "v-126.4" ] ],
    [ "lcd1:VCC", "bb1:bp.23", "red", [ "h-19.2", "v-67.1", "h67.2" ] ],
    [ "esp:21", "lcd1:SDA", "green", [ "h67.2", "v96", "h9.6", "v134.4",
     ↪  "h-451.2" ] ],
    [ "lcd1:SCL", "esp:22", "green", [ "h-38.4", "v-105.3", "h508.8",
     ↪  "v-249.6" ] ],
    [ "esp:13", "keypad1:R1", "green", [ "h-9.45", "v172.8", "h86.4",
     ↪  "v336", "h115.2" ] ],
    [
      "esp:12",
      "keypad1:R2",
      "green",
      [ "v0", "h-19.05", "v182.4", "h86.4", "v355.2", "h134.4" ]
    ],
    [ "esp:14", "keypad1:R3", "green", [ "h-28.65", "v192", "h86.4",
     ↪  "v364.8", "h153.6" ] ],
    [ "esp:27", "keypad1:R4", "green", [ "h-38.25", "v163.2", "h86.4",
     ↪  "v412.8", "h172.8" ] ],
    [ "esp:26", "keypad1:C1", "green", [ "h-47.85", "v230.4", "h86.4",
     ↪  "v364.8", "h192" ] ],
    [
      "esp:25",
      "keypad1:C2",
      "green",
      [ "v0", "h-57.45", "v249.6", "h86.4", "v364.8", "h211.2" ]
    ],
    [ "esp:33", "keypad1:C3", "green", [ "h-67.05", "v230.4", "h153.6",
     ↪  "v403.2", "h163.2" ] ],
    [ "esp:32", "keypad1:C4", "green", [ "h-76.65", "v278.4", "h96",
     ↪  "v374.4", "h240" ] ],
    [ "r4:1", "bb1:19t.a", "green", [ "v-86.4" ] ],
    [ "r3:1", "bb1:18t.a", "green", [ "v0" ] ],
    [ "led3:C", "bb1:tn.25", "green", [ "v0" ] ],
    [ "led4:C", "bb1:tn.25", "green", [ "v19.2", "h-259.6" ] ],
    [ "r2:2", "bb1:21t.a", "green", [ "v48", "h248.4", "v-105.6",
     ↪  "h-19.2" ] ],
```

```
    [ "led2:C", "bb1:bn.25", "green", [ "v0" ] ],
    [ "r1:2", "bb1:22t.a", "green", [ "v9.6", "h334.8" ] ],
    [ "led1:C", "bb1:bn.24", "green", [ "v0" ] ],
    [ "esp:0", "bb1:20t.b", "", [ "$bb" ] ],
    [ "esp:2", "bb1:21t.b", "", [ "$bb" ] ],
    [ "esp:4", "bb1:19t.b", "", [ "$bb" ] ],
    [ "esp:5", "bb1:16t.b", "", [ "$bb" ] ],
    [ "esp:12", "bb1:19b.j", "", [ "$bb" ] ],
    [ "esp:13", "bb1:21b.j", "", [ "$bb" ] ],
    [ "esp:14", "bb1:18b.j", "", [ "$bb" ] ],
    [ "esp:15", "bb1:22t.b", "", [ "$bb" ] ],
    [ "esp:16", "bb1:18t.b", "", [ "$bb" ] ],
    [ "esp:17", "bb1:17t.b", "", [ "$bb" ] ],
    [ "esp:18", "bb1:15t.b", "", [ "$bb" ] ],
    [ "esp:19", "bb1:14t.b", "", [ "$bb" ] ],
    [ "esp:21", "bb1:12t.b", "", [ "$bb" ] ],
    [ "esp:22", "bb1:9t.b", "", [ "$bb" ] ],
    [ "esp:23", "bb1:8t.b", "", [ "$bb" ] ],
    [ "esp:25", "bb1:15b.j", "", [ "$bb" ] ],
    [ "esp:26", "bb1:16b.j", "", [ "$bb" ] ],
    [ "esp:27", "bb1:17b.j", "", [ "$bb" ] ],
    [ "esp:32", "bb1:13b.j", "", [ "$bb" ] ],
    [ "esp:33", "bb1:14b.j", "", [ "$bb" ] ],
    [ "esp:34", "bb1:11b.j", "", [ "$bb" ] ],
    [ "esp:35", "bb1:12b.j", "", [ "$bb" ] ],
    [ "esp:3V3", "bb1:7b.j", "", [ "$bb" ] ],
    [ "esp:EN", "bb1:8b.j", "", [ "$bb" ] ],
    [ "esp:VP", "bb1:9b.j", "", [ "$bb" ] ],
    [ "esp:VN", "bb1:10b.j", "", [ "$bb" ] ],
    [ "esp:GND.1", "bb1:20b.j", "", [ "$bb" ] ],
    [ "esp:D2", "bb1:22b.j", "", [ "$bb" ] ],
    [ "esp:D3", "bb1:23b.j", "", [ "$bb" ] ],
    [ "esp:CMD", "bb1:24b.j", "", [ "$bb" ] ],
    [ "esp:5V", "bb1:25b.j", "", [ "$bb" ] ],
    [ "esp:GND.2", "bb1:7t.b", "", [ "$bb" ] ],
    [ "esp:TX", "bb1:10t.b", "", [ "$bb" ] ],
    [ "esp:RX", "bb1:11t.b", "", [ "$bb" ] ],
    [ "esp:GND.3", "bb1:13t.b", "", [ "$bb" ] ],
    [ "esp:D1", "bb1:23t.b", "", [ "$bb" ] ],
    [ "esp:D0", "bb1:24t.b", "", [ "$bb" ] ],
    [ "esp:CLK", "bb1:25t.b", "", [ "$bb" ] ],
    [ "bb1:tp.14", "bb1:tp.16", "green", [ "h0" ] ]
  ],
  "dependencies": {}
}
```

PIN CONFIGURATION



EX.NO: 14	TIC-TAC-TOE
20/03/2026	

AIM

To Develop Tic Tac Toe Game using MQTT Broker.

APPARATUS REQUIRED

- ESP 32 Dev Module

PRE-CONDITIONS

POST-CONDITIONS

CODE

```
/*
 *
 *  Tic-Tac-Toe MQTT Game Server - ESP32 (Wokwi)
 *
 *  Role  : Authoritative game server + AI engine
 *  Broker: 100.64.0.1:1883  (Tailscale VPC, via Wokwi Gateway)
 *
 *  Topic schema
 *
 *  SUB  ttt/game/new               Client creates a game session
 *  SUB  ttt/game/+/move            Client sends a move
 *  SUB  ttt/lobby/announce         Client announces presence
 *  SUB  ttt/lobby/invite/#         Client invites another player
 *  SUB  ttt/lobby/response/#       Client accepts/declines invite
 *
 *  PUB  ttt/game/{pid}/created     Notify client of new game + gid
 *  PUB  ttt/game/{gid}/state       Full board state after every move
 *  PUB  ttt/game/{gid}/result      Game over payload
 *  PUB  ttt/sys/players/online     Live player roster (retained)
 *  PUB  ttt/sys/heartbeat          Alive ping every 5 s
 *  PUB  ttt/sys/server             Server status (retained)
 *
 */

#include <WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <map>
```

```
//   Network
const char* WIFI_SSID     = "Wokwi-GUEST";  // Wokwi virtual WiFi
const char* WIFI_PASSWORD = "";

const char* MQTT_HOST     = "100.64.0.1";
const int   MQTT_PORT     = 1883;
const char* MQTT_CLIENT   = "esp32-ttt-server";

//   Structs
struct GameSession {
  char    board[9];        // ' '=empty  'X'=X  'O'=O
  String  player1;         // pid of creator (always X)
  String  player2;         // pid of opponent, or "AI"
  String  currentTurn;    // pid whose turn it is, "AI" when AI thinking
  bool    aiMode;
  bool    active;
};

struct Player {
  String name;
  String client;  // "web" or "flutter"
};

//   State
std::map<String, GameSession> sessions;
std::map<String, Player>      players;

WiFiClient    wifiClient;
PubSubClient  mqtt(wifiClient);
unsigned long lastHeartbeat = 0;

//
//  Helpers
//

String genId(int len = 4) {
  const char hex[] = "0123456789abcdef";
  String id = "";
  for (int i = 0; i < len; i++) id += hex[random(16)];
  return id;
}

//   Minimax AI

int winLines[8][3] = {
  {0,1,2},{3,4,5},{6,7,8},   // rows
  {0,3,6},{1,4,7},{2,5,8},   // cols
  {0,4,8},{2,4,6}            // diags
```

```
};

int evaluate(char b[9]) {
  for (auto& w : winLines) {
    if (b[w[0]] != ' ' && b[w[0]] == b[w[1]] && b[w[1]] == b[w[2]])
      return b[w[0]] == 'O' ? 10 : -10;
  }
  return 0;
}

bool boardFull(char b[9]) {
  for (int i = 0; i < 9; i++) if (b[i] == ' ') return false;
  return true;
}

int minimax(char b[9], int depth, bool isMax, int alpha, int beta) {
  int score = evaluate(b);
  if (score != 0)     return score - (isMax ? depth : -depth);
  if (boardFull(b))   return 0;

  if (isMax) {
    int best = -100;
    for (int i = 0; i < 9; i++) {
      if (b[i] == ' ') {
        b[i] = 'O';
        best = max(best, minimax(b, depth + 1, false, alpha, beta));
        b[i] = ' ';
        alpha = max(alpha, best);
        if (beta <= alpha) break;
      }
    }
    return best;
  } else {
    int best = 100;
    for (int i = 0; i < 9; i++) {
      if (b[i] == ' ') {
        b[i] = 'X';
        best = min(best, minimax(b, depth + 1, true, alpha, beta));
        b[i] = ' ';
        beta = min(beta, best);
        if (beta <= alpha) break;
      }
    }
    return best;
  }
}

int bestAiMove(char b[9]) {
```

```
  int bestVal = -100, bestCell = -1;
  for (int i = 0; i < 9; i++) {
    if (b[i] == ' ') {
      b[i] = 'O';
      int val = minimax(b, 0, false, -100, 100);
      b[i] = ' ';
      if (val > bestVal) { bestVal = val; bestCell = i; }
    }
  }
  return bestCell;
}

//   Win detection

// Returns  ""          : game ongoing
// Returns  "draw"      : board full, no winner
// Returns  "X:0,4,8"   : X wins, winning cell indices
String checkResult(char b[9]) {
  for (auto& w : winLines) {
    if (b[w[0]] != ' ' && b[w[0]] == b[w[1]] && b[w[1]] == b[w[2]]) {
      char sym = b[w[0]];
      String s = String(sym) + ":" +
                 String(w[0]) + "," +
                 String(w[1]) + "," +
                 String(w[2]);
      return s;
    }
  }
  return boardFull(b) ? "draw" : "";
}

//
//  MQTT Publishers
//

void pubState(const String& gid, GameSession& g) {
  StaticJsonDocument<512> doc;
  JsonArray board = doc.createNestedArray("board");
  for (int i = 0; i < 9; i++)
    board.add(g.board[i] == ' ' ? "" : String(g.board[i]));

  doc["gid"]      = gid;
  doc["turn"]     = g.currentTurn;
  doc["p1"]       = g.player1;
  doc["p2"]       = g.player2;
  doc["ai_mode"]  = g.aiMode;
  doc["status"]   = "ongoing";
```

```
  String payload, topic = "ttt/game/" + gid + "/state";
  serializeJson(doc, payload);
  mqtt.publish(topic.c_str(), payload.c_str(), true);
  Serial.println("[PUB] " + topic);
}

void pubResult(const String& gid, const String& result, GameSession& g) {
  StaticJsonDocument<256> doc;
  String topic = "ttt/game/" + gid + "/result";

  if (result == "draw") {
    doc["winner"]        = "draw";
    doc["winner_symbol"] = "";
    doc["winning_cells"] = "";
  } else {
    String sym   = result.substring(0, 1);
    String cells = result.substring(2);
    String winner = (sym == "X") ? g.player1 : g.player2;
    doc["winner"]        = winner;
    doc["winner_symbol"] = sym;
    doc["winning_cells"] = cells;
  }

  doc["p1"] = g.player1;
  doc["p2"] = g.player2;

  String payload;
  serializeJson(doc, payload);
  mqtt.publish(topic.c_str(), payload.c_str(), true);
  Serial.println("[PUB] " + topic + " result=" + result);

  g.active = false;
}

void pubOnlinePlayers() {
  StaticJsonDocument<1024> doc;
  JsonArray arr = doc.createNestedArray("players");
  for (auto& kv : players) {
    JsonObject o = arr.createNestedObject();
    o["pid"]    = kv.first;
    o["name"]   = kv.second.name;
    o["client"] = kv.second.client;
  }
  String payload;
  serializeJson(doc, payload);
  mqtt.publish("ttt/sys/players/online", payload.c_str(), true);
}
```

```
void pubCreated(const String& pid, const String& gid,
                const String& sym, const String& opponentPid,
                const String& opponentName, bool aiMode) {
  StaticJsonDocument<256> doc;
  doc["gid"]            = gid;
  doc["symbol"]         = sym;
  doc["opponent_pid"]   = opponentPid;
  doc["opponent_name"]  = opponentName;
  doc["ai_mode"]        = aiMode;

  String payload, topic = "ttt/game/" + pid + "/created";
  serializeJson(doc, payload);
  mqtt.publish(topic.c_str(), payload.c_str());
}

//
//  Move handling (shared by human + AI)
//

void applyMoveAndRespond(const String& gid, GameSession& g, int cell,
 ↪  char sym) {
  g.board[cell] = sym;
  String result = checkResult(g.board);

  if (result != "") {
    pubState(gid, g);
    pubResult(gid, result, g);
    return;
  }

  if (g.aiMode) {
    // AI's turn
    g.currentTurn = "AI";
    pubState(gid, g);

    delay(400);  // brief pause so clients see "AI thinking"

    int aiCell = bestAiMove(g.board);
    g.board[aiCell] = 'O';
    g.currentTurn   = g.player1;

    String aiResult = checkResult(g.board);
    pubState(gid, g);
    if (aiResult != "") pubResult(gid, aiResult, g);
  } else {
    g.currentTurn = (g.currentTurn == g.player1) ? g.player2 : g.player1;
    pubState(gid, g);
  }
```

```
}

//
//  MQTT Callback
//

void onMessage(char* topicC, byte* payload, unsigned int len) {
  String topic = String(topicC);
  String msg   = "";
  for (unsigned int i = 0; i < len; i++) msg += (char)payload[i];

  Serial.println("[SUB] " + topic + " | " + msg);

  StaticJsonDocument<512> doc;
  if (deserializeJson(doc, msg) != DeserializationError::Ok) {
    Serial.println("  JSON parse error");
    return;
  }

  //   ttt/game/new
  if (topic == "ttt/game/new") {
    String pid  = doc["pid"].as<String>();
    String mode = doc["mode"].as<String>();   // "ai" | "human"
    String name = doc["name"].as<String>();

    String gid = genId(4);
    GameSession g;
    for (int i = 0; i < 9; i++) g.board[i] = ' ';
    g.player1     = pid;
    g.player2     = (mode == "ai") ? "AI" : "";
    g.currentTurn = pid;
    g.aiMode      = (mode == "ai");
    g.active      = true;
    sessions[gid] = g;

    pubCreated(pid, gid, "X", "AI", "AI", g.aiMode);
    pubState(gid, sessions[gid]);
    Serial.println("  Created session " + gid + " mode=" + mode);
    return;
  }

  //   ttt/game/{gid}/move
  if (topic.startsWith("ttt/game/") && topic.endsWith("/move")) {
    // Extract gid: "ttt/game/XXXX/move" → skip 9, trim 5
    String gid = topic.substring(9, topic.length() - 5);

    if (sessions.find(gid) == sessions.end()) {
      Serial.println("  Unknown session: " + gid); return;
```

```
    }
    GameSession& g = sessions[gid];
    if (!g.active) { Serial.println("  Session inactive"); return; }

    String pid = doc["player"].as<String>();
    int    cell = doc["cell"].as<int>();

    // Validate
    if (pid != g.currentTurn) {
      Serial.println("  Wrong turn. Expected: " + g.currentTurn + " got:
↪   " + pid);
      return;
    }
    if (cell < 0 || cell > 8 || g.board[cell] != ' ') {
      Serial.println("  Invalid cell " + String(cell));
      return;
    }

    char sym = (pid == g.player1) ? 'X' : 'O';
    applyMoveAndRespond(gid, g, cell, sym);
    return;
  }

  //   ttt/lobby/announce
  if (topic == "ttt/lobby/announce") {
    String pid    = doc["pid"].as<String>();
    String name   = doc["name"].as<String>();
    String client = doc["client"].as<String>();
    players[pid]  = { name, client };
    Serial.println("  Player online: " + name + " (" + pid + ")");
    pubOnlinePlayers();
    return;
  }

  //   ttt/lobby/leave
  if (topic.startsWith("ttt/lobby/leave/")) {
    String pid = topic.substring(16);
    players.erase(pid);
    pubOnlinePlayers();
    return;
  }

  //   ttt/lobby/invite/{target_pid}
  // Clients subscribe directly to their own pid topic;
  // ESP32 just validates the sender is a known player.
  if (topic.startsWith("ttt/lobby/invite/")) {
    // The broker delivers this directly to subscribers.
    // ESP32 doesn't need to relay - just log it.
```

```
    String targetPid = topic.substring(17);
    String fromPid   = doc["from_pid"].as<String>();
    Serial.println("  Invite: " + fromPid + " -> " + targetPid);
    return;
  }

  //   ttt/lobby/response/{inviter_pid}
  if (topic.startsWith("ttt/lobby/response/")) {
    String inviterPid    = topic.substring(19);
    bool   accepted      = doc["accepted"].as<bool>();
    String responderPid  = doc["pid"].as<String>();
    String responderName = doc["name"].as<String>();

    if (!accepted) {
      // Tell inviter they were declined
      StaticJsonDocument<128> d;
      d["from_pid"]  = responderPid;
      d["from_name"] = responderName;
      d["accepted"]  = false;
      String dp, dt = "ttt/game/" + inviterPid + "/created";
      serializeJson(d, dp);
      mqtt.publish(dt.c_str(), dp.c_str());
      return;
    }

    // Both accepted → create PvP session
    String gid = genId(4);
    GameSession g;
    for (int i = 0; i < 9; i++) g.board[i] = ' ';
    g.player1     = inviterPid;
    g.player2     = responderPid;
    g.currentTurn = inviterPid;
    g.aiMode      = false;
    g.active      = true;
    sessions[gid] = g;

    // Notify inviter (X)
    String inviterName = players.count(inviterPid) ?
↪   players[inviterPid].name : inviterPid;
    pubCreated(inviterPid, gid, "X", responderPid, responderName, false);
    // Notify responder (O)
    pubCreated(responderPid, gid, "O", inviterPid, inviterName, false);

    pubState(gid, sessions[gid]);
    Serial.println("  PvP session " + gid + ": " + inviterPid + " vs " +
↪   responderPid);
    return;
  }
```

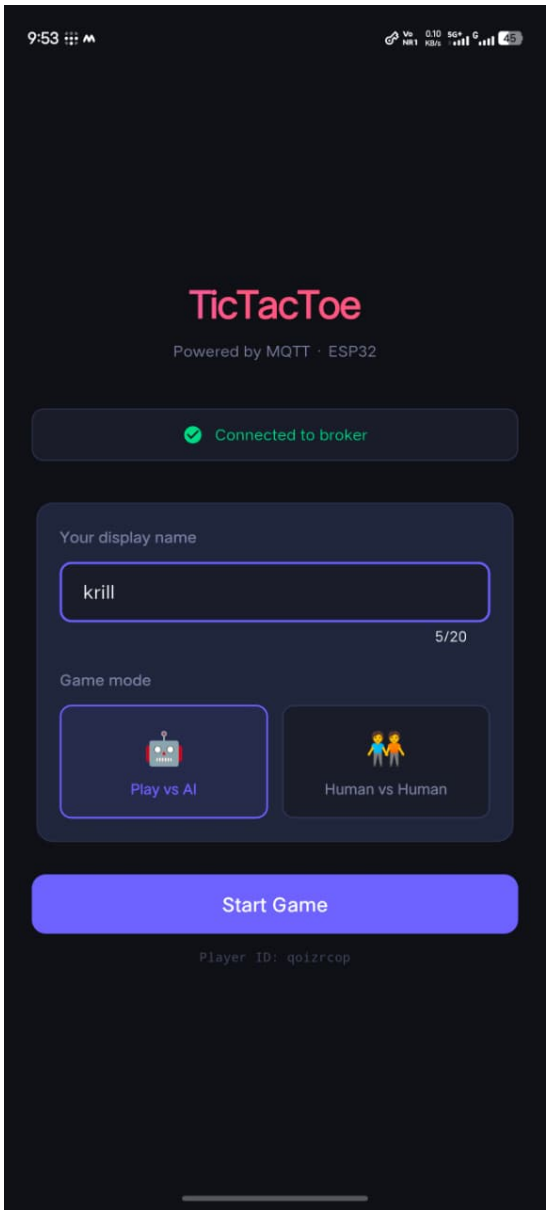
```
}

//
//  Setup helpers
//

void connectWiFi() {
  Serial.print("[WiFi] Connecting to " + String(WIFI_SSID));
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500); Serial.print(".");
  }
  Serial.println("\n[WiFi] Connected: " + WiFi.localIP().toString());
}

void connectMQTT() {
  while (!mqtt.connected()) {
    Serial.print("[MQTT] Connecting to " + String(MQTT_HOST) + "...");
    if (mqtt.connect(MQTT_CLIENT, nullptr, nullptr, "ttt/sys/server",
                     0, true, "{\"status\":\"offline\"}")) {
      Serial.println("OK");

      // Subscriptions
      mqtt.subscribe("ttt/game/new");
      mqtt.subscribe("ttt/game/+/move");
      mqtt.subscribe("ttt/lobby/announce");
      mqtt.subscribe("ttt/lobby/leave/#");
      mqtt.subscribe("ttt/lobby/invite/#");
      mqtt.subscribe("ttt/lobby/response/#");

      // Announce online (retained)
      mqtt.publish("ttt/sys/server", "{\"status\":\"online\"}", true);
      Serial.println("[MQTT] Subscriptions registered");
    } else {
      Serial.println("failed rc=" + String(mqtt.state()) + " retrying in
 ↪  3s");
      delay(3000);
    }
  }
}

//
//  Arduino entry points
//

void setup() {
  Serial.begin(115200);
  randomSeed(esp_random());
```

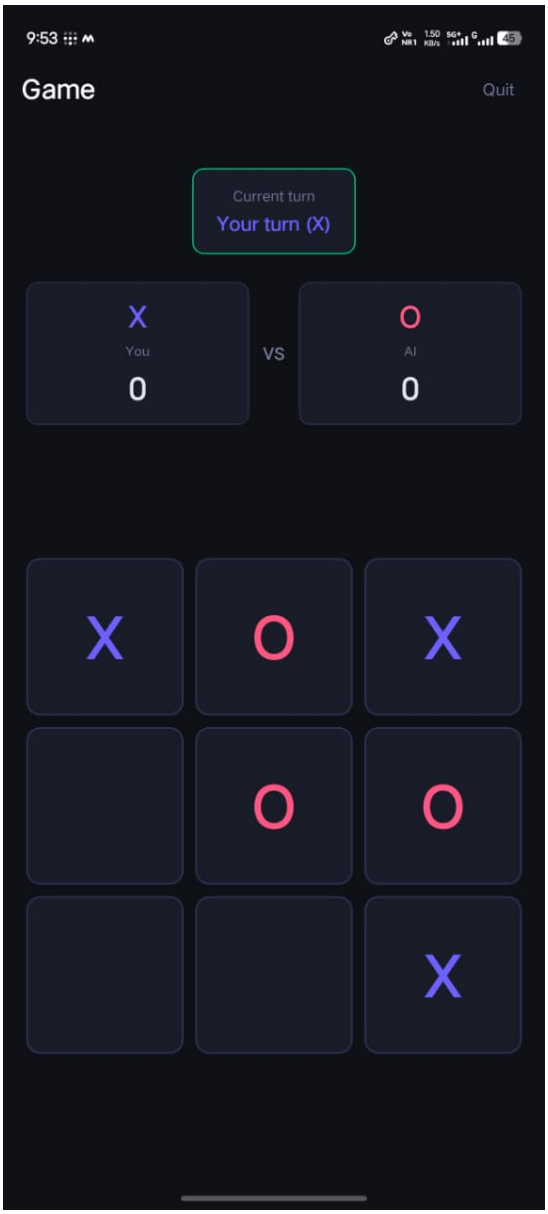
```
  connectWiFi();
  mqtt.setServer(MQTT_HOST, MQTT_PORT);
  mqtt.setCallback(onMessage);
  mqtt.setBufferSize(1024);
  connectMQTT();

  Serial.println("[TTT] Server ready");
}

void loop() {
  if (!mqtt.connected()) connectMQTT();
  mqtt.loop();

  // Heartbeat every 5 s
  if (millis() - lastHeartbeat >= 5000) {
    lastHeartbeat = millis();
    String hb = "{\"ts\":" + String(millis()) + ",\"sessions\":" +
                String(sessions.size()) + ",\"players\":" +
                String(players.size()) + "}";
    mqtt.publish("ttt/sys/heartbeat", hb.c_str());
  }
}
```

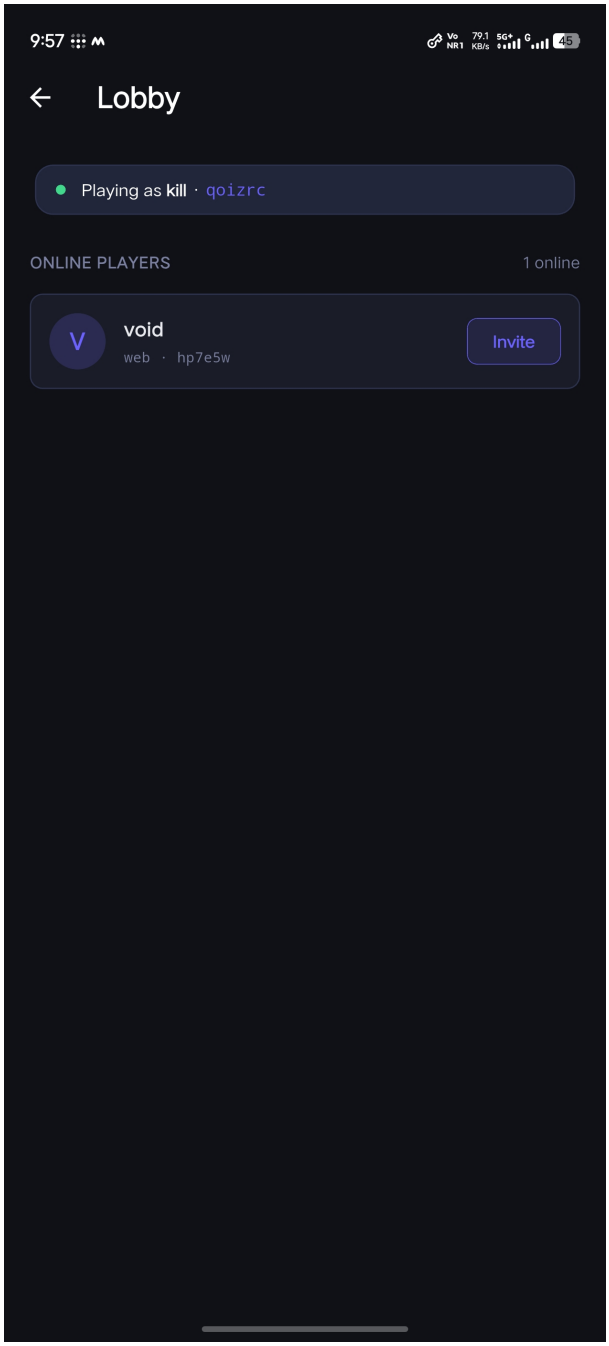
JSON

```
{
  "version": 1,
  "author": "TTT MQTT Game",
  "editor": "wokwi",
  "parts": [
    {
      "type": "wokwi-esp32-devkit-v1",
      "id": "esp",
      "top": 10,
      "left": 10,
      "attrs": {}
    }
  ],
  "connections": []
}
```

APP


9:53
TicTacToe
Powered by MQTT · ESP32
Connected to broker
Your display name
krill
5/20
Game mode
Play vs AI
Human vs Human
Start Game
Player ID: qoizrcop


9:53
Game
Quit
Current turn
Your turn (X)
X
You
0
VS
O
AI
0
X
O
X
O
O
X


9:57
Lobby
Playing as kill · qoizrc
ONLINE PLAYERS
1 online
V
void
web · hp7e5w
Invite